

# ESWIN

## ENNP开发者手册

---

Rev 1.0

2026-04-17

## 声明

### 知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

### 责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

| 文档版本     | 日期         | 说明                                                                              |
|----------|------------|---------------------------------------------------------------------------------|
| Rev 1.0  | 2026/04/17 | 重构文档结构                                                                          |
| Rev 0.95 | 2025/12/26 | AcceleratorKit 修改了 AK 相关接口参数为指针，新增 Canny、Rgb2Gray 接口。                           |
| Rev 0.94 | 2025/9/29  | AcceleratorKit 修改了 ES_AK_DSP_WarpAffine 接口参数和文档说明，新增 RGB2HSV、Sobel,DFT 等 DSP 算子 |
| Rev 0.93 | 2025/1/21  | npu runtime 增加从内存加载模型接口                                                         |
| Rev 0.92 | 2024/10/23 | npu runtime 增加复合模型相关接口                                                          |
| Rev 0.6  | 2024/02/29 | 初始版本                                                                            |

# 目录

|                                        |           |
|----------------------------------------|-----------|
| <b>声明</b>                              | <b>1</b>  |
| 知识产权声明                                 | 1         |
| 责任限制                                   | 1         |
| <b>修订记录</b>                            | <b>2</b>  |
| <b>1. 概述</b>                           | <b>9</b>  |
| 1.1. 异构计算核心概念                          | 9         |
| 1.2. 开发流程                              | 9         |
| 1.3. 同步与异步执行模型                         | 9         |
| <b>2. DSP Runtime 开发指南</b>             | <b>11</b> |
| 2.1. 功能描述                              | 11        |
| 2.1.1. 核心概念解析                          | 11        |
| 2.1.2. 模块参数配置                          | 11        |
| 2.1.3. API 类型                          | 11        |
| 2.2. 开发流程                              | 11        |
| 2.3. C API 详解                          | 12        |
| 2.3.1. 函数                              | 12        |
| 2.3.1.1. ES_DSP_GetVersion             | 12        |
| 2.3.1.2. ES_DSP_GetVersionString       | 12        |
| 2.3.1.3. ES_DSP_GetCapability          | 13        |
| 2.3.1.4. ES_DSP_SetLogLevel            | 13        |
| 2.3.1.5. ES_DSP_Open                   | 14        |
| 2.3.1.6. ES_DSP_Close                  | 14        |
| 2.3.1.7. ES_DSP_LoadOperator           | 15        |
| 2.3.1.8. ES_DSP_UnloadOperator         | 15        |
| 2.3.1.9. ES_DSP_SubmitTask             | 16        |
| 2.3.1.10. ES_DSP_SubmitTaskAsync       | 16        |
| 2.3.1.11. ES_DSP_QueryTask             | 17        |
| 2.3.1.12. ES_DSP_LL_GetVersion         | 17        |
| 2.3.1.13. ES_DSP_LL_GetVersionString   | 18        |
| 2.3.1.14. ES_DSP_LL_GetCapability      | 18        |
| 2.3.1.15. ES_DSP_LL_SetLogLevel        | 19        |
| 2.3.1.16. ES_DSP_LL_Open               | 19        |
| 2.3.1.17. ES_DSP_LL_Close              | 20        |
| 2.3.1.18. ES_DSP_LL_LoadOperator       | 20        |
| 2.3.1.19. ES_DSP_LL_UnloadOperator     | 21        |
| 2.3.1.20. ES_DSP_LL_PrepareDMABuffer   | 21        |
| 2.3.1.21. ES_DSP_LL_UnprepareDMABuffer | 22        |
| 2.3.1.22. ES_DSP_LL_SubmitTask         | 22        |
| 2.3.1.23. ES_DSP_LL_SubmitTaskAsync    | 23        |
| 2.3.1.24. ES_DSP_LL_ProcessReport      | 23        |
| 2.3.2. 数据类型和数据结构                       | 24        |
| 2.3.2.1. ES_DSP_Capability_S           | 24        |
| 2.3.2.2. ES_DSP_ID_E                   | 24        |
| 2.3.2.3. ES_DSP_PRI_E                  | 25        |
| 2.3.2.4. ES_DSP_HANDLE                 | 26        |
| 2.3.2.5. ES_DSP_TASK_CALLBACK          | 26        |
| 2.3.2.6. ES_DSP_TASK_S                 | 26        |
| 2.4. Python API 详解                     | 27        |
| 2.4.1. API 类型                          | 27        |
| 2.4.1.1. 类型一（High-level API）           | 27        |
| 2.4.1.2. 类型二（Low-level API）            | 27        |
| 2.4.2. 高级 API (类型一)                    | 28        |

|                                         |    |
|-----------------------------------------|----|
| 2.4.2.1. 获取版本号                          | 28 |
| 2.4.2.2. 获取版本字符串                        | 28 |
| 2.4.2.3. 获取能力信息                         | 28 |
| 2.4.2.4. 设置日志级别                         | 29 |
| 2.4.2.5. 打开 DSP 设备                      | 29 |
| 2.4.2.6. 关闭 DSP 设备                      | 30 |
| 2.4.2.7. 加载算子                           | 30 |
| 2.4.2.8. 卸载算子                           | 31 |
| 2.4.2.9. 同步提交任务                         | 31 |
| 2.4.2.10. 异步提交任务                        | 32 |
| 2.4.2.11. 查询任务状态                        | 33 |
| 2.4.3. 低级 API (类型二)                     | 33 |
| 2.4.3.1. 获取版本号                          | 33 |
| 2.4.3.2. 获取版本字符串                        | 34 |
| 2.4.3.3. 获取能力信息                         | 34 |
| 2.4.3.4. 设置日志级别                         | 34 |
| 2.4.3.5. 打开 DSP 设备                      | 35 |
| 2.4.3.6. 关闭 DSP 设备                      | 35 |
| 2.4.3.7. 加载算子                           | 36 |
| 2.4.3.8. 卸载算子                           | 36 |
| 2.4.3.9. 准备 DMA Buffer                  | 37 |
| 2.4.3.10. 释放 DMA Buffer                 | 37 |
| 2.4.3.11. 同步提交任务                        | 38 |
| 2.4.3.12. 异步提交任务                        | 38 |
| 2.4.3.13. 处理异步回调                        | 39 |
| 2.4.3.14. Low-level 调用流程示例              | 39 |
| 2.4.4. 数据类型和数据结构                        | 40 |
| 2.4.4.1. common.ES_DSP_Capability_S     | 41 |
| 2.4.4.2. common.ES_DSP_ID_E 枚举          | 41 |
| 2.4.4.3. common.ES_DSP_LOAD_POLICY_E 枚举 | 41 |
| 2.4.4.4. common.ES_DSP_PRI_E 枚举         | 42 |
| 2.4.4.5. 句柄值                            | 42 |
| 2.4.4.6. common.ES_DSP_TASK_S           | 42 |
| 2.4.4.7. DSP 任务回调                       | 43 |
| 2.4.4.8. common.ES_DEV_BUF_S            | 43 |
| 2.5. 错误码                                | 44 |
| 2.6. 调试信息                               | 45 |
| 2.6.1. Proc 调试信息                        | 45 |
| 2.6.2. Proc 信息说明                        | 45 |
| 2.7. 使用注意事项                             | 47 |

### 3. NPU Runtime 开发指南 48

|                                     |    |
|-------------------------------------|----|
| 3.1. 功能描述                           | 48 |
| 3.2. 开发流程                           | 48 |
| 3.3. C API 详解                       | 49 |
| 3.3.1. 函数                           | 49 |
| 3.3.1.1. ES_NPU_GetVersion          | 49 |
| 3.3.1.2. ES_NPU_GetVersionString    | 49 |
| 3.3.1.3. ES_NPU_GetCapability       | 50 |
| 3.3.1.4. ES_NPU_SetLogLevel         | 50 |
| 3.3.1.5. ES_NPU_GetDeviceProperties | 51 |
| 3.3.1.6. ES_NPU_GetNumDevices       | 51 |
| 3.3.1.7. ES_NPU_SetDevice           | 52 |
| 3.3.1.8. ES_NPU_ReleaseDevice       | 52 |
| 3.3.1.9. ES_NPU_GetDevice           | 53 |
| 3.3.1.10. ES_NPU_CreateContext      | 53 |
| 3.3.1.11. ES_NPU_SetCurrentContext  | 54 |

|           |                              |    |
|-----------|------------------------------|----|
| 3.3.1.12. | ES_NPU_GetCurrentContext     | 54 |
| 3.3.1.13. | ES_NPU_DestroyContext        | 55 |
| 3.3.1.14. | ES_NPU_CreateStream          | 55 |
| 3.3.1.15. | ES_NPU_DestroyStream         | 56 |
| 3.3.1.16. | ES_NPU_SynchronizeStream     | 56 |
| 3.3.1.17. | ES_NPU_AbortStream           | 56 |
| 3.3.1.18. | ES_NPU_LoadModelFromFile     | 57 |
| 3.3.1.19. | ES_NPU_LoadModelFromMemory   | 57 |
| 3.3.1.20. | ES_NPU_UnloadModel           | 58 |
| 3.3.1.21. | ES_NPU_LoadCompositeModel    | 58 |
| 3.3.1.22. | ES_NPU_GetCompositeModelInfo | 59 |
| 3.3.1.23. | ES_NPU_UnloadCompositeModel  | 60 |
| 3.3.1.24. | ES_NPU_GetNumInputTensors    | 60 |
| 3.3.1.25. | ES_NPU_GetNumOutputTensors   | 61 |
| 3.3.1.26. | ES_NPU_GetInputTensorDesc    | 61 |
| 3.3.1.27. | ES_NPU_GetOutputTensorDesc   | 62 |
| 3.3.1.28. | ES_NPU_Prepare               | 62 |
| 3.3.1.29. | ES_NPU_Unprepare             | 63 |
| 3.3.1.30. | ES_NPU_Submit                | 63 |
| 3.3.1.31. | ES_NPU_SubmitAsync           | 64 |
| 3.3.1.32. | ES_NPU_AllocTaskMemory       | 64 |
| 3.3.1.33. | ES_NPU_ReleaseTaskMemory     | 65 |
| 3.3.1.34. | ES_NPU_SetFlexibleTaskAttr   | 65 |
| 3.3.1.35. | ES_NPU_SubmitFlexibleTask    | 66 |
| 3.3.1.36. | ES_NPU_ProcessReport         | 67 |
| 3.3.1.37. | ES_NPU_ExecuteTaskSync       | 67 |
| 3.3.1.38. | ES_NPU_DumpOpStatus          | 68 |
| 3.3.1.39. | ES_NPU_GetDdrBandwidth       | 68 |
| 3.3.2.    | 数据类型和数据结构                    | 69 |
| 3.3.2.1.  | NPU_DIMS4_S                  | 69 |
| 3.3.2.2.  | NPU_TENSOR_S                 | 69 |
| 3.3.2.3.  | NPU_TASK_S                   | 70 |
| 3.3.2.4.  | NPU_TASK_MEM_S               | 70 |
| 3.3.2.5.  | NPU_MODEL_INFO_S             | 71 |
| 3.3.2.6.  | NPU_COMPOSITE_MODEL_INFO_S   | 71 |
| 3.3.2.7.  | NPU_FLEXIBLE_TASK_ATTR_S     | 72 |
| 3.3.2.8.  | NPU_TaskCallback             | 72 |
| 3.3.2.9.  | npu_context                  | 72 |
| 3.3.2.10. | npu_stream                   | 73 |
| 3.4.      | Python API 详解                | 73 |
| 3.4.1.    | 版本信息和系统配置                    | 73 |
| 3.4.1.1.  | 获取版本号                        | 73 |
| 3.4.1.2.  | 设置日志级别                       | 74 |
| 3.4.2.    | 设备管理                         | 74 |
| 3.4.2.1.  | 获取设备数量                       | 74 |
| 3.4.2.2.  | 设置当前设备                       | 75 |
| 3.4.2.3.  | 获取当前设备                       | 75 |
| 3.4.2.4.  | 获取设备属性                       | 75 |
| 3.4.2.5.  | 释放设备                         | 76 |
| 3.4.3.    | 上下文管理                        | 76 |
| 3.4.3.1.  | 创建 Context                   | 76 |
| 3.4.3.2.  | 设置当前 Context                 | 77 |
| 3.4.3.3.  | 获取当前 Context                 | 77 |
| 3.4.3.4.  | 销毁 Context                   | 78 |
| 3.4.4.    | 流管理                          | 78 |
| 3.4.4.1.  | 创建 Stream                    | 78 |

|                                             |           |
|---------------------------------------------|-----------|
| 3.4.4.2. 销毁 Stream                          | 79        |
| 3.4.4.3. 同步 Stream                          | 79        |
| 3.4.4.4. 中止 Stream                          | 80        |
| 3.4.5. 模型加载                                 | 80        |
| 3.4.5.1. 从文件加载模型                            | 80        |
| 3.4.5.2. 加载复合模型                             | 81        |
| 3.4.5.3. 卸载模型                               | 81        |
| 3.4.6. 张量信息查询                               | 82        |
| 3.4.6.1. 获取输入张量数量                           | 82        |
| 3.4.6.2. 获取输出张量数量                           | 82        |
| 3.4.6.3. 获取输入/输出张量信息                        | 82        |
| 3.4.7. 内存管理                                 | 83        |
| 3.4.7.1. 准备和释放 DMA Buffer                   | 83        |
| 3.4.7.2. 分配和释放任务内存                          | 84        |
| 3.4.8. 任务提交                                 | 85        |
| 3.4.8.1. 同步提交任务                             | 85        |
| 3.4.8.2. 异步提交任务                             | 86        |
| 3.4.8.3. 提交复合模型任务                           | 87        |
| 3.4.9. 调试和性能分析                              | 88        |
| 3.4.9.1. 启用 Dump 功能                         | 88        |
| 3.4.9.2. 禁用 Dump 功能                         | 88        |
| 3.4.9.3. 获取算子状态                             | 88        |
| 3.4.9.4. 启用性能监测                             | 89        |
| 3.4.9.5. 禁用性能监测                             | 89        |
| 3.4.9.6. 获取 DDR 带宽                          | 90        |
| 3.4.10. 数据类型和数据结构                           | 90        |
| 3.4.10.1. common.ES_NPU_Capability_S        | 90        |
| 3.4.10.2. common.NPU_DEVICE_PROP_S          | 91        |
| 3.4.10.3. common.NPU_DIMS4_S                | 91        |
| 3.4.10.4. common.NPU_TENSOR_S               | 91        |
| 3.4.10.5. common.NPU_TASK_S                 | 92        |
| 3.4.10.6. common.NPU_TASK_MEM_S             | 94        |
| 3.4.10.7. common.NPU_MODEL_INFO_S           | 94        |
| 3.4.10.8. common.NPU_COMPOSITE_MODEL_INFO_S | 94        |
| 3.4.10.9. common.NPU_FLEXIBLE_TASK_ATTR_S   | 94        |
| 3.4.10.10. common.ES_DEV_BUF_S              | 95        |
| 3.5. 错误码                                    | 95        |
| 3.6. ES_SDK_VERSION_U                       | 96        |
| 3.7. 调试信息                                   | 96        |
| 3.7.1. 日志等级                                 | 96        |
| 3.7.2. Python 调试示例                          | 97        |
| 3.8. 使用注意事项                                 | 97        |
| 3.8.1. 资源管理                                 | 97        |
| 3.8.2. 内存管理                                 | 97        |
| 3.8.3. 异步操作                                 | 98        |
| 3.8.4. 错误处理                                 | 98        |
| 3.8.5. 同步 vs 异步                             | 98        |
| <b>4. AcceleratorKit 开发指南</b>               | <b>99</b> |
| 4.1. 功能描述                                   | 99        |
| 4.2. C API 详解                               | 99        |
| 4.2.1. 函数                                   | 99        |
| 4.2.1.1. ES_AK_GetVersion                   | 99        |
| 4.2.1.2. ES_AK_GetVersionString             | 100       |
| 4.2.1.3. ES_AK_GetCapability                | 100       |
| 4.2.1.4. ES_AK_SetLogLevel                  | 101       |
| 4.2.1.5. ES_AK_Init                         | 101       |

|                                                   |     |
|---------------------------------------------------|-----|
| 4.2.1.6. ES_AK_Deinit . . . . .                   | 102 |
| 4.2.1.7. ES_AK_SetDevice . . . . .                | 102 |
| 4.2.1.8. ES_AK_GetDevice . . . . .                | 102 |
| 4.2.1.9. ES_AK_DSP_CosDistance . . . . .          | 103 |
| 4.2.1.10. ES_AK_DSP_PerspectiveAffine . . . . .   | 104 |
| 4.2.1.11. ES_AK_DSP_WarpAffine . . . . .          | 105 |
| 4.2.1.12. ES_AK_DSP_DetectionOut . . . . .        | 107 |
| 4.2.1.13. ES_AK_DSP_Argmax . . . . .              | 108 |
| 4.2.1.14. ES_AK_DSP_Softmax . . . . .             | 109 |
| 4.2.1.15. ES_AK_CPU_SimilarityTransform . . . . . | 109 |
| 4.2.1.16. ES_AK_DSP_RGB2HSV . . . . .             | 110 |
| 4.2.1.17. ES_AK_DSP_Sobel . . . . .               | 111 |
| 4.2.1.18. ES_AK_DSP_HoughLines . . . . .          | 112 |
| 4.2.1.19. ES_AK_DSP_GaussianBlur . . . . .        | 113 |
| 4.2.1.20. ES_AK_DSP_Dilate . . . . .              | 114 |
| 4.2.1.21. ES_AK_DSP_Erode . . . . .               | 114 |
| 4.2.1.22. ES_AK_DSP_DFT . . . . .                 | 115 |
| 4.2.1.23. ES_AK_DSP_Rgb2Gray . . . . .            | 116 |
| 4.2.1.24. ES_AK_DSP_Canny . . . . .               | 117 |
| 4.2.2. 数据类型和数据结构 . . . . .                        | 118 |
| 4.2.2.1. ES_AK_Capability_S . . . . .             | 118 |
| 4.2.2.2. ES_AK_DEVICE_E . . . . .                 | 118 |
| 4.2.2.3. ES_TENSOR_S . . . . .                    | 119 |
| 4.2.2.4. ES_DET_NETWORK_E . . . . .               | 120 |
| 4.2.2.5. ES_NMS_METHOD_E . . . . .                | 120 |
| 4.2.2.6. ES_IOU_METHOD_E . . . . .                | 120 |
| 4.2.2.7. ES_BOX_TYPE_E . . . . .                  | 121 |
| 4.2.2.8. ES_INTER_FLAG_E . . . . .                | 121 |
| 4.2.2.9. ES_BORDER_TYPES_E . . . . .              | 121 |
| 4.2.2.10. ES_DETECTION_OUT_CFG . . . . .          | 122 |
| 4.2.2.11. ES_RGB_CFG . . . . .                    | 123 |
| 4.2.2.12. ES_HOUGH_LINES_CFG . . . . .            | 123 |
| 4.2.2.13. ES_DILATE_ERODE_KERNEL_CFG . . . . .    | 124 |
| 4.2.2.14. ES_RGB_CONFIG . . . . .                 | 124 |
| 4.2.2.15. ES_CANNY_CONFIG . . . . .               | 125 |
| 4.3. Python API 详解 . . . . .                      | 125 |
| 4.3.1. 版本信息和系统配置 . . . . .                        | 125 |
| 4.3.1.1. 获取版本号 . . . . .                          | 125 |
| 4.3.1.2. 设置日志级别 . . . . .                         | 126 |
| 4.3.1.3. 初始化和释放 . . . . .                         | 126 |
| 4.3.2. 设备管理 . . . . .                             | 127 |
| 4.3.2.1. 配置计算设备 . . . . .                         | 127 |
| 4.3.3. DSP 加速算子 . . . . .                         | 128 |
| 4.3.3.1. 余弦距离计算 . . . . .                         | 128 |
| 4.3.3.2. 仿射变换 . . . . .                           | 129 |
| 4.3.3.3. 透视变换 . . . . .                           | 131 |
| 4.3.3.4. Softmax 激活 . . . . .                     | 131 |
| 4.3.3.5. Argmax 计算 . . . . .                      | 132 |
| 4.3.3.6. 检测输出后处理 . . . . .                        | 133 |
| 4.3.4. CPU 加速算子 . . . . .                         | 134 |
| 4.3.4.1. 相似度变换计算 . . . . .                        | 134 |
| 4.3.5. 数据类型和数据结构 . . . . .                        | 135 |
| 4.3.5.1. common.ES_AK_Capability_S . . . . .      | 135 |
| 4.3.5.2. common.ES_AK_DEVICE_E . . . . .          | 136 |
| 4.3.5.3. common.ES_TENSOR_S . . . . .             | 137 |
| 4.3.5.4. 数据类型枚举 . . . . .                         | 138 |



- 4.3.5.5. common.ES\_DET\_NETWORK\_E . . . . . 138
  - 4.3.5.6. common.ES\_NMS\_METHOD\_E . . . . . 139
  - 4.3.5.7. common.ES\_IOU\_METHOD\_E . . . . . 139
  - 4.3.5.8. common.ES\_BOX\_TYPE\_E . . . . . 139
  - 4.3.5.9. common.ES\_INTER\_FLAG\_E . . . . . 140
  - 4.3.5.10. common.ES\_BORDER\_TYPES\_E . . . . . 140
  - 4.3.5.11. common.ES\_DETECTION\_OUT\_CFG . . . . . 141
- 4.4. 错误码 . . . . . 142
- 4.5. 调试信息 . . . . . 143
  - 4.5.1. 常见日志提示 . . . . . 143
  - 4.5.2. 日志级别 . . . . . 143
  - 4.5.3. Python 调试示例 . . . . . 143
- 4.6. 使用注意事项 . . . . . 143
  - 4.6.1. 参数与 Tensor 配置 . . . . . 143
  - 4.6.2. 初始化和清理 . . . . . 144
  - 4.6.3. 内存管理 . . . . . 144
  - 4.6.4. 设备选择 . . . . . 144
  - 4.6.5. 错误处理 . . . . . 145
  - 4.6.6. 性能优化建议 . . . . . 145

## 1. 概述

本文档旨在为 EIC77 系列芯片的 ENNP(ESWIN Neural Network Processing) 异构计算平台提供全面的开发指导，通过详细阐述 NPU、DSP、HAE 等异构硬件的统一软件栈，明确 C/C++ 与 Python 双 API 体系的适用场景与核心对象模型（Device/Context/Stream），帮助开发者规范接口使用、掌握异构调度机制，从而高效完成底层系统开发或算法快速验证。

### 1.1. 异构计算核心概念

ENNP 平台通过统一的运行时（Runtime）管理多种硬件资源。理解以下核心对象是正确使用 API 的前提：

| 概念             | 描述                       | 关键作用                                                                           |
|----------------|--------------------------|--------------------------------------------------------------------------------|
| <b>Device</b>  | 物理硬件单元（如 NPU_0，DSP_1）。   | 计算发生的物理载体。通过 SetDevice 绑定，决定任务在哪块硬件上执行。                                        |
| <b>Context</b> | 异步任务的资源容器和管理单元。          | 管理 Stream 的生命周期。不同 Context 之间的任务调度相互隔离，类似于进程的概念。                               |
| <b>Stream</b>  | 任务执行队列。                  | Stream 用于维护异步操作，任务完成后等待用户取回的队列。同一 Stream 内的任务严格串行执行，不同 Stream 之间可以并行，类似于线程的概念。 |
| <b>Task</b>    | 最小的执行单元（如一次模型推理、一次算子计算）。 | 提交给 Stream，由 Runtime 调度到具体的 Device 执行。                                         |

### 1.2. 开发流程

无论使用哪种编程语言或哪个计算模块，典型的开发流程都遵循以下步骤：

- 初始化 (Initialization)：**配置日志等级、设置运行设备。
- 资源创建 (Resource Creation)：**
  - 若使用**异步接口**：必须创建 Context 和 Stream。
  - 若使用**同步接口**：通常无需手动创建 Context 和 Stream。
- 数据准备 (Data Preparation)：**加载模型或准备输入数据，并根据需要建立 DMA/SMMU 映射（C 接口需显式调用，Python 接口通常自动处理）。
- 任务提交 (Task Submission)：**
  - 同步**：调用接口后阻塞等待硬件执行完成。
  - 异步**：调用接口后立即返回，任务在后台执行。
- 资源释放 (Cleanup)：**按照与创建相反的顺序（Task → Stream → Context → Device）释放资源，防止内存泄漏。

### 1.3. 同步与异步执行模型

选择合适的执行模型对应用性能至关重要：

| 特性        | 同步 (Synchronous)           | 异步 (Asynchronous)          |
|-----------|----------------------------|----------------------------|
| <b>行为</b> | 调用 API 后，线程阻塞，直到硬件计算完成才返回。 | 调用 API 后立即返回，任务在后台队列中排队执行。 |
| <b>优点</b> | 代码逻辑简单，易于理解和调试。            | 吞吐量高，支持多任务并行，适合高并发场景。      |

| 特性     | 同步 (Synchronous)         | 异步 (Asynchronous)                                  |
|--------|--------------------------|----------------------------------------------------|
| 缺点     | 无法利用硬件并行能力，CPU 利用率低。     | 代码逻辑复杂，需要处理回调或轮询查询。                                |
| API 特征 | 函数名通常不含 Async 或 Submit 。 | 涉及 CreateStream , SubmitAsync , ProcessReport 等调用。 |
| 适用场景   | 简单的 Demo、对实时性要求不高的脚本。    | 高性能服务器、多模型流水线、实时推理应用。                              |

## 2. DSP Runtime 开发指南

DSP Runtime 专注于**可编程的硬件加速**，不同于 NPU 运行固定的神经网络模型，它允许开发者直接调用底层的数字信号处理算子，实现极致的性能与控制力。

### 2.1. 功能描述

#### 2.1.1. 核心概念解析

DSP Runtime 的运行机制围绕以下五个核心对象展开：

- **算子句柄 (Operator Handle)**：加载 DSP 算子库时，系统为每个算子实例分配的唯一标识符，用于区分不同的算子功能。
- **算子任务描述 (Task Description)**：提交计算任务时需填充的数据结构（详见 `ES_DSP_TASK_S`），包含输入/输出 Buffer、优先级等必要信息。
- **算子任务句柄 (Task Handle)**：异步提交任务后，系统返回的唯一任务标识符，用于追踪和管理特定任务的生命周期。
- **任务状态查询 (Query)**：针对**简易接口** API（详见 [API 类型](#)）提交的异步任务，用户需使用此 Handle 调用 `ES_DSP_QueryTask` 轮询任务是否完成。
- **任务回调处理 (Process Report)**：针对**专家接口** API（详见 [API 类型](#)）提交的异步任务，用户需调用 `ES_DSP_LL_ProcessReport` 来处理底层上报的事件，该接口会触发已注册任务的回调函数。

#### 2.1.2. 模块参数配置

**fw\_timeout**：DSP 固件任务超时时间（单位：秒）。

用户可根据实际业务场景配置合理的超时阈值。配置方法如下（DSP 驱动加载后生效）：

```
echo 10 > /sys/module/eic7700_dsp/parameters/fw_timeout
```

#### 2.1.3. API 类型

DSP 提供两种不同类型的 API，简易接口对底层接口进行了部分隐藏，接口使用较为简单；专家二接口暴露了更多底层功能，适用于专业用户。

| 特性                | 简易接口                                             | 专家接口                                                                                  |
|-------------------|--------------------------------------------------|---------------------------------------------------------------------------------------|
| <b>C API 前缀</b>   | <code>ES_DSP_*</code>                            | <code>ES_DSP_LL_*</code>                                                              |
| <b>Python API</b> | <code>es_py.ak.dsp_*</code>                      | <code>es_py.ak.dsp_ll_*</code>                                                        |
| <b>内存管理</b>       | <b>自动</b> 。Runtime 自动处理 DMA Buffer 的申请与 SMMU 映射。 | <b>手动</b> 。开发者需显式调用 <code>PrepareDMABuffer</code> 和 <code>UnprepareDMABuffer</code> 。 |
| <b>控制粒度</b>       | 粗。适合快速调用。                                        | 细。允许精确控制 Buffer 生命周期和底层回调。                                                            |
| <b>适用场景</b>       | 应用层开发、快速原型验证。                                    | 系统级集成、追求极致性能优化的场景。                                                                    |

#### 提示

用户不能混合使用两种类型的接口，以防止计算过程出错导致该进程内部逻辑出现问题。

### 2.2. 开发流程

无论使用哪种 API，DSP 算子的执行通常遵循以下步骤：

1. 初始化：

调用 `ES_DSP_Init` 或 `ES_DSP_Open` 初始化 DSP 设备环境。
2. 加载算子：

调用 `LoadOperator` 加载所需的 DSP 库（如 `libsoftmax.so`）。
3. 准备数据：

• (类型一) 直接传入 Host 内存地址。

• (类型二) 调用 `PrepareDMABuffer` 建立 SMMU 映射。
4. 提交任务：

• 同步：调用 `SubmitTask`，等待 DSP 计算完成。

• 异步：调用 `SubmitTaskAsync`，立即返回任务句柄 (Task Handle)。
5. 查询与回调：

• (类型一) 调用 `QueryTask` 轮询任务状态。

• (类型二) 调用 `ProcessReport` 触发回调函数。
6. 资源释放：

调用 `UnloadOperator` 和 `Close/Deinit`。

## 2.3. C API 详解

### 2.3.1. 函数

#### 2.3.1.1. ES\_DSP\_GetVersion

【函数体】

```
ES_S32 ES_DSP_GetVersion(ES_U64 *version)
```

【描述】

获取 DSP 数字版本信息。

【参数】

| 参数      | 描述      | 输入/输出                               |
|---------|---------|-------------------------------------|
| version | DSP 版本号 | 输出，参见 <code>ES_SDK_VERSION_U</code> |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

#### 2.3.1.2. ES\_DSP\_GetVersionString

【函数体】

```
ES_S32 ES_DSP_GetVersionString(  
    ES_CHAR *version,  
    ES_U32 maxSize)
```

【描述】

获取 DSP 字符串版本信息。

【参数】

| 参数      | 描述            | 输入/输出 |
|---------|---------------|-------|
| version | DSP 版本号       | 输出    |
| maxSize | DSP 版本字符串最大长度 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.3. ES\_DSP\_GetCapability

【函数体】

```
ES_S32 ES_DSP_GetCapability(ES_DSP_Capability_S *capability)
```

【描述】

获取 DSP 能力信息。

【参数】

| 参数         | 描述       | 输入/输出                     |
|------------|----------|---------------------------|
| capability | DSP 能力信息 | 输出，参见 ES_DSP_Capability_S |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.4. ES\_DSP\_SetLogLevel

【函数体】

```
ES_S32 ES_DSP_SetLogLevel(ES_U32 level)
```

【描述】

设置 DSP 运行时的打印日志级别。

【参数】

| 参数    | 描述         | 输入/输出 |
|-------|------------|-------|
| level | DSP 打印日志等级 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.5. ES\_DSP\_Open

【函数体】

```
ES_S32 ES_DSP_Open(  
    ES_DSP_ID_E dspId,  
    ES_DSP_LOAD_POLICY_E opLoadPolicy)
```

【描述】

根据 DSP ID 号打开 DSP 设备。

【参数】

| 参数           | 描述         | 输入/输出                      |
|--------------|------------|----------------------------|
| dspId        | DSP ID 序号  | 输入，参见 ES_DSP_ID_E          |
| opLoadPolicy | DSP 算子加载策略 | 输入，参见 ES_DSP_LOAD_POLICY_E |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.6. ES\_DSP\_Close

【函数体】

```
ES_S32 ES_DSP_Close(ES_DSP_ID_E dspId)
```

【描述】

根据 DSP ID 号关闭 DSP 设备。

【参数】

| 参数    | 描述        | 输入/输出             |
|-------|-----------|-------------------|
| dspld | DSP ID 序号 | 输入，参见 ES_DSP_ID_E |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.7. ES\_DSP\_LoadOperator

【函数体】

```
ES_S32 ES_DSP_LoadOperator(  
    ES_DSP_ID_E dspld,  
    const ES_CHAR* operatorName,  
    const ES_CHAR* operatorLibDir,  
    ES_DSP_HANDLE* operatorHandle)
```

【描述】

加载指定名称和库路径的算子到指定的 DSP 设备，并返回算子句柄。

【参数】

| 参数             | 描述        | 输入/输出             |
|----------------|-----------|-------------------|
| dspld          | DSP ID 序号 | 输入，参见 ES_DSP_ID_E |
| operatorName   | DSP 算子名   | 输入                |
| operatorLibDir | DSP 算子库目录 | 输入                |
| operatorHandle | DSP 算子句柄  | 输出                |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.8. ES\_DSP\_UnloadOperator

【函数体】

```
ES_S32 ES_DSP_UnloadOperator(  
    ES_DSP_ID_E dspld,  
    const ES_DSP_HANDLE operatorHandle)
```

【描述】



在指定的 DSP 设备通过已加载的算子句柄卸载该算子。

【参数】

| 参数             | 描述        | 输入/输出 |
|----------------|-----------|-------|
| dspId          | DSP ID 序号 | 输入    |
| operatorHandle | DSP 算子句柄  | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.9. ES\_DSP\_SubmitTask

【函数体】

```
ES_S32 ES_DSP_SubmitTask(  
    ES_DSP_ID_E dspId,  
    ES_DSP_TASK_S* operatorTask)
```

【描述】

同步提交算子任务到指定的 DSP 设备，接口等待 DSP 完成本次计算，并返回即代表任务完成。

【参数】

| 参数           | 描述          | 输入/输出               |
|--------------|-------------|---------------------|
| dspId        | DSP ID 序号   | 输入                  |
| operatorTask | DSP 算子任务消息体 | 输入，参见 ES_DSP_TASK_S |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.10. ES\_DSP\_SubmitTaskAsync

【函数体】

```
ES_S32 ES_DSP_SubmitTaskAsync(  
    ES_DSP_ID_E dspId,  
    ES_DSP_TASK_S* operatorTask,  
    ES_DSP_HANDLE* taskHandle)
```

【描述】

异步提交算子任务到指定的 DSP 设备，并返回算子任务句柄。需要使用 ES\_DSP\_QueryTask 查询任务是否完成。

【参数】

| 参数           | 描述          | 输入/输出               |
|--------------|-------------|---------------------|
| dspId        | DSP ID 序号   | 输入                  |
| operatorTask | DSP 算子任务消息体 | 输入，参见 ES_DSP_TASK_S |
| taskHandle   | DSP 算子任务句柄  | 输出                  |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.11. ES\_DSP\_QueryTask

【函数体】

```
ES_S32 ES_DSP_QueryTask(  
    ES_DSP_ID_E dspId,  
    ES_DSP_HANDLE taskHandle,  
    ES_BOOL block,  
    ES_BOOL* taskFinish)
```

【描述】

根据算子任务句柄查询该算子在指定的 DSP 设备是否完成运算。

【参数】

| 参数         | 描述             | 输入/输出 |
|------------|----------------|-------|
| dspId      | DSP ID 序号      | 输入    |
| taskHandle | DSP 算子任务句柄     | 输入    |
| block      | 是否阻塞查询         | 输入    |
| taskFinish | DSP 算子任务完成状态指针 | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.12. ES\_DSP\_LL\_GetVersion

【函数体】

```
ES_S32 ES_DSP_LL_GetVersion(ES_U64 *version)
```

【描述】

获取 DSP 数字版本信息。

【参数】

| 参数      | 描述      | 输入/输出                  |
|---------|---------|------------------------|
| version | DSP 版本号 | 输出，参见 ES_SDK_VERSION_U |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.13. ES\_DSP\_LL\_GetVersionString

【函数体】

```
ES_S32 ES_DSP_LL_GetVersionString(  
    ES_CHAR* version,  
    ES_U32 maxSize)
```

【描述】

获取 DSP 字符串版本信息。

【参数】

| 参数      | 描述            | 输入/输出 |
|---------|---------------|-------|
| version | DSP 版本号       | 输出    |
| maxSize | DSP 版本字符串最大长度 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.14. ES\_DSP\_LL\_GetCapability

【函数体】

ES\_S32 ES\_DSP\_LL\_GetCapability(ES\_DSP\_Capability\_S \*capability)

【描述】

获取 DSP 能力信息。

【参数】

| 参数         | 描述       | 输入/输出                     |
|------------|----------|---------------------------|
| capability | DSP 能力信息 | 输出，参见 ES_DSP_Capability_S |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.15. ES\_DSP\_LL\_SetLogLevel

【函数体】

ES\_S32 ES\_DSP\_LL\_SetLogLevel(ES\_U32 level)

【描述】

设置 DSP 运行时的打印日志级别。

【参数】

| 参数    | 描述         | 输入/输出 |
|-------|------------|-------|
| level | DSP 打印日志等级 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.16. ES\_DSP\_LL\_Open

【函数体】

ES\_S32 ES\_DSP\_LL\_Open(  
ES\_DSP\_ID\_E dspId,  
ES\_S32 \*dspFd)

【描述】

根据 DSP ID 号打开 DSP 设备。

【参数】

| 参数    | 描述        | 输入/输出             |
|-------|-----------|-------------------|
| dspId | DSP ID 序号 | 输入，参见 ES_DSP_ID_E |
| dspFd | DSP 设备描述符 | 输出                |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.17. ES\_DSP\_LL\_Close

【函数体】

```
ES_S32 ES_DSP_LL_Close(ES_S32 dspFd)
```

【描述】

根据 DSP ID 号关闭 DSP 设备。

【参数】

| 参数    | 描述        | 输入/输出 |
|-------|-----------|-------|
| dspFd | DSP 设备描述符 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.18. ES\_DSP\_LL\_LoadOperator

【函数体】

```
ES_S32 ES_DSP_LL_LoadOperator(  
    ES_S32 dspFd,  
    const ES_CHAR *operatorName,  
    const ES_CHAR *operatorLibDir,  
    ES_DSP_HANDLE *operatorHandle)
```

【描述】

加载指定名称和库路径的算子到指定的 DSP 设备，并返回算子句柄。

【参数】

| 参数             | 描述        | 输入/输出 |
|----------------|-----------|-------|
| dspFd          | DSP 设备描述符 | 输入    |
| operatorName   | DSP 算子名   | 输入    |
| operatorLibDir | DSP 算子库目录 | 输入    |
| operatorHandle | DSP 算子句柄  | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.19. ES\_DSP\_LL\_UnloadOperator

【函数体】

```
ES_S32 ES_DSP_LL_UnloadOperator(  
    ES_S32 dspFd,  
    const ES_DSP_HANDLE operatorHandle)
```

【描述】

在指定的 DSP 设备通过已加载的算子句柄卸载该算子。

【参数】

| 参数             | 描述        | 输入/输出 |
|----------------|-----------|-------|
| dspFd          | DSP 设备描述符 | 输入    |
| operatorHandle | DSP 算子句柄  | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.20. ES\_DSP\_LL\_PrepareDMABuffer

【函数体】

```
ES_S32 ES_DSP_LL_PrepareDMABuffer(  
    ES_S32 dspFd,  
    ES_DEV_BUF_S buffer)
```

【描述】

为算子任务所需的 DMA Buffer 提前建立在该 DSP 设备上的 SMMU 映射，这样可以加速后续的数据执行过程。

【参数】

| 参数     | 描述         | 输入/输出 |
|--------|------------|-------|
| dspFd  | DSP 设备描述符  | 输入    |
| buffer | DMA Buffer | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.21. ES\_DSP\_LL\_UnprepareDMABuffer

【函数体】

```
ES_S32 ES_DSP_LL_UnprepareDMABuffer(  
    ES_S32 dspFd,  
    ES_U64 fd)
```

【描述】

根据申请的 DMA Buffer 对应的 fd 释放在该 DSP 上的设备 SMMU 映射。需要与 ES\_DSP\_LL\_PrepereDMABuffer 成对使用，并且在不需要对该 Buffer 再次执行该 DSP 设备上任务时调用。

【参数】

| 参数    | 描述             | 输入/输出 |
|-------|----------------|-------|
| dspFd | DSP 设备描述符      | 输入    |
| fd    | DMA Buffer 描述符 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.22. ES\_DSP\_LL\_SubmitTask

【函数体】

```
ES_S32 ES_DSP_LL_SubmitTask(  
    ES_S32 dspFd,  
    ES_DSP_TASK_S *operatorTask)
```

【描述】

提交同步算子任务到指定的 DSP 设备。

【参数】

| 参数           | 描述          | 输入/输出               |
|--------------|-------------|---------------------|
| dspFd        | DSP 设备描述符   | 输入                  |
| operatorTask | DSP 算子任务消息体 | 输入，参见 ES_DSP_TASK_S |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.23. ES\_DSP\_LL\_SubmitTaskAsync

【函数体】

```
ES_S32 ES_DSP_LL_SubmitTaskAsync(  
    ES_S32 dspFd,  
    ES_DSP_TASK_S *operatorTask)
```

【描述】

提交异步算子任务到指定的 DSP 设备。异步任务需要使用 ES\_DSP\_LL\_ProcessReport 查询并等待执行任务完成，并在完成时调用回调函数。

【参数】

| 参数           | 描述          | 输入/输出               |
|--------------|-------------|---------------------|
| dspFd        | DSP 设备描述符   | 输入                  |
| operatorTask | DSP 算子任务消息体 | 输入，参见 ES_DSP_TASK_S |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.1.24. ES\_DSP\_LL\_ProcessReport



【函数体】

```
ES_S32 ES_DSP_LL_ProcessReport(
    ES_S32 dspFd,
    ES_S32 timeout)
```

【描述】

执行异步算子任务提交时注册的回调函数。

【参数】

| 参数      | 描述                                        | 输入/输出 |
|---------|-------------------------------------------|-------|
| dspFd   | DSP 设备描述符                                 | 输入    |
| timeout | 单位：ms 没有任务完成时，等待任务完成的最大时间，-1 则阻塞等待直到有任务完成 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

2.3.2. 数据类型和数据结构

2.3.2.1. ES\_DSP\_Capability\_S

【描述】

DSP 能力信息。

【定义】

```
typedef struct DSP_Capability_S {
    ES_U64 reserved;
} ES_DSP_Capability_S;
```

【成员】

| 成员       | 描述   |
|----------|------|
| reserved | 预留字段 |

2.3.2.2. ES\_DSP\_ID\_E

【描述】

DSP ID。

【定义】

```
typedef enum DSP_ID_E {
    ES_DSP_ID_0 = 0x0,
    ES_DSP_ID_1 = 0x1,
    ES_DSP_ID_2 = 0x2,
    ES_DSP_ID_3 = 0x3,
    ES_DSP_ID_4 = 0x4,
    ES_DSP_ID_5 = 0x5,
    ES_DSP_ID_6 = 0x6,
    ES_DSP_ID_7 = 0x7,
    ES_DSP_ID_BUTT
} ES_DSP_ID_E;
```

【成员】

| 成员             | 描述          |
|----------------|-------------|
| ES_DSP_ID_0    | DSP ID 0    |
| ES_DSP_ID_1    | DSP ID 1    |
| ES_DSP_ID_2    | DSP ID 2    |
| ES_DSP_ID_3    | DSP ID 3    |
| ES_DSP_ID_4    | DSP ID 4    |
| ES_DSP_ID_5    | DSP ID 5    |
| ES_DSP_ID_6    | DSP ID 6    |
| ES_DSP_ID_7    | DSP ID 7    |
| ES_DSP_ID_BUTT | DSP ID 枚举个数 |

2.3.2.3. ES\_DSP\_PRI\_E

【描述】

DSP 任务优先级。同一个 DSP 设备会根据优先级调度排队的任务。

【定义】

```
typedef enum DSP_PRI_E {
    ES_DSP_PRI_0 = 0x0,
    ES_DSP_PRI_1 = 0x1,
    ES_DSP_PRI_2 = 0x2,
    ES_DSP_PRI_3 = 0x3,
    ES_DSP_PRI_BUTT
} ES_DSP_PRI_E;
```

【成员】

| 成员              | 描述       |
|-----------------|----------|
| ES_DSP_PRI_0    | 优先级 0 最高 |
| ES_DSP_PRI_1    | 优先级 1    |
| ES_DSP_PRI_2    | 优先级 2    |
| ES_DSP_PRI_3    | 优先级 3    |
| ES_DSP_PRI_BUTT | 优先级枚举个数  |

2.3.2.4. ES\_DSP\_HANDLE

【描述】

DSP 任务句柄。

【定义】

```
typedef ES_U64 ES_DSP_HANDLE;
```

2.3.2.5. ES\_DSP\_TASK\_CALLBACK

【描述】

DSP 算子任务回调函数。

【定义】

```
typedef void (*ES_DSP_TASK_CALLBACK)(void *arg, ES_S32 state);
```

2.3.2.6. ES\_DSP\_TASK\_S

【描述】

DSP 算子任务描述信息。

【定义】

```
typedef struct DSP_TASK_S {
    ES_DSP_HANDLE operatorHandle;
    ES_DEV_BUF_S dspBuffers[BUFFER_CNT_MAXSIZE];
    ES_U32 bufferCntCfg;
    ES_U32 bufferCntInput;
    ES_U32 bufferCntOutput;
    ES_DSP_PRI_E priority;
    ES_BOOL syncCache;
    ES_BOOL pollMode;
    ES_DSP_HANDLE taskHandle;
    ES_DSP_TASK_CALLBACK callback;
    ES_VOID *cbArg;
} ES_DSP_TASK_S;
```

【成员】

| 成员              | 描述              |
|-----------------|-----------------|
| operatorHandle  | DSP 已加载的算子句柄    |
| dspBuffers      | DSP 算子将使用的设备缓冲区 |
| bufferCntCfg    | 算子配置信息的缓冲区总数    |
| bufferCntInput  | 算子输入信息的缓冲区总数    |
| bufferCntOutput | 算子输出信息的缓冲区总数    |
| priority        | 算子任务优先级         |
| syncCache       | 驱动程序负责主机端缓存同步   |

| 成员         | 描述                |
|------------|-------------------|
| pollMode   | DSP 使用 polling 模式 |
| taskHandle | 算子任务提交后返回的句柄      |
| callback   | 任务完成后需要调用的回调函数    |
| cbArg      | 回调函数的参数           |

## 2.4. Python API 详解

### 2.4.1. API 类型

DSP 提供两种不同类型的 API，其中类型一接口对底层接口进行了部分隐藏，接口使用较为简单；类型二接口暴露了更多底层功能，适用于专业用户。用户不能混合使用两种类型的接口，以防止计算过程出错导致该进程内部逻辑出现问题。

#### 2.4.1.1. 类型一（High-level API）

- `dsp_get_version()`：获取 DSP Runtime 接口数字信息版本
- `dsp_get_version_string(maxSize)`：获取 DSP Runtime 接口字符串版本
- `dsp_get_capability()`：获取 DSP 能力信息
- `dsp_set_log_level(level)`：设置 DSP 打印日志等级
- `dsp_open(dsp_id, op_load_policy)`：打开 DSP 设备
- `dsp_close(dsp_id)`：关闭 DSP 设备
- `dsp_load_operator(dsp_id, operator_name, operator_lib_dir)`：加载 DSP 算子
- `dsp_unload_operator(dsp_id, operator_handle)`：卸载 DSP 算子
- `dsp_submit_task(dsp_id, operator_task)`：同步执行 DSP 算子任务
- `dsp_submit_task_async(dsp_id, operator_task)`：异步执行 DSP 算子任务
- `dsp_query_task(dsp_id, task_handle, block)`：查询 DSP 算子任务是否完成

#### 2.4.1.2. 类型二（Low-level API）

- `dsp_ll_get_version()`：获取 DSP Runtime 接口数字信息版本
- `dsp_ll_get_version_string(maxSize)`：获取 DSP Runtime 接口字符串版本
- `dsp_ll_get_capability()`：获取 DSP 能力信息
- `dsp_ll_set_log_level(level)`：设置 DSP 打印日志等级
- `dsp_ll_open(dsp_id)`：打开 DSP 设备，获取设备 fd
- `dsp_ll_close(dsp_fd)`：根据设备 fd 关闭 DSP 设备
- `dsp_ll_load_operator(dsp_fd, operator_name, operator_lib_dir)`：加载算子，获取算子句柄
- `dsp_ll_unload_operator(dsp_fd, operator_handle)`：根据算子句柄卸载算子
- `dsp_ll_prepare_dma_buffer(dsp_fd, buffer)`：为算子数据申请 DMA Buffer 并建立 SMMU 映射
- `dsp_ll_unprepare_dma_buffer(dsp_fd, fd)`：释放申请的 DMA Buffer 及 SMMU 映射
- `dsp_ll_submit_task(dsp_fd, operator_task)`：同步方式执行算子任务
- `dsp_ll_submit_task_async(dsp_fd, operator_task)`：异步方式执行算子任务
- `dsp_ll_process_report(dsp_fd, timeout)`：执行异步算子任务提交时注册的回调函数

2.4.2. 高级 API (类型一)

2.4.2.1. 获取版本号

```
from es_py import ak, common

# 获取 DSP 版本号 (数值格式)
version = ak.dsp_get_version()
```

【参数】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

int：DSP 版本号 (数值格式)

【异常】

失败时抛出 Exception 异常

2.4.2.2. 获取版本字符串

```
# 获取 DSP 版本字符串
version_string = ak.dsp_get_version_string(max_size=256)
```

【参数】

| 参数       | 类型  | 描述        | 输入/输出 |
|----------|-----|-----------|-------|
| max_size | int | 版本字符串最大长度 | 输入    |

【返回值】

str：DSP 版本字符串

【异常】

失败时抛出 Exception 异常

2.4.2.3. 获取能力信息

```
from es_py import ak, common

# 获取 DSP 能力信息
capability = ak.dsp_get_capability()
print(capability.reserved)
```

【参数】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

`common.ES_DSP_Capability_S`：DSP 能力信息对象

【说明】

该接口返回 `common.ES_DSP_Capability_S` 类型。调用前需要导入 `common` 模块完成返回类型注册，推荐统一使用 `from es_py import ak, common`。

【异常】

失败时抛出 `Exception` 异常

2.4.2.4. 设置日志级别

```
# 设置 DSP 日志级别
ak.dsp_set_log_level(level=2)
```

【参数】

| 参数    | 类型  | 描述                           | 输入/输出 |
|-------|-----|------------------------------|-------|
| level | int | 日志级别：0=无日志, 1=错误, 2=警告, 3=信息 | 输入    |

【返回值】

`int`：0 表示成功

【异常】

失败时抛出 `Exception` 异常

【说明】

- 用于控制 DSP 运行时的日志输出详细程度
- 调试时建议使用 level=3（信息级别）

2.4.2.5. 打开 DSP 设备

```
from es_py import ak, common

# 打开 DSP 设备
ak.dsp_open(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
            op_load_policy=common.ES_DSP_LOAD_POLICY_E.ES_LOAD_CACHED)
```

【参数】

| 参数     | 类型                              | 描述        | 输入/输出 |
|--------|---------------------------------|-----------|-------|
| dsp_id | <code>common.ES_DSP_ID_E</code> | DSP 设备 ID | 输入    |

| 参数             | 类型                          | 描述         | 输入/输出 |
|----------------|-----------------------------|------------|-------|
| op_load_policy | common.ES_DSP_LOAD_POLICY_E | DSP 算子加载策略 | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- 使用 DSP 前必须先打开设备
- 需要与 ak.dsp\_close() 成对使用

2.4.2.6. 关闭 DSP 设备

```
# 关闭 DSP 设备
ak.dsp_close(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0)
```

【参数】

| 参数     | 类型                 | 描述        | 输入/输出 |
|--------|--------------------|-----------|-------|
| dsp_id | common.ES_DSP_ID_E | DSP 设备 ID | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- 关闭前需确保所有任务已完成
- 关闭前需卸载所有已加载的算子

2.4.2.7. 加载算子

```
# 加载 DSP 算子
operator_handle = ak.dsp_load_operator(
    dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
    operator_name="softmax",
    operator_lib_dir="/path/to/actual/dsp/operator/lib"
)
```

【参数】

| 参数     | 类型                 | 描述        | 输入/输出 |
|--------|--------------------|-----------|-------|
| dsp_id | common.ES_DSP_ID_E | DSP 设备 ID | 输入    |

| 参数               | 类型  | 描述    | 输入/输出 |
|------------------|-----|-------|-------|
| operator_name    | str | 算子名称  | 输入    |
| operator_lib_dir | str | 算子库路径 | 输入    |

【返回值】

int：算子句柄

【异常】

失败时抛出 Exception 异常

【说明】

- 需要将 operator\_lib\_dir 替换为部署环境提供的真实 DSP 算子库目录。
- 上述路径仅为占位路径，不是当前板卡验证过的默认路径。
- 返回的算子句柄用于后续任务提交
- 需要与 ak.dsp\_unload\_operator() 成对使用

2.4.2.8. 卸载算子

```
# 卸载 DSP 算子
ak.dsp_unload_operator(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0, operator_handle=operator_handle)
```

【参数】

| 参数              | 类型                 | 描述        | 输入/输出 |
|-----------------|--------------------|-----------|-------|
| dsp_id          | common.ES_DSP_ID_E | DSP 设备 ID | 输入    |
| operator_handle | int                | 算子句柄      | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

卸载前需确保使用该算子的所有任务已完成

2.4.2.9. 同步提交任务

```
# 创建任务描述。buffer 必须与已加载算子要求匹配。
task = common.ES_DSP_TASK_S()
task.operatorHandle = operator_handle
task.bufferCntCfg = 0
task.bufferCntInput = 1
task.bufferCntOutput = 1
task.priority = common.ES_DSP_PRI_E.ES_DSP_PRI_0
task.syncCache = True
task.pollMode = False
```



```
task.set_dspBuffer(0, input_buffer)
task.set_dspBuffer(1, output_buffer)

# 同步提交任务
result = ak.dsp_submit_task(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0, operator_task=task)
```

【参数】

| 参数     | 类型                   | 描述        | 输入/输出 |
|--------|----------------------|-----------|-------|
| dsp_id | common.ES_DSP_ID_E   | DSP 设备 ID | 输入    |
| task   | common.ES_DSP_TASK_S | 任务描述信息    | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- operator\_handle、input\_buffer、output\_buffer 必须来自已成功加载的 DSP 算子和有效的 common.ES\_DEV\_BUF\_S 缓冲区。
- 任务 buffer 数量和顺序依赖具体算子，提交任务前需参考算子自身说明。
- 同步执行，接口阻塞直到任务完成
- 返回即代表任务已完成

2.4.2.10. 异步提交任务

```
# 异步提交任务
ret, task_handle = ak.dsp_submit_task_async(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
↪ operator_task=task)
```

【参数】

| 参数     | 类型                   | 描述        | 输入/输出 |
|--------|----------------------|-----------|-------|
| dsp_id | common.ES_DSP_ID_E   | DSP 设备 ID | 输入    |
| task   | common.ES_DSP_TASK_S | 任务描述信息    | 输入    |

【返回值】

(ret, task\_handle)：返回状态码和任务句柄

【异常】

失败时抛出 Exception 异常

【说明】

- 异步执行，接口立即返回
- 返回的任务句柄用于查询任务状态

2.4.2.11. 查询任务状态

```
# 查询任务完成状态
ret, finished = ak.dsp_query_task(
    dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
    task_handle=task_handle,
    block=True
)
```

【参数】

| 参数          | 类型                 | 描述        | 输入/输出 |
|-------------|--------------------|-----------|-------|
| dsp_id      | common.ES_DSP_ID_E | DSP 设备 ID | 输入    |
| task_handle | int                | 任务句柄      | 输入    |
| block       | bool               | 是否阻塞查询    | 输入    |

【返回值】

(ret, finished)：返回状态码和任务是否完成（True=已完成，False=未完成）

【异常】

失败时抛出 Exception 异常

【说明】

- block=True 时，接口阻塞直到任务完成
- block=False 时，接口立即返回当前任务状态

2.4.3. 低级 API (类型二)

Low-level API 暴露了更多底层功能，适合专业用户使用。提供了细粒度的控制，包括 DMA Buffer 管理和异步任务回调处理。

2.4.3.1. 获取版本号

```
from es_py import ak, common

# 获取 DSP 版本号（数值格式）
version = ak.dsp_ll_get_version()
```

【参数】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

int：DSP 版本号（数值格式）

【异常】

失败时抛出 Exception 异常

2.4.3.2. 获取版本字符串

```
# 获取 DSP 版本字符串
version_string = ak.dsp_ll_get_version_string(max_size=256)
```

【参数】

| 参数       | 类型  | 描述        | 输入/输出 |
|----------|-----|-----------|-------|
| max_size | int | 版本字符串最大长度 | 输入    |

【返回值】

str：DSP 版本字符串

【异常】

- 失败时抛出 Exception 异常

2.4.3.3. 获取能力信息

```
from es_py import ak, common

# 获取 DSP 能力信息
capability = ak.dsp_ll_get_capability()
print(capability.reserved)
```

【参数】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

common.ES\_DSP\_Capability\_S：DSP 能力信息对象

【说明】

该接口返回 common.ES\_DSP\_Capability\_S 类型。调用前需要导入 common 模块完成返回类型注册，推荐统一使用 from es\_py import ak, common。

【异常】

失败时抛出 Exception 异常

2.4.3.4. 设置日志级别

```
# 设置 DSP 日志级别
ak.dsp_ll_set_log_level(level=3)
```

【参数】

| 参数    | 类型  | 描述                           | 输入/输出 |
|-------|-----|------------------------------|-------|
| level | int | 日志级别：0=无日志, 1=错误, 2=警告, 3=信息 | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 `Exception` 异常

2.4.3.5. 打开 DSP 设备

```
from es_py import ak, common

# 打开 DSP 设备并获取 fd
ret, dsp_fd = ak.dsp_ll_open(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0)
```

【参数】

| 参数     | 类型                 | 描述        | 输入/输出 |
|--------|--------------------|-----------|-------|
| dsp_id | common.ES_DSP_ID_E | DSP 设备 ID | 输入    |

【返回值】

(ret, dsp\_fd)：返回状态码和 DSP 设备文件描述符

【异常】

失败时抛出 `Exception` 异常

【说明】

- 返回的 fd 用于后续所有 Low-level API 调用
- 需要与 `ak.dsp_ll_close()` 成对使用

2.4.3.6. 关闭 DSP 设备

```
# 关闭 DSP 设备
ak.dsp_ll_close(dsp_fd=dsp_fd)
```

【参数】

| 参数     | 类型  | 描述          | 输入/输出 |
|--------|-----|-------------|-------|
| dsp_fd | int | DSP 设备文件描述符 | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 `Exception` 异常

【说明】

- 关闭前需确保所有任务已完成
- 关闭前需卸载所有已加载的算子

2.4.3.7. 加载算子

```
# 加载算子，获取算子句柄
ret, operator_handle = ak.dsp_ll_load_operator(
    dsp_fd=dsp_fd,
    operator_name="softmax",
    operator_lib_dir="/path/to/actual/dsp/operator/lib"
)
```

【参数】

| 参数               | 类型  | 描述          | 输入/输出 |
|------------------|-----|-------------|-------|
| dsp_fd           | int | DSP 设备文件描述符 | 输入    |
| operator_name    | str | 算子名称        | 输入    |
| operator_lib_dir | str | 算子库路径       | 输入    |

【返回值】

(ret, operator\_handle)：返回状态码和算子句柄

【异常】

失败时抛出 `Exception` 异常

【说明】

- 需要将 `operator_lib_dir` 替换为部署环境提供的真实 DSP 算子库目录。
- 上述路径仅为占位路径，不是当前板卡验证过的默认路径。
- 返回的算子句柄用于后续任务提交
- 需要与 `ak.dsp_ll_unload_operator()` 成对使用

2.4.3.8. 卸载算子

```
# 根据算子句柄卸载算子
ak.dsp_ll_unload_operator(dsp_fd=dsp_fd, operator_handle=operator_handle)
```

【参数】

| 参数              | 类型  | 描述          | 输入/输出 |
|-----------------|-----|-------------|-------|
| dsp_fd          | int | DSP 设备文件描述符 | 输入    |
| operator_handle | int | 算子句柄        | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

卸载前需确保使用该算子的所有任务已完成

2.4.3.9. 准备 DMA Buffer

```
# 为算子数据申请 DMA Buffer 并建立 SMMU 映射
ak.dsp_ll_prepare_dma_buffer(dsp_fd=dsp_fd, buffer=input_buffer)
```

【参数】

| 参数     | 类型                  | 描述          | 输入/输出 |
|--------|---------------------|-------------|-------|
| dsp_fd | int                 | DSP 设备文件描述符 | 输入    |
| buffer | common.ES_DEV_BUF_S | 设备缓冲区对象     | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- 在提交任务前需要为输入输出数据准备 DMA Buffer
- 建立 SMMU 映射，使 DSP 可以访问该内存
- 需要与 ak.dsp\_ll\_unprepare\_dma\_buffer() 成对使用

2.4.3.10. 释放 DMA Buffer

```
# 释放申请的 DMA Buffer 及 SMMU 映射
ak.dsp_ll_unprepare_dma_buffer(dsp_fd=dsp_fd, fd=buffer_fd)
```

【参数】

| 参数     | 类型  | 描述           | 输入/输出 |
|--------|-----|--------------|-------|
| dsp_fd | int | DSP 设备文件描述符  | 输入    |
| fd     | int | Buffer 文件描述符 | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

释放前需确保使用该 Buffer 的所有任务已完成

2.4.3.11. 同步提交任务

```
# 同步方式执行算子任务。buffer 必须与已加载算子要求匹配。
task = common.ES_DSP_TASK_S()
task.operatorHandle = operator_handle
task.bufferCntCfg = 0
task.bufferCntInput = 1
task.bufferCntOutput = 1
task.priority = common.ES_DSP_PRI_E.ES_DSP_PRI_0
task.syncCache = True
task.pollMode = False
task.set_dspBuffer(0, input_buffer)
task.set_dspBuffer(1, output_buffer)

ak.dsp_ll_submit_task(dsp_fd=dsp_fd, operator_task=task)
```

【参数】

| 参数     | 类型                   | 描述          | 输入/输出 |
|--------|----------------------|-------------|-------|
| dsp_fd | int                  | DSP 设备文件描述符 | 输入    |
| task   | common.ES_DSP_TASK_S | 任务描述信息      | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- operator\_handle、input\_buffer、output\_buffer 必须来自已成功加载的 DSP 算子和有效的 common.ES\_DEV\_BUF\_S 缓冲区。
- 任务 buffer 数量和顺序依赖具体算子，提交任务前需参考算子自身说明。
- 同步执行，接口阻塞直到任务完成

2.4.3.12. 异步提交任务

```
# 异步方式执行算子任务
task.set_callback_py(my_callback_function, custom_data)

ak.dsp_ll_submit_task_async(dsp_fd=dsp_fd, operator_task=task)
```

【参数】

| 参数     | 类型  | 描述          | 输入/输出 |
|--------|-----|-------------|-------|
| dsp_fd | int | DSP 设备文件描述符 | 输入    |

| 参数   | 类型                   | 描述                   | 输入/输出 |
|------|----------------------|----------------------|-------|
| task | common.ES_DSP_TASK_S | 任务描述信息（需设置 callback） | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- 异步执行，接口立即返回
- 任务完成后会调用注册的回调函数
- 需要调用 ak.dsp\_ll\_process\_report() 处理回调

2.4.3.13. 处理异步回调

```
# 处理异步任务回调
ak.dsp_ll_process_report(dsp_fd=dsp_fd, timeout=1000)
```

【参数】

| 参数      | 类型  | 描述          | 输入/输出 |
|---------|-----|-------------|-------|
| dsp_fd  | int | DSP 设备文件描述符 | 输入    |
| timeout | int | 超时时间（毫秒）    | 输入    |

【返回值】

int：0 表示成功

【异常】

失败时抛出 Exception 异常

【说明】

- 等待并执行异步任务提交时注册的回调函数
- timeout 为-1 时表示无限等待
- timeout 为 0 时表示非阻塞查询

2.4.3.14. Low-level 调用流程示例

```
from es_py import ak, common

# 定义回调函数。当前 Python 绑定通过 set_callback_py(callback, arg) 传入用户参数。
def task_complete_callback(user_data=None):
    print(f"任务完成，用户数据: {user_data}")
    return 0

try:
    # 1. 打开 DSP 设备
    ret, dsp_fd = ak.dsp_ll_open(dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0)
```



```

# 2. 加载算子。operator_lib_dir 需要替换为真实 DSP 算子库目录。
ret, operator_handle = ak.dsp_ll_load_operator(
    dsp_fd=dsp_fd,
    operator_name="softmax",
    operator_lib_dir="/path/to/actual/dsp/operator/lib"
)

# 3. 准备 DMA Buffer。input_buffer/output_buffer 需为有效 common.ES_DEV_BUF_S。
ak.dsp_ll_prepare_dma_buffer(dsp_fd=dsp_fd, buffer=input_buffer)
ak.dsp_ll_prepare_dma_buffer(dsp_fd=dsp_fd, buffer=output_buffer)

# 4. 创建并提交异步任务
task = common.ES_DSP_TASK_S()
task.operatorHandle = operator_handle
task.bufferCntCfg = 0
task.bufferCntInput = 1
task.bufferCntOutput = 1
task.priority = common.ES_DSP_PRI_E.ES_DSP_PRI_0
task.syncCache = True
task.pollMode = False
task.set_dspBuffer(0, input_buffer)
task.set_dspBuffer(1, output_buffer)
task.set_callback_py(task_complete_callback, {"task_id": 123})

ak.dsp_ll_submit_task_async(dsp_fd=dsp_fd, operator_task=task)

# 5. 处理异步回调
ak.dsp_ll_process_report(dsp_fd=dsp_fd, timeout=5000)

# 6. 清理资源
ak.dsp_ll_unprepare_dma_buffer(dsp_fd=dsp_fd, fd=input_buffer.memFd)
ak.dsp_ll_unprepare_dma_buffer(dsp_fd=dsp_fd, fd=output_buffer.memFd)
ak.dsp_ll_unload_operator(dsp_fd=dsp_fd, operator_handle=operator_handle)
ak.dsp_ll_close(dsp_fd=dsp_fd)

except Exception as e:
    print(f"DSP 错误: {type(e).__name__}: {e}")

```

#### 【说明】

- 该示例用于说明 Low-level API 调用顺序，不是当前板卡可直接复制运行的完整样例。
- operator\_lib\_dir、operator\_name、input\_buffer、output\_buffer 必须与实际部署的 DSP 算子库和算子说明匹配。
- 当前板卡环境未找到可用于 Python ak.dsp\_ll\_load\_operator() 的真实 DSP 算子库目录，因此完整 load\_operator -> submit\_task 链路未作为文档回归项。

#### 2.4.4. 数据类型和数据结构

DSP Runtime 相关数据类型、数据结构由 es\_py.common 模块导出；示例代码应通过 from es\_py import ak, common 使用以下类型。

- common.ES\_DSP\_Capability\_S：DSP 能力信息数据结构
- common.ES\_DSP\_ID\_E：DSP ID 枚举值
- common.ES\_DSP\_LOAD\_POLICY\_E：DSP 算子加载策略枚举值
- common.ES\_DSP\_PRI\_E：DSP 算子任务优先级枚举值
- DSP 算子/任务句柄：当前 Python 中为 int
- common.ES\_DSP\_TASK\_S：DSP 算子任务消息格式

- `common.ES_DEV_BUF_S`：设备缓冲区结构

2.4.4.1. `common.ES_DSP_Capability_S`

定义 DSP 能力信息。

```
from es_py import ak, common

capability = ak.dsp_get_capability()
print(capability)
print(capability.reserved)
```

【说明】

`ak.dsp_get_capability()` 和 `ak.dsp_ll_get_capability()` 返回 `common.ES_DSP_Capability_S`。调用前需要导入 `common`，否则当前 wheel 中可能因返回类型尚未注册而如下错误：

`TypeError: Unable to convert function return value to a Python type。`

2.4.4.2. `common.ES_DSP_ID_E` 枚举

定义 DSP ID 枚举值，用于标识不同的 DSP 设备。

| 枚举值                                            | 描述          |
|------------------------------------------------|-------------|
| <code>common.ES_DSP_ID_E.ES_DSP_ID_0</code>    | DSP 设备 0    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_1</code>    | DSP 设备 1    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_2</code>    | DSP 设备 2    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_3</code>    | DSP 设备 3    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_4</code>    | DSP 设备 4    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_5</code>    | DSP 设备 5    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_6</code>    | DSP 设备 6    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_7</code>    | DSP 设备 7    |
| <code>common.ES_DSP_ID_E.ES_DSP_ID_BUTT</code> | DSP ID 枚举边界 |

【使用示例】

```
from es_py import ak, common

ak.dsp_open(
    dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
    op_load_policy=common.ES_DSP_LOAD_POLICY_E.ES_LOAD_CACHED,
)
```

2.4.4.3. `common.ES_DSP_LOAD_POLICY_E` 枚举

定义 DSP 算子加载策略。

| 枚举值                                        | 描述          |
|--------------------------------------------|-------------|
| common.ES_DSP_LOAD_POLICY.E.ES_LOAD_CACHED | cached 加载策略 |

2.4.4.4. common.ES\_DSP\_PRI\_E 枚举

定义 DSP 任务优先级。同一个 DSP 设备会根据优先级调度排队的任务，优先级 0 最高。

| 枚举值                                 | 描述          |
|-------------------------------------|-------------|
| common.ES_DSP_PRI_E.ES_DSP_PRI_0    | 优先级 0，最高优先级 |
| common.ES_DSP_PRI_E.ES_DSP_PRI_1    | 优先级 1       |
| common.ES_DSP_PRI_E.ES_DSP_PRI_2    | 优先级 2       |
| common.ES_DSP_PRI_E.ES_DSP_PRI_3    | 优先级 3       |
| common.ES_DSP_PRI_E.ES_DSP_PRI_BUTT | 优先级枚举边界     |

2.4.4.5. 句柄值

DSP 算子句柄和任务句柄在当前 Python 绑定中均为 int。

```
operator_handle: int
task_handle: int
```

【说明】

- 用于标识加载的算子或提交的任务。
- 由系统自动分配，用户不应修改。
- 用于后续操作，例如卸载算子、查询任务等。

2.4.4.6. common.ES\_DSP\_TASK\_S

定义 DSP 算子任务描述信息。板卡实测该类型可用属性和方法包括 operatorHandle、bufferCntCfg、bufferCntInput、bufferCntOutput、priority、syncCache、pollMode、taskHandle、reserved、set\_dspBuffer()、get\_dspBuffer()、set\_callback\_py()、clear\_callback\_py()、get\_c\_task()。

| 成员/方法           | 类型                  | 描述                                                               | 输入/输出 |
|-----------------|---------------------|------------------------------------------------------------------|-------|
| operatorHandle  | int                 | 已加载的算子句柄,由 ak.dsp_load_operator() 或 ak.dsp_ll_load_operator() 返回 | 输入    |
| bufferCntCfg    | int                 | 配置缓冲区数量                                                          | 输入    |
| bufferCntInput  | int                 | 输入缓冲区数量                                                          | 输入    |
| bufferCntOutput | int                 | 输出缓冲区数量                                                          | 输入    |
| priority        | common.ES_DSP_PRI_E | 任务优先级，优先级 0 最高                                                   | 输入    |
| syncCache       | bool                | 是否由驱动同步缓存                                                        | 输入    |
| pollMode        | bool                | 是否使用 polling 模式                                                  | 输入    |

| 成员/方法                          | 类型  | 描述                              | 输入/输出 |
|--------------------------------|-----|---------------------------------|-------|
| taskHandle                     | int | 任务句柄                            | 输出    |
| set_dspBuffer(index, buffer)   | 方法  | 设置任务使用的 common.ES_DEV_BUF_S 缓冲区 | 输入    |
| get_dspBuffer(index)           | 方法  | 获取指定索引的缓冲区                      | 输出    |
| set_callback_py(callback, arg) | 方法  | 设置 Python 异步回调和用户参数             | 输入    |
| clear_callback_py()            | 方法  | 清理 Python 异步回调                  | 输入    |

【使用示例】

```
from es_py import ak, common

# 该示例展示 task 构造模式，依赖真实加载的 DSP 算子和该算子要求的 buffer。
task = common.ES_DSP_TASK_S()
task.operatorHandle = operator_handle
task.bufferCntCfg = 1
task.bufferCntInput = 1
task.bufferCntOutput = 1
task.priority = common.ES_DSP_PRI_E.ES_DSP_PRI_0
task.syncCache = True
task.pollMode = False
task.set_dspBuffer(0, config_buffer)
task.set_dspBuffer(1, input_buffer)
task.set_dspBuffer(2, output_buffer)

ak.dsp_submit_task(
    dsp_id=common.ES_DSP_ID_E.ES_DSP_ID_0,
    operator_task=task,
)
```

【说明】

- 该示例不是独立可运行样例；buffer 数量和顺序需与具体 DSP 算子要求一致。
- 通过 set\_dspBuffer(index, buffer) 设置 buffer，bufferCntCfg + bufferCntInput + bufferCntOutput 应与实际设置的 buffer 数量匹配。

2.4.4.7. DSP 任务回调

异步任务回调通过 common.ES\_DSP\_TASK\_S.set\_callback\_py(callback, arg) 设置。

```
def on_task_complete(task_handle, user_arg):
    print(f"任务完成，句柄: {task_handle}, 参数: {user_arg}")

task = common.ES_DSP_TASK_S()
task.set_callback_py(on_task_complete, {"name": "image_processing"})

ak.dsp_ll_submit_task_async(dsp_fd=dsp_fd, operator_task=task)
ak.dsp_ll_process_report(dsp_fd=dsp_fd, timeout=5000)

task.clear_callback_py()
```

2.4.4.8. common.ES\_DEV\_BUF\_S

定义设备缓冲区结构，用于在主机和 DSP 之间传递数据。板卡实测该结构字段如下。

| 字段      | 类型  | 描述          |
|---------|-----|-------------|
| memFd   | int | 缓冲区文件描述符    |
| offset  | int | 缓冲区偏移       |
| size    | int | 缓冲区大小，单位：字节 |
| reserve | int | 预留字段        |

【使用示例】

```
input_buffer = common.ES_DEV_BUF_S()
input_buffer.memFd = input_mem_fd
input_buffer.offset = 0
input_buffer.size = input_size
input_buffer.reserve = 0

ak.dsp_ll_prepare_dma_buffer(dsp_fd=dsp_fd, buffer=input_buffer)

# ... 提交使用该 buffer 的任务 ...

ak.dsp_ll_unprepare_dma_buffer(dsp_fd=dsp_fd, fd=input_buffer.memFd)
```

【说明】

- 使用 Low-level API 时需要手动准备和释放 DMA Buffer。
- ES\_DSP\_TASK\_S 中的 buffer 通过 set\_dspBuffer(index, buffer) 写入。
- 缓冲区地址对齐要求取决于具体算子。

2.5. 错误码

| 错误码                        | 值         | 描述                    |
|----------------------------|-----------|-----------------------|
| ES_DSP_SUCCESS             | 0x0       | 操作执行成功                |
| ES_DSP_ERROR_INVALID_DEVID | 0x2030001 | 设备 ID 超出合法范围          |
| ES_DSP_ERROR_INVALID_CHNID | 0x2030002 | 通道组号错误或无效区域句柄         |
| ES_DSP_ERROR_ILLEGAL_PARAM | 0x2030003 | 参数超出合法范围              |
| ES_DSP_ERROR_EXIST         | 0x2030004 | 重复创建已存在的设备、通道或资源      |
| ES_DSP_ERROR_UNEXIST       | 0x2030005 | 试图使用或者销毁不存在的设备、通道或者资源 |
| ES_DSP_ERROR_NULL_PTR      | 0x2030006 | 函数参数中有空指针             |
| ES_DSP_ERROR_NOT_CONFIG    | 0x2030007 | 模块没有配置                |
| ES_DSP_ERROR_NOT_SUPPORT   | 0x2030008 | 不支持的参数或者功能            |
| ES_DSP_ERROR_NOT_PERM      | 0x2030009 | 该操作不允许，如试图修改静态配置参数    |
| ES_DSP_ERROR_NOMEM         | 0x203000C | 分配内存失败，如系统内存不足        |
| ES_DSP_ERROR_NOBUF         | 0x203000D | 分配缓存失败，如申请的图像缓冲区太大    |
| ES_DSP_ERROR_BUF_EMPTY     | 0x203000E | 缓冲区中无图像               |
| ES_DSP_ERROR_BUF_FULL      | 0x203000F | 缓冲区中图像满               |
| ES_DSP_ERROR_NOTREADY      | 0x2030010 | 系统没有初始化或没有加载相应模块      |

| 错误码                        | 值         | 描述         |
|----------------------------|-----------|------------|
| ES_DSP_ERROR_BADADDR       | 0x2030011 | 地址非法       |
| ES_DSP_ERROR_BUSY          | 0x2030012 | 系统忙        |
| ES_DSP_ERROR_SYS_TIMEOUT   | 0x2030040 | 系统超时       |
| ES_DSP_ERROR_QUERY_TIMEOUT | 0x2030041 | Query 查询超时 |
| ES_DSP_ERROR_OPEN_FILE     | 0x2030042 | 打开文件失败     |
| ES_DSP_ERROR_READ_FILE     | 0x2030043 | 读文件失败      |

## 2.6. 调试信息

### 2.6.1. Proc 调试信息

调试信息采用了 Linux 下的 proc 文件系统，可实时反映当前系统的运行状态，所记录的信息可供问题定位及分析时使用。

- 文件目录，/proc/esdsp/info
- 信息查看方法：在控制台上可以使用 cat 命令查看信息，例如：cat /proc/esdsp/info

也可以使用其他常用的文件操作命令，例如：cp /proc/esdsp/info ./

将文件拷贝到当前目录，在应用程序中可以将上述文件当作普通只读文件进行读操作，例如 fopen、fread 等。

### 2.6.2. Proc 信息说明

【调试信息】

```
# cat /proc/esdsp/info
-----DSP PARAM INFO-----
Dield CoreId Enable CmdTOut(s)
0 0 Yes 1000000
0 1 NO 0
0 2 NO 1000000
1 0 NO 0
1 1 NO 0
1 2 NO 0

-----DSP RUNTIME INFO-----
Dield CoreId TotalIntCnt FinishedTaskCnt FailedTaskCnt PendingTaskCnt LTaskRunTm
0 0 32 31 0 0 67780000
0 1 0 0 0 0 0
0 2 0 0 0 0 0
1 0 0 0 0 0 0
1 1 0 0 0 0 0

-----DSP HW PERF INFO-----
Dield CoreId TaskName StartTm PrepSTm PrepETm EvalSTm EvalETm IPCSTm EndTm
0 0 argmax_f16f32 9081856 9081876 9081956 9081976 9091856 9092856 9181856
0 1 NULL 0 0 0 0 0 0 0
0 2 NULL 0 0 0 0 0 0 0
0 3 NULL 0 0 0 0 0 0 0
1 0 NULL 0 0 0 0 0 0 0
1 1 NULL 0 0 0 0 0 0 0
1 2 NULL 0 0 0 0 0 0 0
1 3 NULL 0 0 0 0 0 0 0

-----DSP INVOKE INFO-----
Dield CoreId Pri TaskName TaskHnd TaskStat TaskRunTm
```

```

0    0    0 softmax 7fe069c0 eval  78900000
0    1    0 NULL    0    NULL    0
0    2    0 NULL    0    NULL    0
1    0    0 NULL    0    NULL    0
1    1    0 NULL    0    NULL    0
1    2    0 NULL    0    NULL    0

```

### 【调试信息分析】

记录当前 DSP 工作状态资源信息，主要包括 DSP 队列状态信息，任务状态信息，运行时状态信息和调用信息。

### 【参数说明】

| 模块               | 参数              | 描述                        |
|------------------|-----------------|---------------------------|
| DSP PARAM INFO   | DieId           | Die ID                    |
|                  | CoreId          | DSP 设备 ID                 |
|                  | Enable          | DSP 使能状态：0 未使能，1 使能       |
|                  | TaskTmOut       | 任务执行超时时间。单位：s             |
| DSP RUNTIME INFO | DieId           | Die ID                    |
|                  | CoreId          | DSP 设备 ID                 |
|                  | TotalIntCnt     | DSP 产生中断的总次数              |
|                  | LTaskRunTm      | 最后一个任务执行时间，单位：us          |
|                  | FinishedTaskCnt | 成功任务总数                    |
|                  | FailedTaskCnt   | 失败任务总数                    |
|                  | PendingTaskCnt  | 待执行任务列表                   |
| DSP HW PERF INFO | DieId           | Die ID                    |
|                  | CoreId          | DSP 设备 ID                 |
|                  | Pri             | 任务优先级                     |
|                  | TaskName        | 正在运行的任务名称                 |
|                  | StartTm         | 算子开始时间，单位：ns              |
|                  | PrepSTm         | 任务 prepare 开始时间，单位：ns     |
|                  | PrepETm         | 任务 prepare 结束时间，单位：ns     |
|                  | EvalSTm         | 任务 eval 开始时间，单位：ns        |
|                  | EvalETm         | 任务 eval 结束时间，单位：ns        |
|                  | IPCSTm          | DSP 通知驱动时间，单位：ns          |
|                  | EndTm           | 算子结束时间，单位：ns              |
| DSP INVOKE INFO  | DieId           | Die ID                    |
|                  | CoreId          | DSP 设备 ID                 |
|                  | Pri             | 任务优先级                     |
|                  | TaskName        | 正在运行的任务名称                 |
|                  | TaskHnd         | 正在运行的任务 handle            |
|                  | TaskStat        | 正在运行任务的状态，prepare 或者 eval |
|                  | TaskRunTm       | 任务已运行时间，单位：us             |

## 2.7. 使用注意事项

以下操作在 C/C++ 与 Python 接口中均需成对出现：

- 设备生命周期：ES\_DSP\_Open()/ ES\_DSP\_Close(), open()/ close()
- 专家接口设备生命周期：ES\_DSP\_LL\_Open()/ ES\_DSP\_LL\_Close(), ll\_open()/ ll\_close()
- 算子生命周期：ES\_DSP\_LoadOperator()/ ES\_DSP\_UnloadOperator(), load\_operator()/ unload\_operator()
- 专家接口算子生命周期：ES\_DSP\_LL\_LoadOperator()/ ES\_DSP\_LL\_UnloadOperator(), ll\_load\_operator()/ ll\_unload\_operator()
- DMA Buffer 生命周期：ES\_DSP\_LL\_PrepareDMABuffer()/ ES\_DSP\_LL\_UnprepareDMABuffer(), ll\_prepare\_dma\_buffer()/ ll\_unprepare\_dma\_buffer()

否则会出现不可预期的错误。



### 3. NPU Runtime 开发指南

NPU Runtime 是 ENNP 提供的一套加载 NN 模型并执行推理任务的运行时系统和接口，可以直接加载基于 ENNP 工具量化编译的离线模型，用于实现目标识别、图像分类等功能。用户基于 NPU Runtime 开发智能分析方案，由编译器自动分析并优化执行过程，并自动生成优化好的离线模型，最大化复用 NPU、DSP、HAE 以及 GPU 等硬件，提升硬件利用率并优化系统功耗。

#### 3.1. 功能描述

NPU Runtime 是整个异构计算的核心，主要负责神经网络模型的加载、设备管理、任务调度以及执行。

NPU Runtime 软件栈如下图所示：

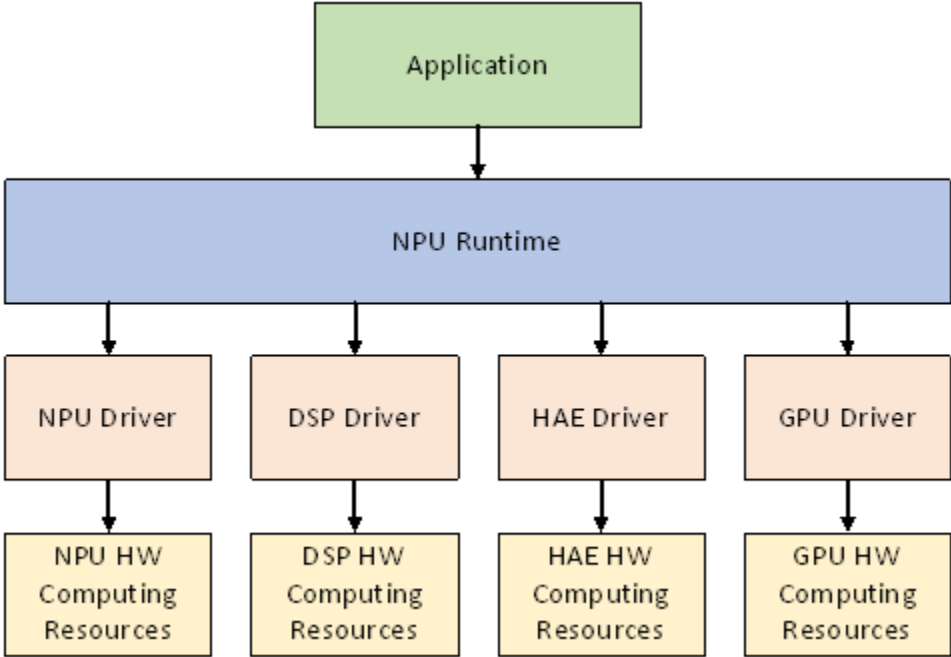


图 1: npu runtime 软件栈

#### 3.2. 开发流程

一个标准的 NPU 推理应用遵循以下生命周期：

1. **设备初始化：**调用 `ES_NPU_SetDevice` 绑定物理设备。
2. **资源创建 (异步必备)：**
  - 调用 `ES_NPU_CreateContext` 创建上下文。
  - 调用 `ES_NPU_CreateStream` 创建任务队列。
3. **模型管理：**
  - 调用 `ES_NPU_LoadModelFromFile` 或 `LoadModelFromMemory` 加载离线模型。
  - 获取 Tensor 描述符（`ES_NPU_GetInputTensorDesc`）。
4. **数据准备：**
  - 调用 `ES_NPU_Prepare` 建立 DMA/SMMU 映射（专家模式常用）。
  - 将数据拷贝或映射到输入 Tensor。
5. **提交执行：**

- 填充 `NPU_TASK_S` 结构体。
  - 调用 `Submit` 或 `SubmitAsync`。
6. 资源释放：
- 调用 `ES_NPU_UnloadModel` 卸载模型。
  - 逆序释放资源：`Stream` → `Context` → `Device`。

3.3. C API 详解

3.3.1. 函数

3.3.1.1. ES\_NPU\_GetVersion

【函数体】

```
ES_S32 ES_NPU_GetVersion(ES_U64 *version)
```

【描述】

获取 NPU runtime 软件栈版本信息。

【参数】

| 参数      | 描述          | 输入/输出                                  |
|---------|-------------|----------------------------------------|
| version | Runtime 版本号 | 输出，参见 <a href="#">ES_SDK_VERSION_U</a> |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.2. ES\_NPU\_GetVersionString

【函数体】

```
ES_S32 ES_NPU_GetVersionString(  
    ES_CHAR *version,  
    ES_U32 maxSize)
```

【描述】

获取字符格式的 NPU runtime 软件栈版本信息。

【参数】

| 参数      | 描述                 | 输入/输出 |
|---------|--------------------|-------|
| version | Runtime 版本号字符串     | 输出    |
| maxSize | Runtime 版本号字符串最大大小 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.3. ES\_NPU\_GetCapability

【函数体】

```
ES_S32 ES_NPU_GetCapability(ES_NPU_Capability_S *capability)
```

【描述】

获取 NPU 能力信息。

【参数】

| 参数         | 描述              | 输入/输出 |
|------------|-----------------|-------|
| capability | 存储 NPU 能力信息的结构体 | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.4. ES\_NPU\_SetLogLevel

【函数体】

```
ES_S32 ES_NPU_SetLogLevel(ES_U32 level)
```

【描述】

设置 NPU 运行时的打印日志级别。

【参数】

| 参数    | 描述         | 输入/输出 |
|-------|------------|-------|
| level | NPU 打印日志等级 | 输入    |

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.5. ES\_NPU\_GetDeviceProperties

【函数体】

```
ES_S32 ES_NPU_GetDeviceProperties(  
    ES_U16 deviceId,  
    NPU_DEVICE_PROP_S* prop)
```

【描述】

查询指定 deviceId 的设备属性信息。

【参数】

| 参数       | 描述              | 输入/输出 |
|----------|-----------------|-------|
| deviceId | 要查询设备的 deviceId | 输入    |
| prop     | 设备的属性信息         | 输出    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.6. ES\_NPU\_GetNumDevices

【函数体】

```
ES_S32 ES_NPU_GetNumDevices(ES_U16 *deviceNum)
```

【描述】

获取当前环境中可用的 device 数量。

【参数】

| 参数        | 描述            | 输入/输出 |
|-----------|---------------|-------|
| deviceNum | 获得的 device 数量 | 输出    |

【返回值】

|     |    |
|-----|----|
| 返回值 | 描述 |
| 0   | 成功 |

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.7. ES\_NPU\_SetDevice

【函数体】

ES\_S32 ES\_NPU\_SetDevice(ES\_U16 deviceId)

【描述】

设置用于运算的 device。

【参数】

|          |             |       |
|----------|-------------|-------|
| 参数       | 描述          | 输入/输出 |
| deviceId | 当前 deviceId | 输入    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

deviceId 应小于通过 ES\_NPU\_GetNumDevices 获取得到的 deviceNum。

3.3.1.8. ES\_NPU\_ReleaseDevice

【函数体】

ES\_S32 ES\_NPU\_ReleaseDevice(ES\_U16 deviceId)

【描述】

释放当前用于运算的 device。

【参数】

|          |             |       |
|----------|-------------|-------|
| 参数       | 描述          | 输入/输出 |
| deviceId | 当前 deviceId | 输入    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

传入参数应为 ES\_NPU\_SetDevice 接口设置的 deviceId。

3.3.1.9. ES\_NPU\_GetDevice

【函数体】

```
ES_S32 ES_NPU_GetDevice(ES_U16 *deviceId)
```

【描述】

获取当前正在使用的 deviceId。

【参数】

| 参数       | 描述          | 输入/输出 |
|----------|-------------|-------|
| deviceId | 当前 deviceId | 输出    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.10. ES\_NPU\_CreateContext

【函数体】

```
ES_S32 ES_NPU_CreateContext(  
    npu_context* context,  
    ES_U16 deviceId)
```

【描述】

创建运行时上下文环境。

【参数】

| 参数       | 描述                         | 输入/输出 |
|----------|----------------------------|-------|
| context  | 返回的 context 指针             | 输出    |
| deviceId | 输入当前 context 要使用的 deviceId | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.11. ES\_NPU\_SetCurrentContext

【函数体】

ES\_S32 ES\_NPU\_SetCurrentContext(npu\_context context)

【描述】

把 context 设置给当前线程。

【参数】

| 参数      | 描述           | 输入/输出 |
|---------|--------------|-------|
| context | 要设置的 context | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.12. ES\_NPU\_GetCurrentContext

【函数体】

ES\_S32 ES\_NPU\_GetCurrentContext(npu\_context \*context)

【描述】

获取当前线程的 context。

【参数】

| 参数      | 描述          | 输入/输出 |
|---------|-------------|-------|
| context | 当前的 context | 输出    |

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.13. ES\_NPU\_DestroyContext

【函数体】

ES\_S32 ES\_NPU\_DestroyContext (npu\_context context)

【描述】

销毁 context。

【参数】

|         |              |       |
|---------|--------------|-------|
| 参数      | 描述           | 输入/输出 |
| context | 要销毁的 context | 输入    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.14. ES\_NPU\_CreateStream

【函数体】

ES\_S32 ES\_NPU\_CreateStream(npu\_stream \*stream)

【描述】

在当前线程对应的 Context 中创建一个 stream。

【参数】

|        |                |       |
|--------|----------------|-------|
| 参数     | 描述             | 输入/输出 |
| stream | 新创建 stream 的指针 | 输出    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |



3.3.1.15. ES\_NPU\_DestroyStream

【函数体】

```
ES_S32 ES_NPU_DestroyStream(npu_stream stream)
```

【描述】

在当前线程中销毁一个 stream。

【参数】

| 参数     | 描述          | 输入/输出 |
|--------|-------------|-------|
| stream | 要销毁的 stream | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.16. ES\_NPU\_SynchronizeStream

【函数体】

```
ES_S32 ES_NPU_SynchronizeStream(npu_stream stream)
```

【描述】

阻塞线程直到指定 stream 中的所有任务都完成。

【参数】

| 参数     | 描述          | 输入/输出 |
|--------|-------------|-------|
| stream | 要同步的 stream | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.17. ES\_NPU\_AbortStream

【函数体】

ES\_S32 ES\_NPU\_AbortStream(npu\_stream stream)

【描述】

取消该 stream 中的所有任务。

【参数】

| 参数     | 描述          | 输入/输出 |
|--------|-------------|-------|
| stream | 要取消的 stream | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.18. ES\_NPU\_LoadModelFromFile

【函数体】

```
ES_S32 ES_NPU_LoadModelFromFile(  
    ES_U32 *modelId,  
    const ES_CHAR *modelPath)
```

【描述】

从文件中加载离线模型。

【参数】

| 参数        | 描述         | 输入/输出 |
|-----------|------------|-------|
| modelId   | 返回的离线模型 Id | 输出    |
| modelPath | 离线模型文件路径   | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.19. ES\_NPU\_LoadModelFromMemory

【函数体】

```
ES_S32 ES_NPU_LoadModelFromMemory(
    ES_U32 * modelId,
    ES_CHAR * pBuffer,
    ES_U32 nBufLen)
```

【描述】

从内存中加载离线模型。

【参数】

| 参数      | 描述            | 输入/输出 |
|---------|---------------|-------|
| modelId | 返回的离线模型 Id    | 输出    |
| pBuffer | 存储离线模型的内存起始地址 | 输入    |
| nBufLen | 存储离线模型的内存长度   | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.20. ES\_NPU\_UnloadModel

【函数体】

```
ES_S32 ES_NPU_UnloadModel(ES_U32 modelId)
```

【描述】

卸载离线模型。

【参数】

| 参数      | 描述          | 输入/输出 |
|---------|-------------|-------|
| modelId | 要卸载的离线模型 Id | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.21. ES\_NPU\_LoadCompositeModel

【函数体】

```
ES_S32 ES_NPU_LoadCompositeModel(
ES_U32 *modelId,
const ES_CHAR *modelPath)
```

【描述】

加载复合模型，复合模型是为了更大化利用 npu/dsp 硬件而制作的模型集。

【参数】

| 参数        | 描述         | 输入/输出 |
|-----------|------------|-------|
| modelId   | 返回的离线模型 Id | 输出    |
| modelPath | 模型目录路径     | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

复合模型加载接口仅用于复合模型加载，无法与普通离线模型通用。

3.3.1.22. ES\_NPU\_GetCompositeModelInfo

【函数体】

```
ES_S32 ES_NPU_GetCompositeModelInfo(
ES_U32 modelId,
NPU_COMPOSITE_MODEL_INFO_S *compositeModelInfo)
```

【描述】

获取复合模型信息，包含模型集中所有模型的详细信息。

【参数】

| 参数                 | 描述        | 输入/输出 |
|--------------------|-----------|-------|
| modelId            | 复合模型 Id   | 输入    |
| compositeModelInfo | 复合模型的模型信息 | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

此接口仅针对复合模型的 modelId 有效。

3.3.1.23. ES\_NPU\_UnloadCompositeModel

【函数体】

```
ES_S32 ES_NPU_UnloadCompositeModel( ES_U32 modelId)
```

【描述】

卸载复合模型。

【参数】

| 参数      | 描述      | 输入/输出 |
|---------|---------|-------|
| modelId | 复合模型 Id | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

此接口仅针对复合模型的 modelId 有效。

3.3.1.24. ES\_NPU\_GetNumInputTensors

【函数体】

```
ES_S32 ES_NPU_GetNumInputTensors(  
    ES_U32 modelId,  
    ES_S32 * inputTensors)
```

【描述】

获取指定模型的 input tensor 的数量。

【参数】

| 参数           | 描述                   | 输入/输出 |
|--------------|----------------------|-------|
| modelId      | 离线模型 Id              | 输入    |
| inputTensors | 返回的 input tensor 的数量 | 输出    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.25. ES\_NPU\_GetNumOutputTensors

【函数体】

```
ES_S32 ES_NPU_GetNumOutputTensors(  
    ES_U32 modelId,  
    ES_S32 *outputTensors)
```

【描述】

获取指定模型的 output tensor 的数量。

【参数】

| 参数            | 描述                    | 输入/输出 |
|---------------|-----------------------|-------|
| modelId       | 离线模型 Id               | 输入    |
| outputTensors | 返回的 output tensor 的数量 | 输出    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.26. ES\_NPU\_GetInputTensorDesc

【函数体】

```
ES_S32 ES_NPU_GetInputTensorDesc(  
    ES_U32 modelId,  
    ES_S32 tensorId,  
    NPU_TENSOR_S *tensor)
```

【描述】

获取指定模型指定的输入 tensor 描述。

【参数】

| 参数       | 描述                                                                    | 输入/输出 |
|----------|-----------------------------------------------------------------------|-------|
| modelId  | 离线模型 Id                                                               | 输入    |
| tensorId | tensorId，从 0 开始，最大值为 ES_NPU_GetNumInputTensors 接口获取的 inputTensors - 1 | 输入    |
| tensor   | 返回 tensor 描述信息                                                        | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.27. ES\_NPU\_GetOutputTensorDesc

【函数体】

```
ES_S32 ES_NPU_GetOutputTensorDesc(  
    ES_U32 modelId,  
    ES_S32 tensorId,  
    NPU_TENSOR_S *tensor)
```

【描述】

获取指定模型指定的输出 tensor 描述。

【参数】

| 参数       | 描述                                                                      | 输入/输出 |
|----------|-------------------------------------------------------------------------|-------|
| modelId  | 离线模型 Id                                                                 | 输入    |
| tensorId | tensorId，从 0 开始，最大值为 ES_NPU_GetNumOutputTensors 接口获取的 outputTensors - 1 | 输入    |
| tensor   | 返回 tensor 描述信息                                                          | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.28. ES\_NPU\_Prepare

【函数体】

```
ES_S32 ES_NPU_Prepare (  
    ES_DEV_BUF_S *devBuf)
```

【描述】

为任务执行中所需的 DMA Buffer 建立对 NPU 设备的 SMMU 映射。调用此接口前必须先调用 ES\_NPU\_SetDevice。

【参数】

| 参数     | 描述                  | 输入/输出 |
|--------|---------------------|-------|
| devBuf | 需要映射的 DMA Buffer 指针 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.29. ES\_NPU\_Unprepare

【函数体】

```
ES_S32 ES_NPU_Unprepare (
    ES_DEV_BUF_S *devBuf)
```

【描述】

释放此 DMA Buffer 对 NPU 设备的 SMMU 映射，与 ES\_NPU\_Prepere 成对使用。

【参数】

| 参数     | 描述                   | 输入/输出 |
|--------|----------------------|-------|
| devBuf | 需要反映射的 DMA Buffer 指针 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.30. ES\_NPU\_Submit

【函数体】

```
ES_S32 ES_NPU_Submit(
    NPU_TASK_S *tasks,
    ES_U32 numTasks)
```

【描述】

提交任务，同步接口。该接口提交 NPU 任务并等待所有任务执行完成。

【参数】

| 参数       | 描述             | 输入/输出 |
|----------|----------------|-------|
| tasks    | 提交的 tasks 数组指针 | 输入    |
| numTasks | 提交的 task 的数量   | 输入    |

【返回值】



| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.31. ES\_NPU\_SubmitAsync

【函数体】

```
ES_S32 ES_NPU_SubmitAsync(  
    NPU_TASK_S *tasks,  
    ES_U32 numTasks,  
    npu_stream stream)
```

【描述】

提交任务，异步接口。该接口提交 NPU 任务后直接返回，用户需要通过 ES\_NPU\_ProcessReport 查询对应 stream 上任务的完成状态。

【参数】

| 参数       | 描述                  | 输入/输出 |
|----------|---------------------|-------|
| tasks    | 提交的 tasks 数组指针      | 输入    |
| numTasks | 提交的 task 的数量        | 输入    |
| stream   | 要提交任务到指定 stream 的指针 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.32. ES\_NPU\_AllocTaskMemory

【函数体】

```
ES_S32 ES_NPU_AllocTaskMemory(  
    ES_U32 modelId,  
    ES_U32 nums,  
    NPU_TASK_MEM_S *taskMem);
```

【描述】

分配任务所需内存。

【参数】

| 参数      | 描述                | 输入/输出 |
|---------|-------------------|-------|
| modelId | 所需分配内存的任务对应的模型 id | 输入    |

| 参数      | 描述          | 输入/输出 |
|---------|-------------|-------|
| nums    | 所需分配内存的任务数量 | 输入    |
| taskMem | 保存分配好的内存信息  | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.33. ES\_NPU\_ReleaseTaskMemory

【函数体】

```
ES_S32 ES_NPU_ReleaseTaskMemory(  
    ES_U32 modelId,  
    ES_U32 nums,  
    NPU_TASK_MEM_S *taskMem);
```

【描述】

释放任务内存。

【参数】

| 参数      | 描述              | 输入/输出 |
|---------|-----------------|-------|
| modelId | 所需释放的内存对应的模型 id | 输入    |
| nums    | 所需释放的内存数量       | 输入    |
| taskMem | 需要释放的内存信息       | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.34. ES\_NPU\_SetFlexibleTaskAttr

【函数体】

```
ES_S32 ES_NPU_SetFlexibleTaskAttr(  
    ES_U32 modelId,  
    NPU_FLEXIBLE_TASK_ATTR_S *attr)
```

【描述】

设置 flexible task 所需属性。

【参数】

| 参数      | 描述               | 输入/输出 |
|---------|------------------|-------|
| modelId | 复合模型 Id          | 输入    |
| attr    | Flexible task 属性 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

此接口仅针对复合模型的 modelId 有效。

3.3.1.35. ES\_NPU\_SubmitFlexibleTask

【函数体】

```
ES_S32 ES_NPU_SubmitFlexibleTask(  
    NPU_TASK_S *tasks,  
    ES_U32 numTasks,  
    npu_stream stream);
```

【描述】

异步提交复合模型的推理任务。相对于普通离线模型任务，用户不必按照批次大小，组装对应的模型任务，可直接根据单帧要求，构造相应任务，降低任务构造复杂度。对于多帧任务，可以通过任务数量指定。同时，npu runtime 对于复合模型任务的调度，会自动调节批大小，以达到更优的性能。

【参数】

| 参数      | 描述              | 输入/输出 |
|---------|-----------------|-------|
| tasks   | 需要提交的任务指针       | 输入    |
| numTask | 提交的任务数量         | 输入    |
| stream  | 提交的任务对应的 stream | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

此接口提交的任务需要使用 ES\_NPU\_AllocTaskMemory 接口分配内存，并且只能用于复合模型的任务提交。

3.3.1.36. ES\_NPU\_ProcessReport

【函数体】

```
ES_S32 ES_NPU_ProcessReport(  
    npu_stream stream,  
    ES_S32 timeoutMs)
```

【描述】

设置等待超时时间，同时查询哪些 task 运行完成，然后调用 task 的回调函数。如果在超时时间到达时没有任务完成，则该函数也会返回。

【参数】

| 参数        | 描述                                                                               | 输入/输出 |
|-----------|----------------------------------------------------------------------------------|-------|
| stream    | 需要处理的 stream 指针                                                                  | 输入    |
| timeoutMs | 单位为 ms。如果 timeoutMs 为-1，此接口会阻塞等待直到有 task 运行完成。如果 timeoutMs >=0，等待指定时间去查询 task 完成 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.37. ES\_NPU\_ExecuteTaskSync

【函数体】

```
ES_S32 ES_NPU_ExecuteTaskSync(  
    const NPU_TASK_S *task);
```

【描述】

同步执行 NPU 模型的推理任务。

【参数】

| 参数    | 描述        | 输入/输出 |
|-------|-----------|-------|
| tasks | 需要同步的任务指针 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.38. ES\_NPU\_DumpOpStatus

【函数体】

```
ES_S32 ES_NPU_DumpOpStatus(  
    ES_U32 modelId);
```

【描述】

导出 NPU 模型中各算子的运行状态。

【参数】

| 参数      | 描述    | 输入/输出 |
|---------|-------|-------|
| modelId | 模型 ID | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.1.39. ES\_NPU\_GetDdrBandwidth

【函数体】

```
ES_S32 ES_NPU_GetDdrBandwidth(  
    ES_U32 modelId,  
    ES_U64 *readSize,  
    ES_U64 *writeSize);
```

【描述】

用于性能统计, 获取模型对 DDR 内存的访问量。

【参数】

| 参数        | 描述              | 输入/输出 |
|-----------|-----------------|-------|
| modelId   | 模型 ID           | 输入    |
| readSize  | 模型从 DDR 读取的总数据量 | 输出    |
| writeSize | 模型向 DDR 写入的总数据量 | 输出    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

3.3.2. 数据类型和数据结构

3.3.2.1. NPU\_DIMS4\_S

【描述】

tensor 维度参数。

【定义】

```
typedef struct {
    ES_S32 n;
    ES_S32 c;
    ES_S32 h;
    ES_S32 w;
} NPU_DIMS4_S
```

【成员】

| 成员 | 描述                |
|----|-------------------|
| n  | 批处理大小，表示 batch 数量 |
| c  | 表示一张图像中的通道数       |
| h  | 表示图像垂直维度的像素数      |
| w  | 表示图像水平维度的像素数      |

3.3.2.2. NPU\_TENSOR\_S

【描述】

tensor 结构参数。

【定义】

```
typedef struct {
    ES_CHAR name[ES_NPU_TENSOR_DESC_NAME_MAX_LEN + 1];
    ES_U64 bufferSize;
    NPU_DIMS4_S dims;
    ES_U8 dataFormat;
    ES_U8 dataType;
    ES_U8 dataCategory;
    ES_U8 pixelFormat;
    ES_U8 pixelMapping;
    ES_U32 stride[ES_NPU_TENSOR_DESC_NUM_STRIDES];
} NPU_TENSOR_S
```

【成员】

| 成员           | 描述                                    |
|--------------|---------------------------------------|
| name         | Tensor 的名字                            |
| bufferSize   | Tensor 的 buffer 大小                    |
| dims         | Tensor 的维度参数                          |
| dataFormat   | Tensor 数据排布格式 (NCHW/NHWC)             |
| dataType     | Tensor 的数据类型 (int8、int16、fp16、fp32 等) |
| dataCategory | Tensor 的数据类别 (image、weight、bias 等)    |
| pixelFormat  | 像素格式                                  |
| pixelMapping | 像素映射方式                                |
| stride       | stride 信息                             |

3.3.2.3. NPU\_TASK\_S

【描述】

task 结构参数。

【定义】

```
typedef struct {
    ES_U32 taskId;
    ES_U32 modelId;
    ES_U8 inputFdNum;
    ES_U8 outputFdNum;
    ES_DEV_BUF_S inputFd[ES_TASK_MAX_FD_CNT];
    ES_DEV_BUF_S outputFd[ES_TASK_MAX_FD_CNT];
    NPU_TaskCallback callback;
    ES_VOID *callbackArg;
    NPU_TASK_STATE_E state;
    ES_U8 sdkPrivate[ES_TASK_SDK_PRIVATE_LEN];
} NPU_TASK_S
```

【成员】

| 成员          | 描述                      |
|-------------|-------------------------|
| taskId      | task 的 ID，task 提交后由系统分配 |
| modelId     | 该 task 所对应的模型 ID        |
| inputFdNum  | 作为输入的内存块数量              |
| outputFdNum | 作为输出的内存块数量              |
| inputFd     | 存放输入内存块的信息              |
| outputFd    | 存放输出内存块的信息              |
| callback    | task 执行完后需调用的回调函数       |
| callbackArg | 回调函数的参数                 |
| state       | task 状态                 |
| sdkPrivate  | SDK 内部使用的私有数据           |

3.3.2.4. NPU\_TASK\_MEM\_S

【描述】

任务内存结构体参数

【定义】

```
typedef struct {
    ES_U8 inputFdNum;
    ES_U8 outputFdNum;
    ES_DEV_BUF_S inputFd[ES_TASK_MAX_FD_CNT];
    ES_DEV_BUF_S outputFd[ES_TASK_MAX_FD_CNT];
} NPU_TASK_MEM_S;
```

【成员】

| 成员          | 描述         |
|-------------|------------|
| inputFdNum  | 作为输入的内存块数量 |
| outputFdNum | 作为输出的内存块数量 |
| inputFd     | 存放输入内存块的信息 |
| outputFd    | 存放输出内存块的信息 |

3.3.2.5. NPU\_MODEL\_INFO\_S

【描述】

模型结构体参数

【定义】

```
typedef struct {
    ES_CHAR modelName[MAX_MODEL_FILE_NAME];
    NPU_MODEL_TYPE_E modelType;
    ES_FLOAT modelCost;
    ES_U32 modelId;
    ES_U32 modelDevices;
} NPU_MODEL_INFO_S;
```

【成员】

| 成员           | 描述               |
|--------------|------------------|
| modelName    | 模型名称             |
| modelType    | 模型类型             |
| modelCost    | 模型代价，消耗的理论时间（ms） |
| modelId      | 模型 Id            |
| modelDevices | 模型需要的硬件数量        |

3.3.2.6. NPU\_COMPOSITE\_MODEL\_INFO\_S

【描述】

复合模型结构体参数



【定义】

```
typedef struct {
    ES_U32 modelNums;
    NPU_MODEL_INFO_S modelsInfo[MAX_MODELS_OF_SET];
} NPU_COMPOSITE_MODEL_INFO_S;
```

【成员】

| 成员         | 描述     |
|------------|--------|
| modelNums  | 模型数量   |
| modelsInfo | 模型信息数组 |

3.3.2.7. NPU\_FLEXIBLE\_TASK\_ATTR\_S

【描述】

灵活任务属性结构参数

【定义】

```
typedef struct {
    ES_S32 timeOut;
} NPU_FLEXIBLE_TASK_ATTR_S;
```

【成员】

| 成员      | 描述                    |
|---------|-----------------------|
| timeOut | 任务超时时间（超时后，尽快调度任务 ms） |

3.3.2.8. NPU\_TaskCallback

【描述】

task 执行完毕后需调用的回调函数。

【定义】

```
typedef ES_S32 (*NPU_TaskCallback)(void *arg)
```

3.3.2.9. npu\_context

【描述】

npu 的 context 指针。

【定义】

```
typedef void * npu_context
```

3.3.2.10. npu\_stream

【描述】

npu 的 stream 指针。

【定义】

```
typedef void * npu_stream
```

3.4. Python API 详解

3.4.1. 版本信息和系统配置

3.4.1.1. 获取版本号

```
from es_py import npu, common

# 获取 NPU 版本号（数值格式）
version = npu.get_version()

# 获取 NPU 版本字符串
version_string = npu.get_version_string(max_size=256)

# 获取硬件能力信息
capability = npu.get_capability()
print(capability.reserved)
```

【参数说明（get\_version）】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【参数说明（get\_version\_string）】

| 参数       | 类型  | 描述        | 输入/输出 |
|----------|-----|-----------|-------|
| max_size | int | 版本字符串最大长度 | 输入    |

【参数说明（get\_capability）】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

- get\_version：返回 int 类型的版本号
- get\_version\_string：返回 str 类型的版本字符串
- get\_capability：返回 common.ES\_NPU\_Capability\_S 硬件能力信息对象

【说明】

`get_capability()` 的返回类型在 `common` 模块中注册，调用前需导入 `common`。

3.4.1.2. 设置日志级别

```
# 设置 NPU 日志级别
npu.set_log_level(level=2) # 0: 无日志, 1: 错误, 2: 警告, 3: 信息
```

【参数说明】

| 参数                 | 类型               | 描述                           | 输入/输出 |
|--------------------|------------------|------------------------------|-------|
| <code>level</code> | <code>int</code> | 日志级别：0=无日志, 1=错误, 2=警告, 3=信息 | 输入    |

【返回值】

`int`：0 表示成功

【说明】

- 用于控制 NPU 运行时的日志输出详细程度
- 调试时建议使用 `level=3`（信息级别）

3.4.2. 设备管理

3.4.2.1. 获取设备数量

```
from es_py import npu

# 获取可用的 NPU 设备数量
device_num = npu.get_num_devices()
print(f"可用 NPU 设备数: {device_num}")
```

【参数说明】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

`int`：NPU 设备总数

【异常】

失败时抛出 `ESNpuError` 异常

【说明】

- 返回系统中所有可用的 NPU 设备数量
- 设备 ID 的有效范围为 `[0, device_num-1]`

3.4.2.2. 设置当前设备

```
# 设置用于运算的设备
npu.set_device(device_id=0)
```

【参数说明】

| 参数        | 类型  | 描述                         | 输入/输出 |
|-----------|-----|----------------------------|-------|
| device_id | int | 设备 ID，范围 [0, device_num-1] | 输入    |

【返回值】

int：0 表示成功

【异常】

- 如果设备 ID 无效，抛出 ESNpuError 异常
- 如果设备已被其他进程占用，可能抛出异常

【说明】

- 设置后的设备为当前线程的默认设备
- 后续创建的 Context 将关联到该设备
- 同一进程中的不同线程可以设置不同的设备

3.4.2.3. 获取当前设备

```
# 获取当前设备
current_device = npu.get_device()
print(f"当前设备 ID: {current_device}")
```

【参数说明】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

int：当前设备 ID

【异常】

如果未设置设备，可能抛出 ESNpuError 异常

3.4.2.4. 获取设备属性

```
from es_py import npu, common

# 获取指定设备的属性信息
device_props = npu.get_device_properties(device_id=0)
```

```
print(f"芯片版本: {device_props.chipVer}")
print(f"设备内存大小: {device_props.memSize}")
print(f"计算单元数量: {device_props.computingCnt}")
print(f"是否支持 FP16: {device_props.isFp16Supported}")
```

【参数说明】

| 参数        | 类型  | 描述    | 输入/输出 |
|-----------|-----|-------|-------|
| device_id | int | 设备 ID | 输入    |

【返回值】

common.NPU\_DEVICE\_PROP\_S：设备属性对象，详见数据结构章节

【异常】

设备不存在时抛出 ESNpuError 异常

【说明】

- 返回设备的详细硬件属性和能力
- 用于判断模型与设备的兼容性
- get\_device\_properties() 的返回类型在 common 模块中注册，调用前需导入 common。

3.4.2.5. 释放设备

```
# 释放设备（释放后该设备的资源不可用）
npu.release_device(device_id=0)
```

【参数说明】

| 参数        | 类型  | 描述    | 输入/输出 |
|-----------|-----|-------|-------|
| device_id | int | 设备 ID | 输入    |

【返回值】

int：0 表示成功

【异常】

设备不存在或未被当前进程占用时抛出 ESNpuError 异常

【说明】

- 释放前必须销毁所有关联的 Context
- Device 被释放后，其对应的资源在本进程不可用

3.4.3. 上下文管理

3.4.3.1. 创建 Context

```
# 创建 Context
context = npu.create_context(device_id=0)
```

【参数说明】

| 参数        | 类型  | 描述    | 输入/输出 |
|-----------|-----|-------|-------|
| device_id | int | 设备 ID | 输入    |

【返回值】

capsule：npu.create\_context() 返回的 Context 句柄

【异常】

- 设备 ID 无效时抛出 ESNpuError 异常
- 内存不足时抛出 ESNpuError 异常

【说明】

- 创建 Context 前必须调用 set\_device() 设置设备
- 同一 Context 可被多个线程使用，但推荐一线程一 Context

3.4.3.2. 设置当前 Context

```
# 设置当前 Context
npu.set_current_context(context)
```

【参数说明】

| 参数      | 类型      | 描述                                  | 输入/输出 |
|---------|---------|-------------------------------------|-------|
| context | capsule | npu.create_context() 返回的 Context 句柄 | 输入    |

【返回值】

int：0 表示成功

【异常】

Context 无效时抛出 ESNpuError 异常

【说明】

- 设置后的 Context 为当前线程的活跃 Context
- 后续 Stream 操作将在该 Context 中进行

3.4.3.3. 获取当前 Context

```
# 获取当前 Context
current_context = npu.get_current_context()
```

【参数说明】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

capsule：当前 Context 句柄

【异常】

未设置 Context 时可能抛出 ESNpuError 异常

3.4.3.4. 销毁 Context

```
# 销毁 Context
npu.destroy_context(context)
```

【参数说明】

| 参数      | 类型      | 描述                                  | 输入/输出 |
|---------|---------|-------------------------------------|-------|
| context | capsule | npu.create_context() 返回的 Context 句柄 | 输入    |

【返回值】

int：0 表示成功

【异常】

Context 无效或仍有未销毁的 Stream 时抛出 ESNpuError 异常

【说明】

- 销毁前必须销毁所有关联的 Stream
- Context 销毁后其中的资源将被释放

3.4.4. 流管理

3.4.4.1. 创建 Stream

```
# 在当前 Context 中创建 Stream
stream = npu.create_stream()
```

【参数说明】

无

【返回值】

capsule：npu.create\_stream() 返回的 Stream 句柄

【异常】

- 未设置 Context 时抛出 ESNpuError 异常
- 内存不足时抛出 ESNpuError 异常

【说明】

- Stream 在当前线程的 Context 中创建
- 一个 Context 可创建多个 Stream

3.4.4.2. 销毁 Stream

```
# 销毁 Stream
npu.destroy_stream(stream)
```

【参数说明】

| 参数     | 类型      | 描述                                | 输入/输出 |
|--------|---------|-----------------------------------|-------|
| stream | capsule | npu.create_stream() 返回的 Stream 句柄 | 输入    |

【返回值】

int：0 表示成功

【异常】

Stream 无效或仍有未完成的任务时抛出 ESNpuError 异常

【说明】

- 销毁前应等待所有任务完成
- 可调用 synchronize\_stream() 确保所有任务完成

3.4.4.3. 同步 Stream

```
# 同步 Stream（等待所有任务完成）
npu.synchronize_stream(stream)
```

【参数说明】

| 参数     | 类型      | 描述                                | 输入/输出 |
|--------|---------|-----------------------------------|-------|
| stream | capsule | npu.create_stream() 返回的 Stream 句柄 | 输入    |

【返回值】

int：0 表示成功

【异常】

- Stream 无效时抛出 ESNpuError 异常
- 任务执行失败时抛出 ESNpuError 异常

【说明】

- 阻塞调用线程直到指定 stream 中的所有任务都完成
- 用于同步异步操作



3.4.4.4. 中止 Stream

```
# 取消 Stream 中的所有任务
npu.abort_stream(stream)
```

【参数说明】

| 参数     | 类型      | 描述                                | 输入/输出 |
|--------|---------|-----------------------------------|-------|
| stream | capsule | npu.create_stream() 返回的 Stream 句柄 | 输入    |

【返回值】

int：0 表示成功

【异常】

Stream 无效时抛出 ESNpuError 异常

【说明】

- 取消该 stream 中的所有待处理任务
- 已经开始执行的任务无法取消

3.4.5. 模型加载

3.4.5.1. 从文件加载模型

```
# 从文件加载离线模型
model_id = npu.load_model_from_file(model_path="/path/to/model.model")

# 从内存加载离线模型
model_data = open("/path/to/model.model", "rb").read()
model_id = npu.load_model_from_memory(model_data)
```

【参数说明（load\_model\_from\_file）】

| 参数         | 类型  | 描述     | 输入/输出 |
|------------|-----|--------|-------|
| model_path | str | 模型文件路径 | 输入    |

【参数说明（load\_model\_from\_memory）】

| 参数         | 类型    | 描述   | 输入/输出 |
|------------|-------|------|-------|
| model_data | bytes | 模型数据 | 输入    |

【返回值】

int：模型 ID

【异常】

- 模型文件不存在时抛出 ESNpuError 异常

- 模型格式不支持时抛出 `ESNpuError` 异常
- 内存不足时抛出 `ESNpuError` 异常

【说明】

- 模型加载后获得一个唯一的 `modelId`，后续操作都使用此 ID
- 支持加载 FP32、FP16、INT8 等格式模型

3.4.5.2. 加载复合模型

```
# 加载复合模型（包含多个模型的集合）
composite_model_id = npu.load_composite_model(model_path="/path/to/model_dir")

# 获取复合模型信息
composite_info = npu.get_composite_model_info(model_id=composite_model_id)
print(f"模型数量: {composite_info.modelNums}")

for i in range(composite_info.modelNums):
    model_info = composite_info.modelsInfo[i]
    print(f"模型名: {model_info.modelName}")
    print(f"模型 ID: {model_info.modelId}")
    print(f"模型类型: {model_info.modelType}")
```

【参数说明】

| 参数                     | 类型  | 描述       | 输入/输出 |
|------------------------|-----|----------|-------|
| model_path 或 model_dir | str | 复合模型目录路径 | 输入    |

【返回值】

`int`：复合模型 ID

【异常】

模型目录不存在或格式不正确时抛出 `ESNpuError` 异常

【说明】

- 复合模型用于包含多个 NPU 子模型或混合硬件任务
- 复合模型内可包含不同批大小的模型

3.4.5.3. 卸载模型

```
# 卸载单个模型
npu.unload_model(model_id)

# 卸载复合模型
npu.unload_composite_model(composite_model_id)
```

【参数说明】

| 参数       | 类型  | 描述                | 输入/输出 |
|----------|-----|-------------------|-------|
| model_id | int | 模型 ID (普通模型或复合模型) | 输入    |

【返回值】

int：0 表示成功

【异常】

- 模型 ID 无效时抛出 ESNpuError 异常
- 模型仍被使用时抛出 ESNpuError 异常

【说明】

- 卸载前确保所有使用该模型的任务已完成
- 卸载后模型 ID 失效

3.4.6. 张量信息查询

3.4.6.1. 获取输入张量数量

```
# 获取输入张量数量
num_input_tensors = npu.get_num_input_tensors(model_id)
print(f"输入张量数: {num_input_tensors}")
```

【参数说明】

model\_id：int 类型的模型 ID

【返回值】

int：输入张量数量

【异常】

模型 ID 无效时抛出 ESNpuError 异常

3.4.6.2. 获取输出张量数量

```
# 获取输出张量数量
num_output_tensors = npu.get_num_output_tensors(model_id)
print(f"输出张量数: {num_output_tensors}")
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

int：输出张量数量

【异常】

模型 ID 无效时抛出 ESNpuError 异常

3.4.6.3. 获取输入/输出张量信息

```
from es_py import npu, common

# 获取第一个输入张量的描述
input_tensor_desc = npu.get_input_tensor_desc(model_id, tensor_id=0)
print(f"张量名: {input_tensor_desc.name}")
print(f"数据类型: {input_tensor_desc.dataType}")
print(f"形状: {input_tensor_desc.dims}")
print(f"缓冲大小: {input_tensor_desc.bufferSize}")

# 获取第一个输出张量的描述
output_tensor_desc = npu.get_output_tensor_desc(model_id, tensor_id=0)
print(f"张量名: {output_tensor_desc.name}")
```

【参数说明】

| 参数        | 类型  | 描述           | 输入/输出 |
|-----------|-----|--------------|-------|
| model_id  | int | 模型 ID        | 输入    |
| tensor_id | int | 张量索引（从 0 开始） | 输入    |

【返回值】

common.NPU\_TENSOR\_S：张量描述结构体，详见数据结构章节

【异常】

模型 ID 无效或张量索引超出范围时抛出 `ESNpuError` 异常

【说明】

- 张量索引范围：[0, num\_input\_tensors) 或 [0, num\_output\_tensors)
- 通过张量信息确定内存分配大小和数据格式

3.4.7. 内存管理

3.4.7.1. 准备和释放 DMA Buffer

```
from es_py import npu, common, memory

# 为 DMA Buffer 准备 SMMU 映射
buffer_size = 4096
buffer_fd = memory.memory_alloc(
    common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_CACHED,
    "npu_buf",
    "",
    buffer_size,
)

dev_buf = common.ES_DEV_BUF_S()
dev_buf.memFd = buffer_fd
dev_buf.offset = 0
dev_buf.size = buffer_size
dev_buf.reserve = 0

npu.prepare(dev_buf)

# ... 执行推理 ...
```

```
# 释放 SMMU 映射
npu.unprepare(dev_buf)
memory.memory_free(buffer_fd)
```

【参数说明（prepare）】

| 参数      | 类型                  | 描述      | 输入/输出 |
|---------|---------------------|---------|-------|
| dev_buf | common.ES_DEV_BUF_S | 设备缓冲区对象 | 输入    |

【参数说明（unprepare）】

| 参数      | 类型                  | 描述            | 输入/输出 |
|---------|---------------------|---------------|-------|
| dev_buf | common.ES_DEV_BUF_S | 要释放映射的设备缓冲区对象 | 输入    |

【返回值】

int：0 表示成功

【说明】

- prepare 用于建立 SMMU 映射，使 NPU 可访问 DMA Buffer
- unprepare 释放 SMMU 映射，释放相关资源
- 必须成对调用，否则可能导致资源泄漏
- common.ES\_DEV\_BUF\_S() 默认字段为 0，只能说明类型可构造；实际调用 prepare() 时必须填入有效的设备缓冲区 memFd 和 size。

3.4.7.2. 分配和释放任务内存

当前 Python 绑定导出了 alloc\_task\_memory() 和 release\_task\_memory() 接口，用于任务内存的申请和释放。普通离线模型推理示例使用自备内存路径：通过 memory.memory\_alloc() 申请输入/输出缓冲区，填充 common.ES\_DEV\_BUF\_S，再设置到 common.NPU\_TASK\_S 中提交任务。

```
# API 调用形式。仅在任务内存通过 alloc_task_memory()/release_task_memory()
# 路径管理时使用。
task_memories = npu.alloc_task_memory(model_id, nums=1)

# 使用后释放
npu.release_task_memory(model_id, task_memories)
```

【参数说明（alloc\_task\_memory）】

| 参数       | 类型  | 描述       | 输入/输出 |
|----------|-----|----------|-------|
| model_id | int | 模型 ID    | 输入    |
| nums     | int | 要分配的任务数量 | 输入    |

【参数说明（release\_task\_memory）】

| 参数            | 类型                          | 描述         | 输入/输出 |
|---------------|-----------------------------|------------|-------|
| model_id      | int                         | 模型 ID      | 输入    |
| task_memories | list[common.NPU_TASK_MEM_S] | 要释放的任务内存列表 | 输入    |

【返回值】

- list[common.NPU\_TASK\_MEM\_S]：任务内存列表（alloc\_task\_memory）
- int：0 表示成功（release\_task\_memory）

【说明】

- alloc\_task\_memory() 和 release\_task\_memory() 需要成对使用。
- 普通离线模型推理优先参考下文 submit() 和 submit\_async() 中的自备内存示例，除非模型/任务流程明确要求使用该接口路径。
- 普通同步/异步推理示例见下文任务提交章节。

3.4.8. 任务提交

3.4.8.1. 同步提交任务

```
from es_py import npu, common

# 创建任务
task = common.NPU_TASK_S()
task.modelId = model_id
task.inputFdNum = 1
task.outputFdNum = 1
task.set_inputFd(0, input_buffer)
task.set_outputFd(0, output_buffer)

# 同步提交任务（等待完成）
npu.submit([task])
print("推理完成，结果已保存到 output_buffer")
```

【参数说明】

| 参数    | 类型                      | 描述   | 输入/输出 |
|-------|-------------------------|------|-------|
| tasks | list[common.NPU_TASK_S] | 任务列表 | 输入    |

【返回值】

int：0 表示成功

【异常】

- 任务参数无效时抛出 ESNpuError 异常
- 模型 ID 无效时抛出 ESNpuError 异常

【说明】

- 同步提交会阻塞直到所有任务完成
- 适用于批量推理场景

3.4.8.2. 异步提交任务

```
from es_py import npu, common

# 创建任务
task = common.NPU_TASK_S()
task.modelId = model_id
task.inputFdNum = 1
task.outputFdNum = 1
task.set_inputFd(0, input_buffer)
task.set_outputFd(0, output_buffer)

# 定义回调函数
def task_callback(arg=None):
    print("任务完成")
    return 0

task.set_callback_py(task_callback, {"case": "npu_async"})

# 异步提交任务
npu.submit_async([task], stream)

# 处理异步回调
npu.process_report(stream=stream, timeout_ms=-1) # 阻塞等待
```

【参数说明（submit\_async）】

| 参数     | 类型                      | 描述                                | 输入/输出 |
|--------|-------------------------|-----------------------------------|-------|
| tasks  | list[common.NPU_TASK_S] | 任务列表                              | 输入    |
| stream | capsule                 | npu.create_stream() 返回的 Stream 句柄 | 输入    |

【参数说明（process\_report）】

| 参数         | 类型      | 描述                                | 输入/输出 |
|------------|---------|-----------------------------------|-------|
| stream     | capsule | npu.create_stream() 返回的 Stream 句柄 | 输入    |
| timeout_ms | int     | 超时时间（毫秒），-1 表示无限等待                | 输入    |

【返回值】

int：0 表示成功

【异常】

- 任务参数无效时抛出 `ESNpuError` 异常
- 超时时可能抛出 `TimeoutError` 异常

【说明】

- 异步提交立即返回，通过回调通知完成
- 必须调用 `process_report` 处理回调
- 适用于需要高吞吐量的流式推理

3.4.8.3. 提交复合模型任务

```
from es_py import npu, common

# 设置复合模型任务属性
task_attr = common.NPU_FLEXIBLE_TASK_ATTR_S()
task_attr.timeOut = 5000 # 5 秒超时

npu.set_flexible_task_attr(model_id=composite_model_id, attr=task_attr)

# 创建任务（不需要按批大小组织）
tasks = []
for i in range(10): # 10 帧数据
    task = common.NPU_TASK_S()
    task.modelId = composite_model_id
    task.inputFdNum = 1
    task.outputFdNum = 1
    task.set_inputFd(0, input_frames[i])
    task.set_outputFd(0, output_results[i])
    tasks.append(task)

# 异步提交复合模型任务
npu.submit_flexible_task(tasks, stream)

# 处理回调
npu.process_report(stream=stream, timeout_ms=1000)
```

【参数说明（set\_flexible\_task\_attr）】

| 参数       | 类型                                   | 描述      | 输入/输出 |
|----------|--------------------------------------|---------|-------|
| model_id | int                                  | 复合模型 ID | 输入    |
| attr     | common.NPU_FLEXI-<br>BLE_TASK_ATTR_S | 灵活任务属性  | 输入    |

【参数说明（submit\_flexible\_task）】

| 参数     | 类型                           | 描述                                    | 输入/输出 |
|--------|------------------------------|---------------------------------------|-------|
| tasks  | list[com-<br>mon.NPU_TASK_S] | 任务列表                                  | 输入    |
| stream | capsule                      | npu.create_stream() 返回<br>的 Stream 句柄 | 输入    |

【返回值】

int：0 表示成功

【说明】

- 复合模型任务支持动态批大小
- 不需要按固定批大小组织任务
- 适用于变长输入场景



3.4.9. 调试和性能分析

3.4.9.1. 启用 Dump 功能

```
# 启用 Dump 功能
npu.enable_dump(model_id)
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

int：0 表示成功

【说明】

- 该接口用于为指定已加载模型启用 Dump 功能。
- Dump 接口属于调试控制接口。除非部署流程明确要求，否则不要把 Dump/Performance 探测混入最小推理示例。

3.4.9.2. 禁用 Dump 功能

```
# 禁用 Dump 功能
npu.disable_dump(model_id)
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

int：0 表示成功

【说明】

禁用后恢复正常推理性能

3.4.9.3. 获取算子状态

```
# 输出模型算子状态
npu.dump_op_status(model_id)
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

int：0 表示成功

【说明】

- 该接口可在模型加载后独立调用，用于输出模型算子状态。
- 收集调试信息时可与 enable\_dump() / disable\_dump() 配合使用。

3.4.9.4. 启用性能监测

```
# 启用性能监测
npu.enable_performance(model_id)

# 执行推理...

# 将性能数据导出到目录
npu.dump_performance_data(model_id, "/tmp/npu_perf")
```

【参数说明（enable\_performance）】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【参数说明（dump\_performance\_data）】

| 参数       | 类型  | 描述       | 输入/输出 |
|----------|-----|----------|-------|
| model_id | int | 模型 ID    | 输入    |
| dir      | str | 性能数据输出目录 | 输入    |

【返回值】

int：0 表示成功

【说明】

- 该接口用于为指定已加载模型启用性能监测。
- Performance 接口可独立于常规 submit() / submit\_async() 示例进行检查。
- 当前接口通过 dump\_performance\_data() 导出性能数据文件，不提供直接返回性能对象的接口。
- 编写基础用法时，建议将性能数据采集与最小推理示例分开。

3.4.9.5. 禁用性能监测

```
# 禁用性能监测
npu.disable_performance(model_id)
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

int：0 表示成功

3.4.9.6. 获取 DDR 带宽

```
# 获取指定模型的 DDR 读写数据量
read_size, write_size = npu.get_ddr_bandwidth(model_id)
print(f"DDR 读取数据量: {read_size}")
print(f"DDR 写入数据量: {write_size}")
```

【参数说明】

| 参数       | 类型  | 描述    | 输入/输出 |
|----------|-----|-------|-------|
| model_id | int | 模型 ID | 输入    |

【返回值】

tuple[int, int]：读数据量和写数据量

【说明】

- 用于监控内存访问瓶颈
- 帮助优化数据传输策略
- Python 绑定返回 (read\_size, write\_size) 元组。

3.4.10. 数据类型和数据结构

NPU Runtime 相关数据类型、数据结构定义如下：

- common.ES\_NPU\_Capability\_S：定义 NPU 能力信息
- common.NPU\_DEVICE\_PROP\_S：定义 NPU 设备属性信息
- common.NPU\_DIMS4\_S：定义 tensor 维度参数
- common.NPU\_TENSOR\_S：定义 tensor 结构参数
- common.NPU\_TASK\_S：定义 NPU task 结构体描述
- common.NPU\_TASK\_MEM\_S：定义任务内存结构
- common.NPU\_MODEL\_INFO\_S：定义模型信息结构体
- common.NPU\_COMPOSITE\_MODEL\_INFO\_S：定义复合模型信息结构体
- common.NPU\_FLEXIBLE\_TASK\_ATTR\_S：定义灵活任务属性
- common.ES\_DEV\_BUF\_S：定义设备缓冲区
- Context 句柄：npu.create\_context() 返回的 capsule
- Stream 句柄：npu.create\_stream() 返回的 capsule

3.4.10.1. common.ES\_NPU\_Capability\_S

定义 NPU 能力信息。

| 参数       | 类型  | 描述   |
|----------|-----|------|
| reserved | int | 保留字段 |

3.4.10.2. common.NPU\_DEVICE\_PROP\_S

定义 NPU 设备属性信息。

| 参数                    | 类型     | 描述            |
|-----------------------|--------|---------------|
| chipVer               | int    | 芯片版本号         |
| memSize               | int    | NPU 可用内存大小    |
| isInt8Supported       | int    | 是否支持 INT8 计算  |
| isInt16Supported      | int    | 是否支持 INT16 计算 |
| isInt32Supported      | int    | 是否支持 INT32 计算 |
| isFp16Supported       | int    | 是否支持 FP16 计算  |
| isFp32Supported       | int    | 是否支持 FP32 计算  |
| computingCnt          | int    | 计算单元数量        |
| get_topologicalInfo() | method | 获取拓扑结构信息列表    |

【使用示例】

```
from es_py import npu, common

npu.set_device(0)
try:
    device_prop = npu.get_device_properties(device_id=0)
    print(device_prop.chipVer)
    print(device_prop.memSize)
    print(device_prop.isFp16Supported)
    print(device_prop.computingCnt)
finally:
    npu.release_device(0)
```

3.4.10.3. common.NPU\_DIMS4\_S

定义 tensor 维度参数。

| 参数 | 类型  | 描述                |
|----|-----|-------------------|
| n  | int | 批处理大小，表示 batch 数量 |
| c  | int | 通道数               |
| h  | int | 图像垂直维度的像素数        |
| w  | int | 图像水平维度的像素数        |

3.4.10.4. common.NPU\_TENSOR\_S

定义 tensor 结构参数。

【成员说明】

| 参数           | 类型                 | 描述                                             |
|--------------|--------------------|------------------------------------------------|
| name         | str                | Tensor 的名字，最大长度 128 字符                         |
| bufferSize   | int                | Tensor 的 buffer 大小，单位：字节                       |
| dims         | common.NPU_DIMS4_S | Tensor 的维度参数 (n, c, h, w)                      |
| dataFormat   | int                | Tensor 数据排布格式：0=NCHW, 1=NHWC                   |
| dataType     | int                | Tensor 的数据类型：0=INT8, 1=INT16, 2=FP16, 3=FP32 等 |
| dataCategory | int                | Tensor 表达的数据类型：0=image, 1=weight, 2=bias 等     |
| pixelFormat  | int                | 像素格式，如 RGB888、YUV420 等                         |
| pixelMapping | int                | 像素映射方式，指定像素在内存中的排列方式 (packed/planar)           |
| stride       | List[int]          | stride 信息，每个维度的跨度（字节数）                         |

【pixel\_mapping 字段说明】

- 0：Packed 格式（RGB 数据交错存储：RGBRGBRGB…）
- 1：Planar 格式（RGB 数据分平面存储：RRR…GGG…BBB…）
- 影响内存访问模式和性能，需与模型要求匹配

【使用示例】

```
from es_py import npu, common

# 获取输入张量描述
input_tensor = npu.get_input_tensor_desc(model_id, tensor_id=0)

# 根据 tensor 信息分配内存
buffer_size = input_tensor.bufferSize
print(f"需要分配内存: {buffer_size}字节")
print(f"数据格式: {'NCHW' if input_tensor.dataFormat == 0 else 'NHWC'}")
print(f"像素映射: {'Packed' if input_tensor.pixelMapping == 0 else 'Planar'}")

# 检查 stride 信息
print(f"Stride: {input_tensor.stride}")
```

【说明】

- bufferSize 用于确定需要分配的内存大小
- pixelMapping 影响图像数据的预处理方式
- stride 信息用于处理内存对齐要求

3.4.10.5. common.NPU\_TASK\_S

定义 task 结构参数。

【成员说明】

| 参数         | 类型  | 描述                             |
|------------|-----|--------------------------------|
| taskId     | int | task 的 ID，task 提交后由系统分配，用于追踪任务 |
| modelId    | int | 该 task 所对应的模型 ID               |
| inputFdNum | int | 输入内存块数量                        |

| 参数                               | 类型  | 描述                           |
|----------------------------------|-----|------------------------------|
| outputFdNum                      | int | 输出内存块数量                      |
| state                            | int | task 当前状态（见任务状态枚举）           |
| set_inputFd(index, buffer)       | 方法  | 设置输入 common.ES_DEV_BUF_S 缓冲区 |
| get_inputFd(index)               | 方法  | 获取输入缓冲区                      |
| set_outputFd(index, buffer)      | 方法  | 设置输出 common.ES_DEV_BUF_S 缓冲区 |
| get_outputFd(index)              | 方法  | 获取输出缓冲区                      |
| set_callback_py(callback[, arg]) | 方法  | 设置异步任务完成回调                   |
| clear_callback_py()              | 方法  | 清除回调                         |
| get_c_task()                     | 方法  | 获取内部 C 结构引用                  |

【任务状态枚举（common.NPU\_TASK\_STATE\_E）】

| 状态值 | 常量名                      | 描述           |
|-----|--------------------------|--------------|
| 0   | NPU_TASK_STATE_NONE      | 初始状态         |
| 1   | NPU_TASK_STATE_READ      | 任务已准备好，等待调度  |
| 2   | NPU_TASK_STATE_RUNNING   | 任务正在 NPU 上执行 |
| 3   | NPU_TASK_STATE_COMPLETED | 任务执行完成       |
| 4   | NPU_TASK_STATE_ABORTED   | 任务被中止        |
| 5   | NPU_TASK_STATE_FAILED    | 任务执行失败       |

【使用示例】

```
from es_py import npu, common

# 创建任务
task = common.NPU_TASK_S()
task.modelId = model_id
task.inputFdNum = 1
task.outputFdNum = 1
task.set_inputFd(0, input_buffer)
task.set_outputFd(0, output_buffer)

# 设置回调函数
def my_callback(arg=None):
    print("任务完成")
    return 0

task.set_callback_py(my_callback)

# 提交异步任务
npu.submit_async([task], stream)

# 处理回调
npu.process_report(stream, timeout_ms=1000)
```

【回调函数说明】

回调函数原型：

```
def task_callback(arg=None) -> int:
    """
    返回:
        0: 成功
        非 0: 失败
    """
    pass
```

【注意事项】

- inputFdNum 和 outputFdNum 必须与实际提供的 buffer 数量匹配
- 输入、输出缓冲区通过 set\_inputFd() 、 set\_outputFd() 设置
- 回调函数在任务完成时由 process\_report() 触发

3.4.10.6. common.NPU\_TASK\_MEM\_S

定义任务内存结构。

| 参数          | 类型     | 描述                        |
|-------------|--------|---------------------------|
| inputFdNum  | int    | 输入内存块数量                   |
| outputFdNum | int    | 输出内存块数量                   |
| inputFd     | C 数组代理 | 输入 common.ES_DEV_BUF_S 数组 |
| outputFd    | C 数组代理 | 输出 common.ES_DEV_BUF_S 数组 |

3.4.10.7. common.NPU\_MODEL\_INFO\_S

定义模型信息结构体。

| 参数           | 类型                      | 描述        |
|--------------|-------------------------|-----------|
| modelName    | str                     | 模型文件路径或名称 |
| modelType    | common.NPU_MODEL_TYPE_E | 模型类型      |
| modelCost    | int                     | 模型代价      |
| modelId      | int                     | 模型 ID     |
| modelDevices | int                     | 模型需要的硬件数量 |

3.4.10.8. common.NPU\_COMPOSITE\_MODEL\_INFO\_S

定义复合模型信息结构体。

| 参数         | 类型     | 描述     |
|------------|--------|--------|
| modelNums  | int    | 模型数量   |
| modelsInfo | C 数组代理 | 模型信息数组 |

3.4.10.9. common.NPU\_FLEXIBLE\_TASK\_ATTR\_S

定义复合模型任务属性。

| 参数      | 类型  | 描述           |
|---------|-----|--------------|
| timeOut | int | 任务超时时间，单位：ms |

3.4.10.10. common.ES\_DEV\_BUF\_S

定义设备缓冲区结构体。

| 参数      | 类型  | 描述        |
|---------|-----|-----------|
| memFd   | int | 设备内存文件描述符 |
| offset  | int | 缓冲区偏移     |
| size    | int | 缓冲区大小     |
| reserve | int | 保留字段      |

3.5. 错误码

| 参数                             | 值          | 描述     |
|--------------------------------|------------|--------|
| ES_NPU_ERROR_BAD_PARAM         | 0xA00F6003 | 非法参数   |
| ES_NPU_ERROR_NULL_PTR          | 0xA00F6006 | 空指针    |
| ES_NPU_ERROR_NO_MEMORY         | 0xA00F600C | 内存不足   |
| ES_NPU_ERROR_INVALID_ADDR      | 0xA00F6011 | 无效地址   |
| ES_NPU_ERROR_BUSY              | 0xA00F6012 | 设备繁忙   |
| ES_NPU_ERROR_NOT_INIT          | 0xA00F6040 | 设备未初始化 |
| ES_NPU_ERROR_TIME_OUT          | 0xA00F6041 | 操作超时   |
| ES_NPU_ERROR_INVALID_STATE     | 0xA00F6042 | 无效状态   |
| ES_NPU_ERROR_INVALID_SIZE      | 0xA00F6043 | 无效尺寸   |
| ES_NPU_ERROR_BAD_VALUE         | 0xA00F6044 | 错误的数值  |
| ES_NPU_ERROR_DEV_NOT_FOUND     | 0xA00F6045 | 设备未找到  |
| ES_NPU_ERROR_DEV_MULTI_SET     | 0xA00F6046 | 多次设备设置 |
| ES_NPU_ERROR_DEV_OPEN_FAILED   | 0xA00F6047 | 设备打开失败 |
| ES_NPU_ERROR_MODEL_NOT_PRESENT | 0xA00F6048 | 模型未加载  |
| ES_NPU_ERROR_MODEL_NOT_FOUND   | 0xA00F6049 | 未找到模型  |
| ES_NPU_ERROR_CONTEXT_NOT_FOUND | 0xA00F604A | 上下文未找到 |
| ES_NPU_ERROR_CONTEXT_INVALID   | 0xA00F604B | 无效上下文  |
| ES_NPU_ERROR_STREAM_NOT_FOUND  | 0xA00F604C | 流未找到   |



| 参数                               | 值          | 描述            |
|----------------------------------|------------|---------------|
| ES_NPU_ERROR_STREAM_INVALID      | 0xA00F604D | 无效流           |
| ES_NPU_ERROR_TASK_NOT_FOUND      | 0xA00F604E | 任务未找到         |
| ES_NPU_ERROR_FILE_WRITE_FAIL     | 0xA00F604F | 文件写入失败        |
| ES_NPU_ERROR_FILE_READ_FAIL      | 0xA00F6050 | 文件读取失败        |
| ES_NPU_ERROR_FILE_OPREATION_FAIL | 0xA00F6051 | 文件操作失败        |
| ES_NPU_ERROR_END_OF_FILE         | 0xA00F6052 | 文件结束          |
| ES_NPU_ERROR_DIR_OPREATION_FAIL  | 0xA00F6053 | 目录操作失败        |
| ES_NPU_ERROR_END_OF_DIR_LIST     | 0xA00F6054 | 目录列表结束        |
| ES_NPU_ERROR_IOCTL_FAIL          | 0xA00F6055 | IOCTL 操作失败    |
| ES_NPU_ERROR_CREAT_EVENTFD_FAIL  | 0xA00F6056 | 创建 EventFD 失败 |
| ES_NPU_ERROR_LOAD_OP_FAILED      | 0xA00F6058 | 加载操作失败        |
| ES_NPU_ERROR_SUBMIT_TASK_FAILED  | 0xA00F6059 | 提交任务失败        |
| ES_NPU_ERROR_LOAD_SRAM_FAIL      | 0xA00F605A | 加载 SRAM 失败    |
| ES_NPU_ERROR_BAD_SRAM_SIZE       | 0xA00F605B | 错误的 SRAM 大小   |
| ES_NPU_ERROR_DSP_QUERY_FAILED    | 0xA00F605C | DSP 查询失败      |

3.6. ES\_SDK\_VERSION\_U

由主版本号, 次版本号, 修订号组成. 其中, 主版本号在高 32 位, 次版本号在高 16 位, 修订号在低位, 举例如下

| 值           | 描述                  |
|-------------|---------------------|
| 0x0         | 主版本号 0 次版本号 0 修订号 0 |
| 0x1         | 主版本号 0 次版本号 0 修订号 1 |
| 0x10000     | 主版本号 0 次版本号 1 修订号 0 |
| 0x100000000 | 主版本号 1 次版本号 0 修订号 0 |
| 0x100010001 | 主版本号 1 次版本号 1 修订号 1 |

3.7. 调试信息

3.7.1. 日志等级

| 日志等级       | 值 | 输出信息    |
|------------|---|---------|
| ES_LOG_ERR | 3 | 仅打印错误日志 |

| 日志等级          | 值 | 输出信息                    |
|---------------|---|-------------------------|
| ES_LOG_WARN   | 4 | 打印错误和警告的日志              |
| ES_LOG_NOTICE | 5 | 打印错误、警告及提示等级日志          |
| ES_LOG_INFO   | 6 | 信息级别，用于调试和开发阶段的详细信息     |
| ES_LOG_DBG    | 7 | 调试级别，提供更为详尽的信息，通常用于深度调试 |

3.7.2. Python 调试示例

```
# 启用详细日志
npu.set_log_level(level=6)

# 检查任务执行状态
task_state = task.state # 可能的值: PENDING, RUNNING, COMPLETED, FAILED
```

3.8. 使用注意事项

3.8.1. 资源管理

构造顺序：Device → Context → Stream  
析构顺序：Stream → Context → Device  
这个顺序确保了对象间的依赖关系被正确处理。

```
from es_py import npu

try:
    # 1. 选择设备
    device_id = 0
    npu.set_device(device_id)

    # 2. 创建 Context
    context = npu.create_context(device_id)
    npu.set_current_context(context)

    # 3. 创建 Stream
    stream = npu.create_stream()

    # 4. 使用 Stream 执行异步任务
    # ...

finally:
    # 5. 析构：逆序释放
    if 'stream' in locals():
        npu.destroy_stream(stream)
    if 'context' in locals():
        npu.destroy_context(context)
    npu.release_device(device_id)
```

3.8.2. 内存管理

- 对于需要 NPU 访问的 DMA 缓冲，必须调用 prepare() 进行 SMMU 映射
- 不再使用时立即调用 unprepare() 释放映射

- 分配和释放的内存块数量必须保持一致

```
# 准备内存映射
npu.prepare(dev_buf)
# ... 使用内存 ...
npu.unprepare(dev_buf)
```

### 3.8.3. 异步操作

- 使用 `submit_async()` 提交异步任务后，必须定期调用 `process_report()`
- `process_report()` 会检查完成的任务并触发回调函数
- 使用 `synchronize_stream()` 等待所有任务完成

```
while not all_tasks_done:
    ret = npu.process_report(stream, timeout_ms=100)
    if ret == 0:
        # 有任务完成，回调已被触发
        pass
```

### 3.8.4. 错误处理

建议始终检查返回值或捕获异常。

```
try:
    model_id = npu.load_model_from_file(model_path)
except Exception as e:
    print(f"模型加载失败: {e}")
    return False

# 同步方式也需要检查返回值
ret = npu.submit(tasks)
if ret != 0:
    print(f"任务提交失败，错误码: 0x{ret:x}")
```

### 3.8.5. 同步 vs 异步

- 使用同步接口时，不需要创建 Context 和 Stream
- 使用异步接口时，必须创建 Context 和 Stream 进行任务管理

```
# 同步方式 - 简单直接
npu.set_device(device_id)
model_id = npu.load_model_from_file(model_path)
npu.submit(tasks) # 阻塞直到完成

# 异步方式 - 需要管理 Context 和 Stream
npu.set_device(device_id)
context = npu.create_context(device_id)
stream = npu.create_stream()
npu.submit_async(tasks, stream)
npu.process_report(stream) # 处理完成回调
```

## 4. AcceleratorKit 开发指南

AcceleratorKit 封装了对于 NPU、DSP、HAE、GPU 等硬件的调用并将其导出成对应的计算加速功能接口。它包括一些较为复杂的前处理、后处理算子，图像处理，机器视觉等功能。AcceleratorKit 库提供了可在不同类型设备上执行的算子和功能，以便最大化发挥不同类型设备的计算能力。

### 4.1. 功能描述

AcceleratorKit 提供了如下表所示的常用计算加速算子供用户使用：

| 算子类型                | 加速接口功能                         | 运行设备 |
|---------------------|--------------------------------|------|
| cosDistance         | 计算两组数据之间的余弦距离                  | DSP  |
| argmax              | 进行 Argmax 计算，对输入的数据的指定维度进行最大排序 | DSP  |
| detectionOut        | 对检测网络的输出 feature map 做后处理      | DSP  |
| softmax             | 进行 Softmax 计算                  | DSP  |
| perspectiveAffine   | 对图像做透视变换                       | DSP  |
| warpAffine          | 对图像做仿射变换                       | DSP  |
| similarityTransform | 计算相似变换系数矩阵                     | CPU  |
| rgb2hsv             | 将 RGB 图像转换为 HSV 图像             | DSP  |
| sobel               | 计算图像在水平和垂直方向上的梯度，用于边缘提取。       | DSP  |
| houghlines          | 使用霍夫变换算法检测图像中的直线。              | DSP  |
| gaussian_blur       | 对输入图像应用高斯模糊。                   | DSP  |
| dilate              | 对图像进行膨胀操作。                     | DSP  |
| erode               | 对图像执行形态学腐蚀操作。                  | DSP  |
| dft                 | 对输入数据执行离散傅里叶变换。                | DSP  |
| rgb2gray            | 将图像从 RGB 转换为灰度                 | DSP  |
| cany                | 对图像进行边缘检测算法                    | DSP  |

### 4.2. C API 详解

#### 4.2.1. 函数

##### 4.2.1.1. ES\_AK\_GetVersion

【函数体】

```
ES_S32 ES_AK_GetVersion(ES_U64 *version)
```

【描述】

获取 Accelerator 数字版本信息。

【参数】

| 参数      | 描述              | 输入/输出                  |
|---------|-----------------|------------------------|
| version | Accelerator 版本号 | 输出，参见 ES_SDK_VERSION_U |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.2. ES\_AK\_GetVersionString

【函数体】

```
ES_S32 ES_AK_GetVersionString(  
    ES_CHAR *version,  
    ES_U32 maxSize)
```

【描述】

获取 Accelerator 字符串版本信息。

【参数】

| 参数      | 描述                    | 输入/输出 |
|---------|-----------------------|-------|
| version | Accelerator 版本号       | 输出    |
| maxSize | Accelerator 版本字符串最大长度 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.3. ES\_AK\_GetCapability

【函数体】

```
ES_S32 ES_AK_GetCapability(ES_AK_Capability_S *capability)
```

【描述】

获取 Accelerator 能力信息。

【参数】

| 参数         | 描述               | 输入/输出                                    |
|------------|------------------|------------------------------------------|
| capability | Accelerator 能力信息 | 输出，参见 <a href="#">ES_AK_Capability_S</a> |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.4. ES\_AK\_SetLogLevel

【函数体】

ES\_S32 ES\_AK\_SetLogLevel(ES\_U32 level)

【描述】

设置 Accelerator 运行时的打印日志级别。

【参数】

| 参数    | 描述                 | 输入/输出 |
|-------|--------------------|-------|
| level | Accelerator 打印日志等级 | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.5. ES\_AK\_Init

【函数体】

ES\_S32 ES\_AK\_Init()

【描述】

初始化设备资源，包括打开硬件设备，如 DSP。

【参数】

无。

【返回值】

| 返回值 | 描述 |
|-----|----|
| 0   | 成功 |

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.6. ES\_AK\_Deinit

【函数体】

```
ES_S32 ES_AK_Deinit()
```

【描述】

释放设备资源，并关闭硬件设备。

【参数】

无。

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.7. ES\_AK\_SetDevice

【函数体】

```
ES_S32 ES_AK_SetDevice (  
    const ES_AK_DEVICE_E* devices,  
    ES_U32 devNum)
```

【描述】

配置用于运算的设备，比如配置指定 DSP 和 GPU 用于运算。

【参数】

| 参数      | 描述          | 输入/输出 |
|---------|-------------|-------|
| devices | 运行计算需要的硬件设备 | 输入    |
| devNum  | 硬件设备配置数目    | 输入    |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.8. ES\_AK\_GetDevice

【函数体】

```
ES_S32 ES_AK_GetDevice (
    ES_AK_DEVICE_E* devices,
    ES_U32 devNum)
```

【描述】

获取当前的运行设备。

【参数】

| 参数      | 描述        | 输入/输出 |
|---------|-----------|-------|
| devices | 当前运算的硬件设备 | 输出    |
| devNum  | 硬件设备数目    | 输入    |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.9. ES\_AK\_DSP\_CosDistance

【函数体】

```
ES_S32 ES_AK_DSP_CosDistance(
    ES_AK_DEVICE_E deviceId,
    const ES_TENSOR_S *inputBase,
    const ES_TENSOR_S *inputQuery,
    ES_TENSOR_S *output)
```

【描述】

计算两组数据之间的余弦距离。

支持情况:

输入 float16，输出 float16。

输入 float16，输出 float32。

输入 float32，输出 float32。

【参数】

| 参数        | 描述            | 约束规格                                                                                                                                   |
|-----------|---------------|----------------------------------------------------------------------------------------------------------------------------------------|
| deviceId  | 设备 ID 值       | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                                                  |
| inputBase | 计算余弦距离的输入数据信息 | 形式为 inputBase[featureNum1, dim]，只支持二维数据运算，featureNum1 表示 inputBase 特征向量个数，dim 表示特征向量长度；<br>featureNum1 值域：[1, 512]<br>dim 值域：[1, 8192] |



| 参数         | 描述                                | 约束规格                                                                                                                             |
|------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| inputQuery | 计算余弦距离的输入数据信息                     | 形式为 inputQuery[featureNum2, dim]，只支持二维数据运算，featureNum2 表示 inputQuery 特征向量个数，dim 表示特征向量长度，与 inputBase 相同，featureNum2 值域：[1, 8192] |
| output     | 输出 InputBase 和 inputQuery 的余弦距离信息 | 形式为 output[featureNum1, featureNum2]                                                                                             |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.10. ES\_AK\_DSP\_PerspectiveAffine

【函数体】

```
ES_S32 ES_AK_DSP_PerspectiveAffine(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    ES_FLOAT M[AFFINE_MATRIX_SIZE],  
    ES_INTER_FLAG_E flag,  
    ES_BORDER_TYPES_E borderMode,  
    ES_U8 borderValue)
```

【描述】

对图像做透视变换。

支持情况:

输入 uint8，输出 uint8。

【参数】

| 参数       | 描述                        | 约束规格                                                                                                                                    |
|----------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| deviceId | 设备 ID 值                   | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                                                   |
| input    | 输入图像数据                    | 只支持四维数据运算，形式为 input[batch, height, width, channel]。<br>batch 值域：[1, 512]；<br>channel 只能为 3，即只支持 3 通道图像数据；<br>height, width 值域：[1, 8192] |
| output   | 用户指定输出图像维度，经计算输出透视变换后图像数据 | 只支持四维数据运算，形式为 output[batch, height, width, channel]。<br>batch 值域：[1, 512]；<br>channel 只能为 3；<br>height, width 值域：[1, 8192]              |
| M        | 透视变换矩阵                    | 形式为 M[m11, m12, m13, m21, m22, m23, m31, m32, m33]                                                                                      |
| flag     | 插值计算方法                    | 当前版本只支持 nearest 插值                                                                                                                      |

| 参数          | 描述                        | 约束规格                     |
|-------------|---------------------------|--------------------------|
| borderMode  | 描述边界填充规则                  | 当前只支持 BORDER_CONSTANT 模式 |
| borderValue | BORDER_CONSTANT 填充时，填充用的值 |                          |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.11. ES\_AK\_DSP\_WarpAffine

【函数体】

```
ES_S32 ES_AK_DSP_WarpAffine(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    ES_S32 channel_first,  
    ES_FLOAT M[AFFINE_MATRIX_SIZE],  
    ES_INTER_FLAG_E flag,  
    ES_BORDER_TYPES_E borderMode,  
    ES_U8 borderValue)
```

【描述】

对图像做仿射变换。

支持情况:

输入 uint8，输出 uint8；

输入 uint16，输出 uint16；

输入输出图像长宽比 [1/5, 5]。

【参数】

| 参数            | 描述                          | 约束规格                                                                                                     |
|---------------|-----------------------------|----------------------------------------------------------------------------------------------------------|
| deviceId      | 设备 ID 值                     | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                    |
| input         | 输入图像数据指针                    | 只支持四维数据：<br>batch 值域：[1]<br>Channel={1, 3}；<br>height, width 值域：详见附注                                     |
| output        | 用户指定输出图像维度，经计算输出仿射变换后图像数据指针 | 只支持四维数据：<br>batch 值域：[1]；<br>Channel={1, 3}；<br>height, width 值域：详见附注                                    |
| channel_first | 数据维度含义                      | 可选配置选项为：0、1。含义如下：<br>0：维度含义为 [batch, height, width, channel]；<br>1：维度含义为 [batch, channel, height, width] |

| 参数          | 描述                        | 约束规格                                             |
|-------------|---------------------------|--------------------------------------------------|
| M           | 仿射变换矩阵                    | 形式为 M[m11, m12, m13, m21, m22, m23] 缩放因子部分 >1/10 |
| flag        | 插值计算方法                    | 支持双线性插值：ES_INTER_LINEAR；支持最近邻插值：ES_INTER_NEAREST |
| borderMode  | 边界填充规则                    | 当前只支持 BORDER_CONSTANT 模式                         |
| borderValue | BORDER_CONSTANT 填充时，填充用的值 | 当前只支持填充 0                                        |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

提示

受硬件空间的约束，输入图像和输出图像的尺寸受到一定的限制，由于约束条件较为复杂，因此给出严格可以执行的尺寸范围较为困难。图像尺寸主要受到以下几个方面的限制：

- 图像位数（8bit/16bit）；
- 图像通道数（单通道/三通道）；
- 图像通道格式（NCHW/NHWC），不同的格式有一些额外的空间使用消耗；
- 输出图像尺寸；
- 输入输出图像长宽比；输出图像尺寸；
- 仿射变换矩阵，主要是放缩和旋转分量；

另外，输入图像的原始尺寸并无约束，受到约束的是输出图像在仿射变换矩阵 M\_inv（M 的逆矩阵）的作用下在输入图像中的有效区域的大小。以下给出一些经典条件下的示例，其余案例下的算子可执行性请关注算子的返回值。

经典实例如下：

1. uint8、单通道图像示例：

单通道图像不区分前后通道格式，尺寸约束一致。

input\_shape: “[1,2200,2200,1]”

output\_shape: “[1,2200,2200,1]”

M: 旋转角度 45° 放缩系数 1.0

2. uint8、channel\_first=1、三通道图像示例：

input\_shape: “[1,3,2200,2200]”

output\_shape: “[1,3,2200,2200]”

M: 旋转角度 45° 放缩系数 1.0

3. uint8、channel\_first=0、三通道图像示例：

input\_shape: “[1,2200,2200,3]”

output\_shape: “[1,1100,1100,3]”

M: 旋转角度 45° 放缩系数 1.0

4. uint16 图像示例：

在以上 uint8 图像上缩小尺寸到一半以内

5. 其他情况:

旋转 45°、135°，输出图在输入图中的对应的仿射变换的区域面积最大，其余角度对应的面积要小于此类情况，因此可以接受更大的输出图尺寸；

输出图相比输入图缩小（M 矩阵缩放分量<1）时，输出图在输入图中的对应的仿射变换的区域面积会相较不放缩时更大，因此接受的输出图尺寸会更小。

4.2.1.12. ES\_AK\_DSP\_DetectionOut

【函数体】

```
ES_S32 ES_AK_DSP_DetectionOut(
    ES_AK_DEVICE_E deviceId,
    const ES_TENSOR_S *inFeaturesPtr,
    ES_S32 inFeaturesNum,
    ES_TENSOR_S *outBoxInfo,
    ES_TENSOR_S *outBoxNum,
    ES_DET_NETWORK_E detectNet,
    const ES_DETECTION_OUT_CFG *cfg)
```

【描述】

对检测网络的输出 feature map 做后处理，输出满足条件的目标 class\_id 、score 以及检测框的坐标。本算子为 box\_decode 和 nms 的融合算子。

支持情况:

输入 float16, outBoxInfo 输出 float16, outBoxNum 输出 int32。

输入 float16, outBoxInfo 输出 float32, outBoxNum 输出 int32。

输入 int16, outBoxInfo 输出 float16, outBoxNum 输出 int32。

输入 int16, outBoxInfo 输出 float32, outBoxNum 输出 int32。

【参数】

| 参数            | 描述                                                        | 约束规格                                                                                                                                                                                                                                                                        |
|---------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| deviceId      | 设备 ID 值                                                   | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                                                                                                                                                                                       |
| inFeaturesPtr | 输入 feature 的 tensor 描述起始地址，该地址存放了 inFeaturesNum 个 feature |                                                                                                                                                                                                                                                                             |
| inFeaturesNum | 输入的 feature 个数                                            | 值域：[1, 9]                                                                                                                                                                                                                                                                   |
| outBoxInfo    | 输出的 box 信息数据指针                                            | 只支持二维数据，形式为 outBoxInfo [dim0, dim1]。其中 dim0 恒为 7，表示每个 box 存放的数据为 [batchId, classId, score, coord1, coord2, coord3, coord4]。用户根据这 7 个值可以找到对应的图片、物体类别、score 以及对应的 box 坐标。其中后 4 个值为 box 坐标，支持 xyxy、xywh 等多种坐标格式的输出，由用户通过 params 进行配置。dim1 取值无限制，为“输入 batch * maxBoxesPerBatch” |
| outBoxNum     | 输出每张图的 box 个数数据指针                                         | 为一维数据，宽度值域：[1, 512]                                                                                                                                                                                                                                                         |
| detectNet     | 检测模型的名称                                                   | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                                                                                                                                                                                       |
| cfg           | 指向用于检测输出参数的配置结构体的指针                                       | 详细信息参见 <a href="#">ES_DETECTION_OUT_CFG</a>                                                                                                                                                                                                                                 |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.13. ES\_AK\_DSP\_Argmax

【函数体】

```
ES_S32 ES_AK_DSP_Argmax(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    ES_TENSOR_S *outputIdx,  
    ES_S32 k,  
    ES_S32 axis)
```

【描述】

调用 DSP 设备进行 argmax 计算。

支持情况:

输入 float16，output 输出 float16，outputIdx 输出 uint16。

输入 float16，output 输出 float32，outputIdx 输出 uint16。

输入 float32，output 输出 float32，outputIdx 输出 uint16。

【参数】

| 参数        | 描述                                            | 约束规格                                                               |
|-----------|-----------------------------------------------|--------------------------------------------------------------------|
| deviceId  | 设备 ID 值                                       | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                              |
| input     | 输入数据的描述，包括 shape、类型和地址，地址内为待排序数值指针            | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]  |
| output    | 输出数据的描述，包括 shape、类型和地址，地址内为（最大）排序后的数值指针       | 只支持四维数据运算，形式为 output[batch, channel, height, width]，值域均为 [1, 8192] |
| outputIdx | 输出数据的描述，包括 shape、类型和地址，地址内为（最大）排序后的数值的原索引数据指针 |                                                                    |
| k         | 输出结果中排序维度的数据个数                                | 值域：[1, 32]                                                         |
| axis      | 指定排序的维度                                       | 当前只支持 channel 维度排序                                                 |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.14. ES\_AK\_DSP\_Softmax

【函数体】

```
ES_S32 ES_AK_DSP_Softmax(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    ES_FLOAT scaleIn)
```

【描述】

调用 DSP 设备进行 Softmax 计算。

支持情况:

输入 int8，输出 float16。

输入 int8，输出 float32。

输入 float16，输出 float16。

输入 int16，输出 float16。

【参数】

| 参数       | 描述                                           | 约束规格                                                               |
|----------|----------------------------------------------|--------------------------------------------------------------------|
| deviceId | 设备 ID 值                                      | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                              |
| input    | 输入数据的描述，包括 shape、类型和地址，地址内为待处理数据指针           | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]  |
| output   | 输出数据的描述，包括 shape、类型和地址，地址内为 softmax 处理后的数据指针 | 只支持四维数据运算，形式为 output[batch, channel, height, width]，值域均为 [1, 8192] |
| scaleIn  | 输入数据量化系数                                     | 值域：[0.00001, 1.0] 输入为 float 时需置 1.0                                |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.15. ES\_AK\_CPU\_SimilarityTransform

【函数体】

```
ES_S32 ES_AK_CPU_SimilarityTransform(  
    ES_AK_DEVICE_E deviceId,  
    ES_TENSOR_S *srcPoint,  
    ES_TENSOR_S *dstPoint,  
    ES_TENSOR_S *transformMat)
```

【描述】

计算相似变换矩阵。

支持情况:

输入 float32，输出 float32。

【参数】

| 参数           | 描述             | 约束规格                                                                                                                |
|--------------|----------------|---------------------------------------------------------------------------------------------------------------------|
| deviceId     | 设备 ID 值        | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                               |
| srcPoints    | 计算相似变换矩阵的源点指针  | 只支持二维数据运算，形式为 srcPoints[dim0, dim1]，dim0 表示源点的个数，当前只支持 5 个点，即 dim0 只能设为 5。dim1 表示源点坐标的维度，只能为 2，即每个点是形如 (x, y) 的二维坐标 |
| dstPoints    | 计算相似变换矩阵的目标点指针 | 同 srcPoints                                                                                                         |
| transformMat | 相似变换矩阵计算结果指针   | 形状为 [3, 2] 的矩阵                                                                                                      |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.16. ES\_AK\_DSP\_RGB2HSV

【函数体】

```
ES_S32 ES_AK_DSP_RGB2HSV (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * output,  
    const ES_RGB_CFG * config)
```

【描述】

将 RGB 图像转换为 HSV 图像。

支持情况:

输入 u8，输出 u8。

输入 s16，输出 s16。

【参数】

| 参数       | 描述      | 约束规格                                  |
|----------|---------|---------------------------------------|
| deviceId | 设备 ID 值 | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a> |

| 参数     | 描述                   | 约束规格                                                                                         |
|--------|----------------------|----------------------------------------------------------------------------------------------|
| input  | 输入的 RGB 张量           | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应 rgb 或 bgr 的数据维度 |
| output | 输出的 HSV 张量           | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应 hsv 的数据维度       |
| config | 用于 RGB 转 HSV 转换的配置参数 | 详细信息参见 <a href="#">ES_RGB_CFG</a>                                                            |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.17. ES\_AK\_DSP\_Sobel

【函数体】

```
ES_S32 ES_AK_DSP_Sobel (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * dstDx,  
    ES_TENSOR_S * dstDy,  
    ES_S32 kernelSize,  
    ES_S32 normalize)
```

【描述】

使用 Sobel 滤波器计算图像在 x 和 y 方向的梯度。

支持情况:

对于 kernel3x3:

输入 s16，输出 s32。

输入 s16，输出 s16。

输入 u8，输出 s16。

对于 kernel5x5:

输入 s16，输出 s16。

输入 u8，输出 s16。

【参数】

| 参数       | 描述       | 约束规格                                  |
|----------|----------|---------------------------------------|
| deviceId | 设备 ID 值  | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a> |
| input    | 源图像的输入张量 | 输入图像（单通道）                             |



| 参数         | 描述                            | 约束规格                                                                                                                                                                                           |
|------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dstDx      | x 方向的一阶导数据 dst_dx，与源数据大小保持一致； | H, W 对应 shape 为 [1, H_in, W_in, 1, 1]                                                                                                                                                          |
| dstDy      | y 方向的一阶导数据 dst_dy，与源数据大小保持一致  | H, W 对应 shape 为 [1, H_dx, W_dx, 1, 1] 和 [1, H_dy, W_dy, 1, 1]                                                                                                                                  |
| kernelSize | 计算 x 方向和 y 方向一阶导使用的 kernel    | 可以为 3 或者 5，分别代表 kernel3x3 或者 kernel5x5                                                                                                                                                         |
| normalize  | 可以为 0 或者 1，分别代表不使用范数或者使用范数    | 对于使用范数的情况，对于 kernel3x3，将计算结果除以 8；对于 kernel5x5，将计算结果除以 96。对于不使用范数的情况，可以认为是将计算结果除以 1。<br>对于 kernel3x3 的 s16s16，无论 normalize 为多少，总是将计算结果除以 8；对于 kernel5x5 的 s16s16，无论 normalize 为多少，总是将计算结果除以 96。 |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.18. ES\_AK\_DSP\_HoughLines

【函数体】

```
ES_S32 ES_AK_DSP_HoughLines (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * output,  
    ES_TENSOR_S * outputLineCount,  
    const ES_HOUGHLINE_CFG * config)
```

【描述】

使用霍夫变换算法检测直线。

支持情况:

输入 u8，输出 s32。

【参数】

| 参数              | 描述                 | 约束规格                                                              |
|-----------------|--------------------|-------------------------------------------------------------------|
| deviceId        | 设备 ID 值            | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                             |
| input           | 源图像的输入张量           | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192] |
| output          | 指向包含检测到的直线数据的输出张量。 | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192] |
| outputLineCount | 指向包含检测到直线数量的输出张量。  | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192] |

| 参数     | 描述             | 约束规格                                      |
|--------|----------------|-------------------------------------------|
| config | 指向霍夫变换配置参数的指针。 | 详细信息参见 <a href="#">ES_HOUGH_LINES_CFG</a> |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.19. ES\_AK\_DSP\_GaussianBlur

【函数体】

```
ES_S32 ES_AK_DSP_GaussianBlur (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * output,  
    ES_S32 kernelSize,  
    ES_FLOAT sigma,  
    ES_S32 truncating,  
    ES_S32 highPrecision)
```

【描述】

使用 DSP 对输入图像应用高斯模糊。

支持情况:

输入 u8，输出 u8。

输入 s16，输出 s16。

【参数】

| 参数            | 描述                    | 约束规格                                                               |
|---------------|-----------------------|--------------------------------------------------------------------|
| deviceId      | 设备 ID 值               | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                              |
| input         | 输入单通道图像/数据            | 对于 kernel=3x3, 5x5                                                 |
| output        | 输出单通道图像/数据            | 输出与输入数据类型相同; 输入需要先进行 padding, pad = kernel_size / 2                |
| kernelSize    | 指向包含检测到直线数量的输出张量。     | 只支持四维数据运算，形式为 input[batch, channel, height, width], 值域均为 [1, 8192] |
| sigma         | 指向霍夫变换配置参数的指针。        | 详细信息参见 <a href="#">ES_HOUGH_LINES_CFG</a>                          |
| truncating    | 代表是否向零截断小数位。          | 可以为 0 或者 1，为 1 代表向零截断小数位，若为 0，代表四舍五入。                              |
| highPrecision | 代表是否采用高精度计算（使用 7 位精度） | 可以为 0 或者 1，为 1 代表采用高精度计算。                                          |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.20. ES\_AK\_DSP\_Dilate

【函数体】

```
ES_S32 ES_AK_DSP_Dilate (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * output,  
    ES_DILATE_ERODE_KERNEL_CFG * cfg)
```

【描述】

使用 DSP 对图像进行膨胀操作。

支持情况:

输入 u8，输出 u8。

输入 s16，输出 s16。

【参数】

| 参数       | 描述         | 约束规格                                                                                         |
|----------|------------|----------------------------------------------------------------------------------------------|
| deviceId | 设备 ID 值    | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                        |
| input    | 输入单通道图像/数据 | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应 rgb 或 bgr 的数据维度 |
| output   | 输出单通道图像/数据 | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应 hsv 的数据维度       |
| cfg      | 指向核配置的指针   | 详细信息参见 <a href="#">ES_DILATE_ERODE_KERNEL_CFG</a>                                            |

【返回值】

|     |                           |
|-----|---------------------------|
| 返回值 | 描述                        |
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.21. ES\_AK\_DSP\_Erode

【函数体】

```
ES_S32 ES_AK_DSP_Erode (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * input,  
    ES_TENSOR_S * output,  
    ES_DILATE_ERODE_KERNEL_CFG * cfg)
```

【描述】

使用 DSP 对图像执行形态学腐蚀操作。

支持情况:

输入 u8，输出 u8。

输入 s16，输出 s16。

【参数】

| 参数       | 描述         | 约束规格                                                                             |
|----------|------------|----------------------------------------------------------------------------------|
| deviceId | 设备 ID 值    | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                            |
| input    | 输入单通道图像/数据 | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应数据维度 |
| output   | 输出单通道图像/数据 | 只支持四维数据运算，形式为 input[batch, channel, height, width]，值域均为 [1, 8192]；channel 对应数据维度 |
| cfg      | 指向核配置的指针   | 详细信息参见 <a href="#">ES_DILATE_ERODE_KERNEL_CFG</a>                                |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.22. ES\_AK\_DSP\_DFT

【函数体】

```
ES_S32 ES_AK_DSP_DFT (  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S * inRe,  
    const ES_TENSOR_S * inIm,  
    ES_TENSOR_S * outRe,  
    ES_TENSOR_S * outIm,  
    ES_TENSOR_S * outNC,  
    ES_S8 normalized,  
    ES_S32 iterationCount)
```

【描述】

使用 DSP 对输入数据执行离散傅里叶变换 (DFT)。

支持情况:

输入 u8, 输出 s32。

【参数】

| 参数             | 描述                                       | 约束规格                                                                                                                                                                                                                                                                                                              |
|----------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| deviceId       | 设备 ID 值                                  | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a>                                                                                                                                                                                                                                                                             |
| inRe           | 时序信号, 输入实部                               | 长度: 16, 32, 64, 128, 256, 512, 1024                                                                                                                                                                                                                                                                               |
| inIm           | 时序信号, 输入虚部                               | 长度: 16, 32, 64, 128, 256, 512, 1024                                                                                                                                                                                                                                                                               |
| outRe          | 频域信号, 输出实部                               | 长度: 16, 32, 64, 128, 256, 512, 1024                                                                                                                                                                                                                                                                               |
| outIm          | 频域信号, 输出虚部                               | 长度: 16, 32, 64, 128, 256, 512, 1024                                                                                                                                                                                                                                                                               |
| outNC          | 各迭代的归一化数组                                | 长度:<br>(1) 当 Normalized=1<br>且 IterationCount=8 时, 长度为 8。<br>(2) 当 Normalized=0<br>或 IterationCount=1 时, 长度为 1。<br>(如果 Normalized=0、IterationCount=8 时, 8 次迭代<br>使用同一个归一化系数)<br>数据类型:<br>(1) 输入输出信号为 I32 时, 为 int8_t 数组;<br>(2) 输入输出信号为 S16 时, 为 int16_t 数组;<br>(3) 输入输出信号为 F32 时, 为 int8_t 数组, 此时该输出无<br>实际实用意义。 |
| normalized     | 对输出结果进行归一化处理, 使得输<br>出结果更加稳定或符合预期。       | int8_t, 0: 不做归一化 1: 做归一化。对于 F32 的输入输出类<br>型不支持做归一化。                                                                                                                                                                                                                                                               |
| iterationCount | 重复执行多个 FFT 操作 (等同于神<br>经网络模型中的 batch 概念) | int32_t, 仅支持=1 or =8                                                                                                                                                                                                                                                                                              |

【返回值】

| 返回值 | 描述                         |
|-----|----------------------------|
| 0   | 成功                         |
| 非 0 | 失败, 参见 <a href="#">错误码</a> |

4.2.1.23. ES\_AK\_DSP\_Rgb2Gray

【函数体】

```
ES_S32 ES_AK_DSP_Rgb2Gray(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    const ES_RGB_CONFIG *rgbCfg);
```

【描述】

使用 DSP 将图像格式从 RGB 转换为灰度 (GRAY)。

支持情况:

输入 u8, 输出 u8。

输入 s16，输出 s16。

【参数】

| 参数       | 描述             | 约束规格                                  |
|----------|----------------|---------------------------------------|
| deviceId | 设备 ID 值        | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a> |
| input    | 待转换的输入数据地址     | 输入数量 1，四维数据                           |
| output   | 存储转换后数据的输出地址   | 输出数量 1，四维数据                           |
| rgbCfg   | rgbCfg 算子的参数配置 | 详细参见 <a href="#">ES_CANNY_CONFIG</a>  |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.1.24. ES\_AK\_DSP\_Canny

【函数体】

```
ES_S32 ES_AK_DSP_Canny(  
    ES_AK_DEVICE_E deviceId,  
    const ES_TENSOR_S *input,  
    ES_TENSOR_S *output,  
    const ES_CANNY_CONFIG *cannyCfg);
```

【描述】

使用 DSP 实现边缘检测算法。

支持情况:

输入 u8，输出 u8。

输入 s16，输出 u8。

【参数】

| 参数       | 描述            | 约束规格                                  |
|----------|---------------|---------------------------------------|
| deviceId | 设备 ID 值       | 详细信息参见 <a href="#">ES_AK_DEVICE_E</a> |
| input    | 待转换的输入数据地址    | 输入数量 1，四维数据                           |
| output   | 存储转换后数据的输出地址  | 输出数量 1，四维数据                           |
| cannyCfg | canny 算子的参数配置 | 详细参见 <a href="#">ES_CANNY_CONFIG</a>  |

【返回值】

| 返回值 | 描述                        |
|-----|---------------------------|
| 0   | 成功                        |
| 非 0 | 失败，参见 <a href="#">错误码</a> |

4.2.2. 数据类型和数据结构

4.2.2.1. ES\_AK\_Capability\_S

【描述】

Accelerator 能力信息。

【定义】

```
typedef struct AK_Capability_S {
    ES_U64 reserved;
} ES_AK_Capability_S
```

【成员】

| 成员       | 描述   |
|----------|------|
| reserved | 预留字段 |

4.2.2.2. ES\_AK\_DEVICE\_E

【描述】

Accelerator 支持的硬件设备。

【定义】

```
typedef enum AK_DEVICE_E {
    ES_AK_DEV_DSP_0 = 0x0,
    ES_AK_DEV_DSP_1 = 0x1,
    ES_AK_DEV_DSP_2 = 0x2,
    ES_AK_DEV_DSP_3 = 0x3,
    ES_AK_DEV_DSP_4 = 0x4,
    ES_AK_DEV_DSP_5 = 0x5,
    ES_AK_DEV_DSP_6 = 0x6,
    ES_AK_DEV_DSP_7 = 0x7,
    ES_AK_DEV_HAE_0 = 0x8,
    ES_AK_DEV_HAE_1 = 0x9,
    ES_AK_DEV_GPU_0 = 0xa,
    ES_AK_DEV_GPU_1 = 0xb,
    ES_AK_DEV_DIE0_CPU = 0xc,
    ES_AK_DEV_DIE1_CPU = 0xd,
    ES_AK_DEV_BUTT
} ES_AK_DEVICE_E
```

【成员】

| 成员              | 描述      |
|-----------------|---------|
| ES_AK_DEV_DSP_0 | DSP0 设备 |
| ES_AK_DEV_DSP_1 | DSP1 设备 |
| ES_AK_DEV_DSP_2 | DSP2 设备 |
| ES_AK_DEV_DSP_3 | DSP3 设备 |
| ES_AK_DEV_DSP_4 | DSP4 设备 |

| 成员                 | 描述       |
|--------------------|----------|
| ES_AK_DEV_DSP_5    | DSP5 设备  |
| ES_AK_DEV_DSP_6    | DSP6 设备  |
| ES_AK_DEV_DSP_7    | DSP7 设备  |
| ES_AK_DEV_HAE_0    | HAE0 设备  |
| ES_AK_DEV_HAE_1    | HAE1 设备  |
| ES_AK_DEV_GPU_0    | GPU0 设备  |
| ES_AK_DEV_GPU_1    | GPU1 设备  |
| ES_AK_DEV_DIE0_CPU | DIE0 CPU |
| ES_AK_DEV_DIE1_CPU | DIE1 CPU |
| ES_AK_DEV_BUTT     | 最大设备数    |

4.2.2.3. ES\_TENSOR\_S

【描述】

Tensor 结构参数。

【定义】

```
typedef struct {
    union {
        ES_DEV_BUF_S pData;
        void *hostBuf;
    };
    ES_DATA_PRECISION_E dataType;
    ES_U32 shapeDim;
    ES_U32 shape[MAX_DIM_CNT];
} ES_TENSOR_S
```

【成员】

| 成员       | 描述                                                                                                                                                                                                                                                                                                                  |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dataType | Tensor 的数据类型，ES_DATA_PRECISION_E 枚举有：<br>ES_PRECISION_INT8 = 1,<br>ES_PRECISION_UINT8 = 2,<br>ES_PRECISION_INT16 = 3,<br>ES_PRECISION_UINT16 = 4,<br>ES_PRECISION_INT32 = 5,<br>ES_PRECISION_UINT32 = 6,<br>ES_PRECISION_INT64 = 7,<br>ES_PRECISION_UINT64 = 8,<br>ES_PRECISION_FP16 = 9,<br>ES_PRECISION_FP32 = 10 |
| pData    | device 侧的 buffer 起始地址                                                                                                                                                                                                                                                                                               |
| hostBuf  | cpu 侧的 buffer 起始地址                                                                                                                                                                                                                                                                                                  |
| shapeDim | 描述 tensor 的真实维度个数                                                                                                                                                                                                                                                                                                   |
| shape    | 长度为 MAX_DIM_CNT 的数组，按顺序存放 tensor 每个维度的宽度信息。<br>MAX_DIM_CNT 值为 6                                                                                                                                                                                                                                                     |



#### 4.2.2.4. ES\_DET\_NETWORK\_E

##### 【描述】

detectionOut 算子支持的检测网络类型。

##### 【定义】

```
typedef enum {  
    yolov3_u = 0,  
    yolov3_d = 1,  
    yolov4 = 2,  
    yolov5 = 3,  
    yolov7 = 4,  
    yolov8 = 5,  
    yolo5face = 6,  
    ppyoloe = 7,  
    damo_yolo = 8,  
    yolov6 = 9,  
    yolov9 = 10,  
    yolov11 = 11,  
    yolov8pose = 12,  
    yolov10 = 13,  
    ssd = 20,  
    frcnn = 30,  
} ES_DET_NETWORK_E;
```

##### 提示

yolov3\_d 指的是 darknet 版，原始的 yolo v3 版本；yolov3\_u 指的是 ultralytics 版，是目前使用最为广泛的版本。用户根据自己模型的来源，自行选择对应的版本，如果不确定模型出处，建议首先尝试配置 yolov3\_u 进行测试。

#### 4.2.2.5. ES\_NMS\_METHOD\_E

##### 【描述】

detectionOut 算子支持的 NMS 计算方法。

##### 【定义】

```
typedef enum {  
    HARD_NMS,  
    SOFT_NMS_GAUSSIAN,  
    SOFT_NMS_LINEAR  
} ES_NMS_METHOD_E
```

#### 4.2.2.6. ES\_IOU\_METHOD\_E

##### 【描述】

detectionOut 算子支持的 iou 方法。

##### 【定义】

```
typedef enum {  
    IOU,  
    GIOU,  
    DIOU  
} ES_IOU_METHOD_E
```

#### 4.2.2.7. ES\_BOX\_TYPE\_E

##### 【描述】

detectionOut 算子支持的输出 box 格式。

##### 【定义】

```
typedef enum {  
    XminYminXmaxYmax,  
    XminYminWH,  
    YminXminYmaxXmax,  
    XmidYmidWH  
} ES_BOX_TYPE_E
```

#### 4.2.2.8. ES\_INTER\_FLAG\_E

##### 【描述】

描述插值方式的枚举类型。

##### 【定义】

```
typedef enum {  
    ES_INTER_NEAREST = 0,  
    ES_INTER_LINEAR = 1,  
    ES_INTER_AREA = 2,  
    ES_INTER_CUBIC = 3,  
    ES_INTER_LANCZOS4 = 4,  
    ES_INTER_NEAREST_EXACT = 6,  
    ES_INTER_STRETCH = 20,  
    ES_INTER_FILTER = 21  
} ES_INTER_FLAG_E
```

#### 4.2.2.9. ES\_BORDER\_TYPES\_E

##### 【描述】

描述边界填充方式的枚举类型。

##### 【定义】

```
typedef enum {  
    BORDER_CONSTANT = 0,  
    BORDER_REPLICATE = 1,  
    BORDER_REFLECT = 2,  
    BORDER_WRAP = 3,  
    BORDER_REFLECT_101 = 4,  
    BORDER_TRANSPARENT = 5,  
    BORDER_ISOLATED = 16
```

```
} ES_BORDER_TYPES_E
```

4.2.2.10. ES\_DETECTION\_OUT\_CFG

【描述】

检测输出参数配置的结构体。

【定义】

```
typedef struct {
    ES_S32 anchorsNum;
    ES_FLOAT anchorScale[MAX_TOTAL_ANCHORS_NUM * 2];
    ES_S32 imgH;
    ES_S32 imgW;
    ES_S32 clsNum;
    ES_FLOAT inputScale[MAX_IN_TENSOR_NUM];
    ES_NMS_METHOD_E nmsMethod;
    ES_IOU_METHOD_E iouMethod;
    ES_BOX_TYPE_E outBoxType;
    ES_BOOL coordNorm;
    ES_S32 maxBoxesPerClass;
    ES_S32 maxBoxesPerBatch;
    ES_FLOAT scoreThreshold;
    ES_FLOAT iouThreshold;
    ES_FLOAT softNmsSigma;
    ES_FLOAT effecImgOffsetX;
    ES_FLOAT effecImgOffsetY
} ES_DETECTION_OUT_CFG
```

【成员】

| 成员               | 描述                                |
|------------------|-----------------------------------|
| anchorsNum       | 检测网络中使用的 anchor 框数量。              |
| anchorScale      | 用于存储 anchor 比例的数组，定义为宽度与高度比率的成对值。 |
| imgH             | 输入图像的高度（以像素为单位）。                  |
| imgW             | 输入图像的宽度（以像素为单位）。                  |
| clsNum           | 检测中的目标类别数量。                       |
| inputScale       | 输入张量归一化的比例数组。                     |
| nmsMethod        | 用于筛选重叠检测结果的非极大值抑制（NMS）方法。         |
| iouMethod        | NMS 计算中使用的交并比（IOU）方法。             |
| outBoxType       | 输出边界框的格式类型（例如：角点坐标或中心坐标）。         |
| coordNorm        | 标记边界框坐标是否归一化到 [0, 1] 的布尔标志。       |
| maxBoxesPerClass | 每个类别经过 NMS 后保留的最大边界框数量。           |
| maxBoxesPerBatch | 整个批次经过 NMS 后保留的最大边界框数量。           |
| scoreThreshold   | 保留检测结果的最小分数阈值。                    |
| iouThreshold     | 用于 NMS 中筛选重叠框的交并比（IOU）阈值。         |
| softNmsSigma     | Soft NMS 中调整重叠框分数的 Sigma 值。       |
| effecImgOffsetX  | 调整边界框坐标时，X 方向的有效偏移量。              |
| effecImgOffsetY  | 调整边界框坐标时，Y 方向的有效偏移量。              |

4.2.2.11. ES\_RGB\_CFG

【描述】

RGB 转 HSV 转换的配置参数。

【定义】

```
typedef struct {
    ES_S16 rgbFmt; /* 1: RGB format; 0: BGR format */
    ES_S16 isAlign; /* 1: align method; 0: not align method */
    ES_S32 width; /* Image width in pixels */
    ES_S32 height; /* Image height in pixels */
} ES_RGB_CFG;
```

【成员】

| 成员      | 描述                              |
|---------|---------------------------------|
| rgbFmt  | RGB 格式标志，1 表示 RGB，0 表示 BGR。     |
| isAlign | 对齐模式，1 表示采用“对齐模式”，0 表示采用不对齐模式”。 |
| width   | 实际图像宽度。                         |
| height  | 实际图像高度。                         |

4.2.2.12. ES\_HOUGH\_LINES\_CFG

【描述】

霍夫变换配置参数。

【定义】

```
typedef struct {
    ES_DOUBLE rho;
    ES_DOUBLE theta;
    ES_S32 threshold;
    ES_DOUBLE minTheta;
    ES_DOUBLE maxTheta;
    ES_S16 width;
    ES_S16 height;
    ES_S16 inputFormat;
} ES_HOUGH_LINES_CFG;
```

【成员】

| 成员        | 描述                            |
|-----------|-------------------------------|
| rho       | 累加器的距离分辨率（以像素为单位， $\rho$ ）。   |
| theta     | 累加器的角度分辨率（以弧度为单位， $\theta$ ）。 |
| threshold | 检测直线所需的最小交点数（霍夫累加器中的投票数）。     |
| minTheta  | 检测直线的最小角度（以弧度为单位）。            |
| maxTheta  | 检测直线的最大角度（以弧度为单位）。            |
| width     | 图像宽度值。                        |

| 成员          | 描述                 |
|-------------|--------------------|
| height      | 图像高度值。             |
| inputFormat | 输入格式，1: YUV，其他值无效。 |

4.2.2.13. ES\_DILATE\_ERODE\_KERNEL\_CFG

【描述】

膨胀/腐蚀核尺寸形状的配置参数。

【定义】

```
typedef struct {
    ES_S32 kWidth;
    ES_S32 kHeight;
    ES_S32 pattern;
    ES_U8 mask[MAX_DILATE_ERODE_KERNEL_MASK];
} ES_DILATE_ERODE_KERNEL_CFG;
```

【成员】

| 成员      | 描述                                                                                                     |
|---------|--------------------------------------------------------------------------------------------------------|
| kWidth  | 核尺寸，宽。                                                                                                 |
| kHeight | 核尺寸，高。                                                                                                 |
| pattern | 表示特殊形态的 kernel。<br>pattern = 0: 矩形（正方形）<br>pattern =1: cross<br>pattern =2: disk<br>pattern = 3: 用户自定义 |
| mask    | 自定义 kernel 形态，pattern 的值设置为 3 时，mask 必须设置。                                                             |

4.2.2.14. ES\_RGB\_CONFIG

【描述】

RGB 转 GRAY 转换的配置参数。

【定义】

```
typedef struct {
    ES_U32 rgbFmt; /**< Format of input tensor, 0:bgr; 1:rgb. */
    ES_U32 width; /**< The width of the input image. */
    ES_U32 height; /**< The height of the input image. */
    ES_S32 thresholdMin; /**< Minimum value of image pixels. */
    ES_S32 thresholdMax; /**< Maximum value of image pixels. */
} ES_RGB_CONFIG;
```

【成员】

| 成员     | 描述                          |
|--------|-----------------------------|
| rgbFmt | RGB 格式标志，1 表示 RGB，0 表示 BGR。 |

| 成员           | 描述       |
|--------------|----------|
| width        | 实际图像宽度。  |
| height       | 实际图像高度。  |
| thresholdMin | 图像像素下限值。 |
| thresholdMax | 图像像素上限值。 |

4.2.2.15. ES\_CANNY\_CONFIG

【描述】

边缘检测算法的配置参数。

【定义】

```
typedef struct {
    ES_S32 lowThreshold;
    ES_S32 highThreshold;
    ES_S32 traceEdges;
    ES_S32 apertureSize;
    ES_S32 l2Gradient;
} ES_CANNY_CONFIG;
```

【成员】

| 成员            | 描述                          |
|---------------|-----------------------------|
| lowThreshold  | 滞后阈值的下限，低于此值的像素将被丢弃。        |
| highThreshold | 滞后阈值的上限，高于此值的像素被视为强边缘。      |
| traceEdges    | 是否启用边缘连接。                   |
| apertureSize  | Sobel 算子的卷积核大小。             |
| l2Gradient    | 选择范数计算梯度幅值：1 表示 L2，0 表示 L1。 |

4.3. Python API 详解

4.3.1. 版本信息和系统配置

4.3.1.1. 获取版本号

```
from es_py import ak, common

# 获取 AK 版本号（数值格式）
version = ak.ak_get_version()

# 获取 AK 版本字符串
version_string = ak.get_version_string(max_size=256)

# 获取 AK 能力信息
capability = ak.get_capability()
```

【参数说明（ak\_get\_version）】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【参数说明 (get\_version\_string)】

| 参数       | 类型  | 描述        | 输入/输出 |
|----------|-----|-----------|-------|
| max_size | int | 版本字符串最大长度 | 输入    |

【参数说明 (get\_capability)】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

- ak\_get\_version：返回 int 类型的版本号
- get\_version\_string：返回 str 类型的版本字符串
- get\_capability：返回 common.ES\_AK\_Capability\_S 对象

【异常】

调用失败时抛出 Exception 异常

4.3.1.2. 设置日志级别

```
# 设置 AK 日志级别
ak.set_log_level(level=2) # 0: 无日志, 1: 错误, 2: 警告, 3: 信息
```

【参数说明】

| 参数    | 类型  | 描述                                | 输入/输出 |
|-------|-----|-----------------------------------|-------|
| level | int | 日志级别: 0=无, 1=错误, 2=警告, 3=信息, 4=调试 | 输入    |

【返回值】

int：0 表示成功

【说明】

- 用于控制 AcceleratorKit 运行时的日志输出详细程度
- 调试时建议使用 level=3 或 4

4.3.1.3. 初始化和释放

```
# 初始化 AK (打开硬件设备如 DSP)
ak.init()

# ... 使用 AK ...

# 释放 AK (关闭硬件设备)
ak.deinit()
```

【参数说明 (init)】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【参数说明 (deinit)】

| 参数 | 类型 | 描述 | 输入/输出 |
|----|----|----|-------|
| 无  | -  | -  | -     |

【返回值】

int：0 表示成功

【异常】

初始化失败时抛出 Exception 异常

【说明】

- init() 必须在调用其他 AK 接口前调用
- deinit() 必须在所有操作完成后调用
- 两者必须成对使用

4.3.2. 设备管理

4.3.2.1. 配置计算设备

```
from es_py import ak, common

# 配置用于运算的设备 (可配置多个 DSP 或 GPU)
devices = [common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0, common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_1]
ak.set_device(devices)

# 获取当前设备配置。dev_num 表示期望读取的设备数量。
current_devices = ak.get_device(dev_num=2)
```

【参数说明 (set\_device)】

| 参数      | 类型                          | 描述                                    | 输入/输出 |
|---------|-----------------------------|---------------------------------------|-------|
| devices | List[common.ES_AK_DEVICE_E] | 设备 ID 列表，取值见 common.ES_AK_DEVICE_E 枚举 | 输入    |



【参数说明 (get\_device)】

| 参数      | 类型  | 描述        | 输入/输出 |
|---------|-----|-----------|-------|
| dev_num | int | 期望读取的设备数量 | 输入    |

【返回值】

- `set_device`：返回 `int`，0 表示成功
- `get_device`：返回 `List[common.ES_AK_DEVICE_E]`，当前配置的设备列表

【异常】

- 设备 ID 无效时抛出 `Exception` 异常
- 设备不可用时抛出 `Exception` 异常

【说明】

- 必须在调用计算算子前配置设备
- 可以配置多个相同或不同类型的设备
- 不同算子可能支持不同的设备类型

4.3.3. DSP 加速算子

本节示例按当前 wheel 和板卡验证结果分级。代码块用于说明接口调用形式和已验证的调用路径，不作为数学或数值正确性验证样例。

- `dsp_argmax`、`dsp_warp_affine`、`dsp_detection_out` 已完成调用路径验证，可作为接口调用参考。
- `dsp_softmax`、`dsp_cos_distance` 已确认接口可调用；文档仅给调用形式和参数说明。
- `dsp_perspective_affine` 当前仅保留 API 说明；板卡上尚未确认稳定可运行的 Python 用例。

4.3.3.1. 余弦距离计算

```
# API 调用形式：计算两组特征之间的余弦距离
result = ak.dsp_cos_distance(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
    input_base=base_tensor, # [featureNum1, dim]
    input_query=query_tensor, # [featureNum2, dim]
    output=output_tensor # [featureNum1, featureNum2]
)
```

【参数说明】

| 参数          | 类型                 | 描述      | 约束规格                                                                        |
|-------------|--------------------|---------|-----------------------------------------------------------------------------|
| device_id   | int                | 设备 ID 值 | 取值见<br>common.ES_AK_DEVICE_E<br>枚举                                          |
| input_base  | common.ES_TENSOR_S | 基准特征向量  | 形式为 [featureNum1, dim]，二维数据<br>featureNum1 值域：[1, 512]<br>dim 值域：[1, 8192]  |
| input_query | common.ES_TENSOR_S | 查询特征向量  | 形式为 [featureNum2, dim]<br>featureNum2 值域：[1, 8192]<br>dim 必须与 input_base 相同 |

| 参数     | 类型                 | 描述       | 约束规格                           |
|--------|--------------------|----------|--------------------------------|
| output | common.ES_TENSOR_S | 输出余弦距离矩阵 | 形式为 [featureNum1, featureNum2] |

【支持的数据类型】

- 输入 float16，输出 float16
- 输入 float16，输出 float32
- 输入 float32，输出 float32

【返回值】

int：0 表示成功

【异常】

- 参数超出范围时抛出 Exception 异常
- 设备不支持时抛出 Exception 异常

【说明】

- 当前接口已确认导出并可进入算子执行路径。
- 文档仅给 API 调用形式；具体结果语义需结合有效输入数据、内存读写流程和业务场景验证。

4.3.3.2. 仿射变换

```
# 调用示例：UINT8 NHWC 图像仿射变换
ak.init()
ak.set_device([common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0])

matrix = [
    1.0, 0.0, 0.0,
    0.0, 1.0, 0.0,
    0.0, 0.0, 1.0,
]

result = ak.dsp_warp_affine(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
    input=input_image,    # common.ES_TENSOR_S, UINT8, NHWC: [1, height, width, channel]
    output=output_image,  # common.ES_TENSOR_S, UINT8, NHWC
    m=matrix,             # 3x3 矩阵，当前 Python 绑定要求 9 个 float
    channel_first=0,       # 0: NHWC, 1: NCHW
    flag=common.ES_INTER_FLAG_E.ES_INTER_LINEAR,
    border_mode=common.ES_BORDER_TYPES_E.BORDER_CONSTANT,
    border_value=0
)

ak.deinit()
```

【参数说明】

| 参数        | 类型  | 描述      | 约束规格                         |
|-----------|-----|---------|------------------------------|
| device_id | int | 设备 ID 值 | 取值见 common.ES_AK_DEVICE_E 枚举 |

| 参数            | 类型                       | 描述     | 约束规格                                                                          |
|---------------|--------------------------|--------|-------------------------------------------------------------------------------|
| input         | common.ES_TENSOR_S       | 输入图像数据 | 四维数据<br>batch 值域：[1]<br>channel：{1, 3}<br>height, width 值域：详见附注               |
| output        | common.ES_TENSOR_S       | 输出图像数据 | 四维数据<br>batch 值域：[1]<br>channel：{1, 3}<br>height, width 值域：详见附注               |
| channel_first | int                      | 数据维度含义 | 0：[batch, height, width, channel]<br>1：[batch, channel, height, width]        |
| m             | List[float]              | 仿射变换矩阵 | 当前 Python 绑定为 FixedSize(9)，按 [m11, m12, m13, m21, m22, m23, m31, m32, m33] 传入 |
| flag          | common.ES_INTER_FLAG_E   | 插值计算方法 | ES_INTER_LINEAR：双线性插值<br>ES_INTER_NEAREST：最近邻插值                               |
| border_mode   | common.ES_BORDER_TYPES_E | 边界填充规则 | 当前推荐使用<br>common.ES_BORDER_TYPES_E.BORDER_CONSTANT                            |
| border_value  | int                      | 边界填充值  | 当前仅支持填充 0                                                                     |

【支持的数据类型】

- 输入 uint8，输出 uint8
- 输入 uint16，输出 uint16
- 输入输出图像长宽比：[1/5, 5]

【返回值】

int：0 表示成功

【异常】

- 参数超出范围时抛出 Exception 异常
- 图像尺寸不满足约束时抛出 Exception 异常

【附注】

受硬件空间约束，输入输出图像尺寸受以下因素限制：

- 图像位数（8bit/16bit）
- 图像通道数（单通道/三通道）
- 图像通道格式（NCHW/NHWC）
- 输入输出图像长宽比
- 仿射变换矩阵（主要是缩放和旋转分量）

【典型尺寸示例】

1. uint8 单通道：[1,2200,2200,1] → [1,2200,2200,1]（旋转 45°，缩放 1.0）
2. uint8 三通道 NCHW：[1,3,2200,2200] → [1,3,2200,2200]（旋转 45°，缩放 1.0）
3. uint8 三通道 NHWC：[1,2200,2200,3] → [1,1100,1100,3]（旋转 45°，缩放 1.0）
4. uint16 图像：在 uint8 尺寸基础上缩小到一半以内

【板卡验证说明】

- 当前 wheel 中 common.AFFINE\_MATRIX\_SIZE 为 9，6 元矩阵会在 Python 绑定层报参数不匹配。
- 已验证 UINT8 + NHWC + channel\_first=0 + 9 元矩阵 + BORDER\_CONSTANT 调用路径可返回 0。
- NCHW 形式不作为推荐调用示例。

4.3.3.3. 透视变换

```
# API 调用形式：透视变换
result = ak.dsp_perspective_affine(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
    input=input_image,
    output=output_image,
    m=perspective_matrix, # 9 个 float, 3x3 透视矩阵
    flag=common.ES_INTER_FLAG_E.ES_INTER_NEAREST,
    border_mode=common.ES_BORDER_TYPES_E.BORDER_CONSTANT,
    border_value=0
)
```

【参数说明】

| 参数           | 类型                       | 描述      | 约束规格                                              |
|--------------|--------------------------|---------|---------------------------------------------------|
| device_id    | int                      | 设备 ID 值 | 取值见 common.ES_AK_DEVICE_E 枚举                      |
| input        | common.ES_TENSOR_S       | 输入图像数据  | 四维图像张量，具体 layout 需与底层算子要求一致                       |
| output       | common.ES_TENSOR_S       | 输出图像数据  | 四维图像张量，需预先分配有效缓冲区                                 |
| m            | List[float]              | 透视变换矩阵  | 形式为 [m11, m12, m13, m21, m22, m23, m31, m32, m33] |
| flag         | common.ES_INTER_FLAG_E   | 插值计算方法  | 当前仅支持 ES_INTER_NEAREST（最近邻插值）                     |
| border_mode  | common.ES_BORDER_TYPES_E | 边界填充规则  | 当前推荐使用 common.ES_BORDER_TYPES_E.BORDER_CONSTANT   |
| border_value | int                      | 边界填充值   | BORDER_CONSTANT 填充使用的值                            |

【支持的数据类型】

输入 uint8，输出 uint8

【返回值】

int：0 表示成功

【异常】

参数超出范围时抛出 Exception 异常

【说明】

- 当前 Python 绑定已导出该接口，矩阵参数为 9 个 float。
- 板卡验证中，多组最小 tensor case 仍出现 ES\_AK\_ERROR\_SUBMIT\_TASK\_FAILED 或超时；本节仅保留 API 说明，不作为已验证可运行示例。

4.3.3.4. Softmax 激活

```
# API 调用形式：执行 Softmax 激活
result = ak.dsp_softmax(
```

```
device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
input=input_data,      # [batch, channel, height, width]
output=output_data,
scale_in=1.0           # 输入量化系数
)
```

【参数说明】

| 参数        | 类型                 | 描述       | 约束规格                                                  |
|-----------|--------------------|----------|-------------------------------------------------------|
| device_id | int                | 设备 ID 值  | 取值见 common.ES_AK_DEVICE_E 枚举                          |
| input     | common.ES_TENSOR_S | 输入数据     | 四维：[batch, channel, height, width]<br>各维度值域：[1, 8192] |
| output    | common.ES_TENSOR_S | 输出数据     | 四维：[batch, channel, height, width]<br>各维度值域：[1, 8192] |
| scale_in  | float              | 输入数据量化系数 | 值域：[0.00001, 1.0]<br>输入为 float 时需置 1.0                |

【支持的数据类型】

- 输入 int8，输出 float16
- 输入 int8，输出 float32
- 输入 float16，输出 float16
- 输入 int16，输出 float16

【返回值】

int：0 表示成功

【异常】

参数超出范围时抛出 Exception 异常

【说明】

- 当前接口已确认导出，多组 INT8/INT16/FP16 输入组合可返回 0。
- 文档仅给 API 调用形式。请结合模型真实输出、量化参数和读取方式验证结果语义。

4.3.3.5. Argmax 计算

```
ak.init()
ak.set_device([common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0])

# 调用示例：FP32 输入，FP32 输出最大值，UINT16 输出索引
result = ak.dsp_argmax(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
    input=input_data,      # common.ES_TENSOR_S, FP32, shape [1, C, 1, 1]
    output=output_values,  # common.ES_TENSOR_S, FP32, shape [1, k, 1, 1]
    output_idx=output_indices, # common.ES_TENSOR_S, UINT16, shape [1, k, 1, 1]
    k=1,
    axis=1                 # 在 channel 维度上进行排序
)

ak.deinit()
```

【参数说明】

| 参数         | 类型                 | 描述             | 约束规格                                                    |
|------------|--------------------|----------------|---------------------------------------------------------|
| device_id  | int                | 设备 ID 值        | 取值见 common.ES_AK_DEVICE_E 枚举                            |
| input      | common.ES_TENSOR_S | 输入数据           | 四维: [batch, channel, height, width]<br>各维度值域: [1, 8192] |
| output     | common.ES_TENSOR_S | 输出排序后的数值       | 四维: [batch, channel, height, width]<br>各维度值域: [1, 8192] |
| output_idx | common.ES_TENSOR_S | 输出排序后的索引       | 与 output shape 相同                                       |
| k          | int                | 输出结果中排序维度的数据个数 | 值域: [1, 32]                                             |
| axis       | int                | 指定排序的维度        | 当前仅支持 channel 维度排序                                      |

【支持的数据类型】

- 输入 float16, output 输出 float16, outputIdx 输出 uint16
- 输入 float16, output 输出 float32, outputIdx 输出 uint16
- 输入 float32, output 输出 float32, outputIdx 输出 uint16

【返回值】

int：0 表示成功

【异常】

参数超出范围时抛出 Exception 异常

【板卡验证说明】

- 已验证 FP32 -> FP32 + UINT16 index、axis=1、k=1 调用路径可返回 0。
- output\_idx 推荐使用 common.ES\_DATA\_PRECISION\_E.ES\_PRECISION\_UINT16。

4.3.3.6. 检测输出后处理

```
# 调用示例：对 NPU YOLOv3 模型真实输出做后处理
# in_features 来自 NPU 输出 buffer 封装的 common.ES_TENSOR_S 列表，并按 H*W 升序排列：
# [1,255,13,13,1]、[1,255,26,26,1]、[1,255,52,52,1]
result = ak.dsp_detection_out(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
    in_features=ak_output_tensors,
    out_box_info=box_output,      # shape [1, 7, 1, maxBoxesPerBatch, 1], FP32
    out_box_num=box_count,       # shape [1, 1, 1, 1, 1], INT32
    detect_net=common.ES_DET_NETWORK_E.yolov3_u,
    cfg=detection_config
)
```

【参数说明】

| 参数          | 类型                       | 描述                   | 约束规格                         |
|-------------|--------------------------|----------------------|------------------------------|
| device_id   | int                      | 设备 ID 值              | 取值见 common.ES_AK_DEVICE_E 枚举 |
| in_features | List[common.ES_TENSOR_S] | 输入 feature tensor 列表 | feature 个数值域: [1, 9]         |

| 参数           | 类型                          | 描述            | 约束规格                                                                                                                          |
|--------------|-----------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------|
| out_box_info | common.ES_TENSOR_S          | 输出 box 信息     | 二维: [dim0, dim1]<br>dim0 恒为 7: [batchId, classId, score, coord1, coord2, coord3, coord4]<br>dim1: 输入 batch × maxBoxesPerBatch |
| out_box_num  | common.ES_TENSOR_S          | 输出每张图的 box 个数 | 一维, 宽度值域: [1, 512]                                                                                                            |
| detect_net   | common.ES_DET_NETWORK_E     | 检测模型名称        | 取值见 common.ES_DET_NETWORK_E 枚举                                                                                                |
| cfg          | common.ES_DETECTION_OUT_CFG | 检测输出参数配置      | 使用 common.ES_DETECTION_OUT_CFG 结构                                                                                             |

【支持的数据类型】

- 输入 float16, outBoxInfo 输出 float16, outBoxNum 输出 int32
- 输入 float16, outBoxInfo 输出 float32, outBoxNum 输出 int32
- 输入 int16, outBoxInfo 输出 float16, outBoxNum 输出 int32
- 输入 int16, outBoxInfo 输出 float32, outBoxNum 输出 int32

【返回值】

int : 0 表示成功

【异常】

参数超出范围时抛出 Exception 异常

【说明】

- 本算子为 box\_decode 和 nms 的融合算子。
- 输出 box 坐标格式由 config 中的 outBoxType 指定 (xyxy/xywh 等)。
- 该接口应接入 NPU 模型真实输出 tensor; 不建议使用占位 feature map 作为文档测试依据。
- YOLOv3 sample 中, 输入 feature tensor 的 dataType 按后处理配置为 common.ES\_DATA\_PRECISION\_E.ES\_PRECISION\_INT16, 输出 box 为 FP32, 输出数量为 INT32。common.ES\_DETECTION\_OUT\_CFG 的 anchorScale、inputScale、阈值等参数应与模型后处理配置一致, 例如 sample 中的 es\_yolov3\_postprocess.json。

4.3.4. CPU 加速算子

4.3.4.1. 相似度变换计算

```
from es_py import ak, common, memory

# 调用示例: CPU host buffer 输入输出。set_hostBuf() 需要传 memory_map 返回的 capsule。
CACHE = common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE
zone = "mmz_nid_0_part_0"

# src_point、dst_point 为 FP32 host buffer tensor, shape [N, 2]
# transform_mat 为 FP32 host buffer tensor, 板卡实测推荐按 [2, 3] 读取仿射矩阵。
ak.init()
ak.set_device([common.ES_AK_DEVICE_E.ES_AK_DEV_DIE0_CPU])

result = ak.cpu_similarity_transform(
    device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DIE0_CPU,
    src_point=source_points,    # 源点 [5, 2]
    dst_point=dest_points,     # 目标点 [5, 2]
    transform_mat=transform_mat # 输出 [2, 3]
```

```
)  
ak.deinit()
```

【参数说明】

| 参数            | 类型                 | 描述       | 约束规格                                             |
|---------------|--------------------|----------|--------------------------------------------------|
| device_id     | int                | 设备 ID 值  | 取值见<br>common.ES_AK_DEVICE_E<br>枚举               |
| src_point     | common.ES_TENSOR_S | 源点坐标张量   | 二维：[N, 2]<br>N 为点的数量，通常 N=5<br>每行为一个点的 (x, y) 坐标 |
| dst_point     | common.ES_TENSOR_S | 目标点坐标张量  | 维度与 src_point 一致                                 |
| transform_mat | common.ES_TENSOR_S | 输出变换矩阵张量 | 当前 Python 板卡实测推荐二维：<br>[2, 3]<br>相似变换矩阵的 6 个参数   |

【支持的数据类型】

输入 float32，输出 float32

【返回值】

int：0 表示成功

【异常】

参数超出范围时抛出 Exception 异常

【说明】

- 计算从源点集到目标点集的相似变换矩阵
- 相似变换包括旋转、缩放和平移，保持形状不变
- 通常用于人脸对齐等应用
- 输出的 [2, 3] 矩阵可用于仿射变换相关函数
- 当前 Python 绑定中 set\_hostBuf() 参数应为 memory.memory\_map() 返回的 capsule，不能直接传 int(arr.ctypes.data)。
- 使用 memory.memory\_map() 后应在不再使用时调用 memory.memory\_unmap(capsule, size)，并释放对应 fd。

4.3.5. 数据类型和数据结构

4.3.5.1. common.ES\_AK\_Capability\_S

【说明】

AcceleratorKit 能力信息结构。

【定义】

```
class common.ES_AK_Capability_S:  
    """AcceleratorKit 能力查询结果"""  
    reserved: int
```

【成员】



| 参数       | 类型  | 描述                |
|----------|-----|-------------------|
| reserved | int | 保留字段，当前板卡实测默认值为 0 |

【注】

ak.get\_capability() 返回 common.ES\_AK\_Capability\_S。当前 wheel 中调用该接口前需要导入 common 完成返回类型注册，推荐统一使用 from es\_py import ak, common。

4.3.5.2. common.ES\_AK\_DEVICE\_E

【说明】

支持的硬件加速器设备枚举。

【定义】

```
class common.ES_AK_DEVICE_E:
    """支持的硬件加速器"""
    # DSP 核心 (8 个)
    ES_AK_DEV_DSP_0 = 0
    ES_AK_DEV_DSP_1 = 1
    ES_AK_DEV_DSP_2 = 2
    ES_AK_DEV_DSP_3 = 3
    ES_AK_DEV_DSP_4 = 4
    ES_AK_DEV_DSP_5 = 5
    ES_AK_DEV_DSP_6 = 6
    ES_AK_DEV_DSP_7 = 7

    # HAE 核心 (2 个)
    ES_AK_DEV_HAE_0 = 8
    ES_AK_DEV_HAE_1 = 9

    # GPU 核心 (2 个)
    ES_AK_DEV_GPU_0 = 10
    ES_AK_DEV_GPU_1 = 11

    # CPU 核心 (2 个)
    ES_AK_DEV_DIE0_CPU = 12
    ES_AK_DEV_DIE1_CPU = 13

    ES_AK_DEV_BUTT = 14 # 最大设备数
```

【成员】

| 参数              | 值 | 描述      |
|-----------------|---|---------|
| ES_AK_DEV_DSP_0 | 0 | DSP0 设备 |
| ES_AK_DEV_DSP_1 | 1 | DSP1 设备 |
| ES_AK_DEV_DSP_2 | 2 | DSP2 设备 |
| ES_AK_DEV_DSP_3 | 3 | DSP3 设备 |
| ES_AK_DEV_DSP_4 | 4 | DSP4 设备 |
| ES_AK_DEV_DSP_5 | 5 | DSP5 设备 |
| ES_AK_DEV_DSP_6 | 6 | DSP6 设备 |
| ES_AK_DEV_DSP_7 | 7 | DSP7 设备 |

| 参数                 | 值  | 描述       |
|--------------------|----|----------|
| ES_AK_DEV_HAE_0    | 8  | HAE0 设备  |
| ES_AK_DEV_HAE_1    | 9  | HAE1 设备  |
| ES_AK_DEV_GPU_0    | 10 | GPU0 设备  |
| ES_AK_DEV_GPU_1    | 11 | GPU1 设备  |
| ES_AK_DEV_DIE0_CPU | 12 | DIE0 CPU |
| ES_AK_DEV_DIE1_CPU | 13 | DIE1 CPU |
| ES_AK_DEV_BUTT     | 14 | 最大设备数    |

4.3.5.3. common.ES\_TENSOR\_S

【说明】

Tensor 张量结构参数。

【定义】

```
class common.ES_TENSOR_S:
    """AcceleratorKit 张量描述"""
    dataType: common.ES_DATA_PRECISION_E # 数据类型
    shapeDim: int # 维度数量
    shape: List[int] # 各维度大小
    set_pData(buffer) # 设置设备侧数据缓冲
    get_pData() # 获取设备侧数据缓冲
    set_hostBuf(capsule) # 设置 host 侧 buffer，参数为 memory_map 返回的 capsule
    get_hostBuf() # 获取 host 侧 buffer 指针
```

【成员】

| 参数                   | 类型                         | 描述                                                    |
|----------------------|----------------------------|-------------------------------------------------------|
| dataType             | common.ES_DATA_PRECISION_E | Tensor 的数据类型，取值见数据类型枚举                                |
| shapeDim             | int                        | 描述 tensor 的真实维度个数                                     |
| shape                | List[int]                  | 按顺序存放 tensor 每个维度的宽度信息，最大维度数为 6                       |
| set_pData(buf)       | 方法                         | 设置设备侧数据缓冲，参数通常为 common.ES_DEV_BUF_                    |
| get_pData()          | 方法                         | 获取设备侧数据缓冲                                             |
| set_hostBuf(capsule) | 方法                         | 设置 host 侧 buffer，参数应为 memory.memory_map() 返回的 capsule |
| get_hostBuf()        | 方法                         | 获取 host 侧 buffer 指针                                   |

【注】

- DSP 设备侧算子通常使用 set\_pData(common.ES\_DEV\_BUF\_S) 传入设备缓冲区。
- CPU host 侧算子(如 ak.cpu\_similarity\_transform() )当前 Python 实测应使用 memory.memory\_map() 返回的 capsule 调用 set\_hostBuf(capsule) ；直接传 Python 整数地址会报参数类型不匹配。

4.3.5.4. 数据类型枚举

| 枚举值                 | 值  | 描述         |
|---------------------|----|------------|
| ES_PRECISION_INT8   | 1  | 有符号 8 位整数  |
| ES_PRECISION_UINT8  | 2  | 无符号 8 位整数  |
| ES_PRECISION_INT16  | 3  | 有符号 16 位整数 |
| ES_PRECISION_UINT16 | 4  | 无符号 16 位整数 |
| ES_PRECISION_INT32  | 5  | 有符号 32 位整数 |
| ES_PRECISION_UINT32 | 6  | 无符号 32 位整数 |
| ES_PRECISION_INT64  | 7  | 有符号 64 位整数 |
| ES_PRECISION_UINT64 | 8  | 无符号 64 位整数 |
| ES_PRECISION_FP16   | 9  | 16 位浮点数    |
| ES_PRECISION_FP32   | 10 | 32 位浮点数    |

4.3.5.5. common.ES\_DET\_NETWORK\_E

【说明】

detectionOut 算子支持的检测网络类型。当前 Python 绑定中的枚举成员为小写名称。

【成员】

| 枚举值                                | 描述                   |
|------------------------------------|----------------------|
| common.ES_DET_NETWORK_E.yolov3_u   | Ultralytics 版 YOLOv3 |
| common.ES_DET_NETWORK_E.yolov3_d   | Darknet 版 YOLOv3     |
| common.ES_DET_NETWORK_E.yolov4     | YOLOv4 检测网络          |
| common.ES_DET_NETWORK_E.yolov5     | YOLOv5 检测网络          |
| common.ES_DET_NETWORK_E.yolov7     | YOLOv7 检测网络          |
| common.ES_DET_NETWORK_E.yolov8     | YOLOv8 检测网络          |
| common.ES_DET_NETWORK_E.yolo5face  | YOLO5Face 人脸检测       |
| common.ES_DET_NETWORK_E.ppyoloe    | PaddlePaddle YOLOE   |
| common.ES_DET_NETWORK_E.damo_yolo  | DAMO-YOLO 检测网络       |
| common.ES_DET_NETWORK_E.yolov6     | YOLOv6 检测网络          |
| common.ES_DET_NETWORK_E.yolov9     | YOLOv9 检测网络          |
| common.ES_DET_NETWORK_E.yolov11    | YOLOv11 检测网络         |
| common.ES_DET_NETWORK_E.yolov8pose | YOLOv8 姿态估计          |
| common.ES_DET_NETWORK_E.ssd        | SSD 目标检测             |
| common.ES_DET_NETWORK_E.frcnn      | Faster R-CNN 检测      |

【注】

yolov3\_d 指的是 darknet 版，原始的 yolo v3 版本；yolov3\_u 指的是 ultralytics 版，是目前使用最为广泛的版本。用户根据自己模型的来源，自行选择对应的版本，如果不确定模型出处，建议首先尝试配置 common.ES\_DET\_NETWORK\_E.yolov3\_u 进行测试。

4.3.5.6. common.ES\_NMS\_METHOD\_E

【说明】

detectionOut 算子支持的 NMS 计算方法。

【定义】

```
class common.ES_NMS_METHOD_E:
    """NMS 方法枚举"""
    HARD_NMS = 0      # 硬 NMS
    SOFT_NMS_GAUSSIAN = 1 # 高斯软 NMS
    SOFT_NMS_LINEAR = 2 # 线性软 NMS
```

【成员】

| 参数                | 值 | 描述                     |
|-------------------|---|------------------------|
| HARD_NMS          | 0 | 硬 NMS，直接删除重叠框          |
| SOFT_NMS_GAUSSIAN | 1 | 高斯软 NMS，使用高斯函数降低重叠框置信度 |
| SOFT_NMS_LINEAR   | 2 | 线性软 NMS，使用线性函数降低重叠框置信度 |

4.3.5.7. common.ES\_IOU\_METHOD\_E

【说明】

detectionOut 算子支持的 IOU 方法。

【定义】

```
class common.ES_IOU_METHOD_E:
    """IOU 计算方法枚举"""
    IOU = 0 # 标准 IOU
    GIOU = 1 # Generalized IOU
    DIOU = 2 # Distance IOU
```

【成员】

| 参数   | 值 | 描述                              |
|------|---|---------------------------------|
| IOU  | 0 | 标准交并比 (Intersection over Union) |
| GIOU | 1 | 广义交并比 (Generalized IOU)         |
| DIOU | 2 | 距离交并比 (Distance IOU)            |

4.3.5.8. common.ES\_BOX\_TYPE\_E

【说明】

detectionOut 算子支持的输出 box 格式。

【成员】

| 枚举值                                   | 描述                                    |
|---------------------------------------|---------------------------------------|
| common.ES_BOX_TYPE_E.XminYminXmaxYmax | (xmin, ymin, xmax, ymax) 左上角+右下角坐标    |
| common.ES_BOX_TYPE_E.XminYminWH       | (xmin, ymin, w, h) 左上角坐标+宽高           |
| common.ES_BOX_TYPE_E.YminXminYmaxXmax | (ymin, xmin, ymax, xmax) 左上角+右下角，Y 优先 |
| common.ES_BOX_TYPE_E.XmidYmidWH       | (xmid, ymid, w, h) 中心点坐标+宽高           |

4.3.5.9. common.ES\_INTER\_FLAG\_E

【说明】

描述插值方式的枚举类型。

【定义】

```
class common.ES_INTER_FLAG_E:
    """插值方法枚举"""
    ES_INTER_NEAREST = 0    # 最近邻插值
    ES_INTER_LINEAR = 1     # 双线性插值
    ES_INTER_AREA = 2       # 区域插值
    ES_INTER_CUBIC = 3      # 双三次插值
    ES_INTER_LANCZOS4 = 4   # Lanczos 插值
    ES_INTER_NEAREST_EXACT = 6 # 精确最近邻
    ES_INTER_STRETCH = 20   # 拉伸插值
    ES_INTER_FILTER = 21    # 滤波插值
```

【成员】

| 参数                     | 值  | 描述              |
|------------------------|----|-----------------|
| ES_INTER_NEAREST       | 0  | 最近邻插值，速度最快      |
| ES_INTER_LINEAR        | 1  | 双线性插值，平衡质量和速度   |
| ES_INTER_AREA          | 2  | 区域插值，用于缩小图像     |
| ES_INTER_CUBIC         | 3  | 双三次插值，质量较高      |
| ES_INTER_LANCZOS4      | 4  | Lanczos 插值，质量最高 |
| ES_INTER_NEAREST_EXACT | 6  | 精确最近邻插值         |
| ES_INTER_STRETCH       | 20 | 拉伸插值            |
| ES_INTER_FILTER        | 21 | 滤波插值            |

4.3.5.10. common.ES\_BORDER\_TYPES\_E

【说明】

描述边界填充方式的枚举类型。

【定义】

```
class common.ES_BORDER_TYPES_E:
    """边界处理方式枚举"""
    BORDER_CONSTANT = 0    # 常量填充
    BORDER_REPLICATE = 1   # 复制边缘
    BORDER_REFLECT = 2     # 反射
```

```
BORDER_WRAP = 3      # 环绕
BORDER_REFLECT_101 = 4 # 反射 101
BORDER_TRANSPARENT = 5 # 透明
BORDER_ISOLATED = 16  # 隔离
```

【成员】

| 参数                 | 值  | 描述          |
|--------------------|----|-------------|
| BORDER_CONSTANT    | 0  | 用常量值填充边界    |
| BORDER_REPLICATE   | 1  | 复制最近的边缘像素   |
| BORDER_REFLECT     | 2  | 边缘反射：fedcba |
| BORDER_WRAP        | 3  | 环绕模式：cdefgh |
| BORDER_REFLECT_101 | 4  | 反射模式：gfedcb |
| BORDER_TRANSPARENT | 5  | 透明边界        |
| BORDER_ISOLATED    | 16 | 隔离边界处理      |

4.3.5.11. common.ES\_DETECTION\_OUT\_CFG

【说明】

定义检测输出参数配置的结构体。当前 Python 绑定字段使用 camelCase 名称。

【定义】

```
cfg = common.ES_DETECTION_OUT_CFG()
cfg.anchorsNum = 3
cfg.imgH = 640
cfg.imgW = 640
cfg.clsNum = 80
cfg.nmsMethod = common.ES_NMS_METHOD_E.HARD_NMS
cfg.iouMethod = common.ES_IOU_METHOD_E.IOU
cfg.outBoxType = common.ES_BOX_TYPE_E.XminYminXmaxYmax
cfg.coordNorm = common.ES_BOOL.ES_FALSE
cfg.maxBoxesPerClass = 100
cfg.maxBoxesPerBatch = 300
cfg.scoreThreshold = 0.25
cfg.iouThreshold = 0.45
cfg.softNmsSigma = 0.5
cfg.effecImgOffsetX = 0.0
cfg.effecImgOffsetY = 0.0
```

【成员】

| 参数          | 类型     | 描述                  |
|-------------|--------|---------------------|
| anchorsNum  | int    | anchor 数量           |
| anchorScale | 固定数组代理 | anchor 缩放参数         |
| imgH        | int    | 输入图像高度              |
| imgW        | int    | 输入图像宽度              |
| clsNum      | int    | 类别数量                |
| inputScale  | 固定数组代理 | 输入 feature scale 参数 |

| 参数               | 类型                     | 描述                                                                                                          |
|------------------|------------------------|-------------------------------------------------------------------------------------------------------------|
| nmsMethod        | common.ES_NMS_METHOD_E | NMS 方法                                                                                                      |
| iouMethod        | common.ES_IOU_METHOD_E | IOU 方法                                                                                                      |
| outBoxType       | common.ES_BOX_TYPE_E   | 输出 box 格式                                                                                                   |
| coordNorm        | common.ES_BOOL         | 标记边界框坐标是否归一化到 [0, 1]。当前绑定需使用 common.ES_BOOL.ES_FALSE 或 common.ES_BOOL.ES_TRUE；不接受 Python False/True 或整数 0/1 |
| maxBoxesPerClass | int                    | 每个类别经过 NMS 后保留的最大边界框数量                                                                                      |
| maxBoxesPerBatch | int                    | 整个批次经过 NMS 后保留的最大边界框数量                                                                                      |
| scoreThreshold   | float                  | 保留检测结果的最小分数阈值                                                                                               |
| iouThreshold     | float                  | 用于 NMS 中筛选重叠框的交并比阈值                                                                                         |
| softNmsSigma     | float                  | Soft NMS 中调整重叠框分数的 Sigma 值                                                                                  |
| effectImgOffsetX | float                  | 调整边界框坐标时，X 方向的有效偏移量                                                                                         |
| effectImgOffsetY | float                  | 调整边界框坐标时，Y 方向的有效偏移量                                                                                         |

#### 4.4. 错误码

| 错误码名称                          | 错误值        | 描述            |
|--------------------------------|------------|---------------|
| ES_AK_SUCCESS                  | 0x0        | 操作执行成功        |
| ES_AK_ERROR_NOMEM              | 0xA014600C | 分配内存失败        |
| ES_AK_ERROR_NOBUF              | 0xA014600D | 分配缓存失败        |
| ES_AK_ERROR_NOTREADY           | 0xA0146010 | 系统未初始化        |
| ES_AK_ERROR_BADADDR            | 0xA0146011 | 地址非法          |
| ES_AK_ERROR_BUSY               | 0xA0146012 | 系统忙           |
| ES_AK_ERROR_INVALID_DEVID      | 0xA0146029 | 设备 ID 超出合法范围  |
| ES_AK_ERROR_ILLEGAL_PARAM      | 0xA014602A | 非法参数          |
| ES_AK_ERROR_LOAD_OP_FAILED     | 0xA014602B | 加载算子失败        |
| ES_AK_ERROR_SUBMIT_TASK_FAILED | 0xA014602C | 提交任务失败        |
| ES_AK_ERROR_QUERY_FAILED       | 0xA014602D | Query 查询超时    |
| ES_AK_ERROR_NOT_SUPPORT        | 0xA014602E | 不支持的参数或者功能    |
| ES_AK_ERROR_OUTBUF             | 0xA014602F | 申请的 buffer 过多 |
| ES_AK_ERROR_GET_OP_FUNC_FAILED | 0xA0146030 | 获取算子执行函数失败    |
| ES_AK_ERROR_OP_EVAL_FAILED     | 0xA0146031 | 算子执行失败        |
| ES_AK_ERROR_OPEN_FILE_FAILED   | 0xA0146032 | 文件打开失败        |
| ES_AK_ERROR_READ_FILE_FAILED   | 0xA0146033 | 文件读取失败        |

| 错误码名称                          | 错误值        | 描述     |
|--------------------------------|------------|--------|
| ES_AK_ER-ROR_WRITE_FILE_FAILED | 0xA0146034 | 文件写入失败 |
| ES_AK_ERROR_INVALID_VER        | 0xA0146035 | 非法版本信息 |

## 4.5. 调试信息

### 4.5.1. 常见日志提示

- invalid vec size: `shape` 字段为空或者无法正常解析。
- The operator registration list is empty, check that compiler.a or manager.so is linked correctly: 算子列表为空，没有正确链接到 `compiler.a` 和 `manager.so`，需要在构建系统中链接这两个库文件。
- No candidate operators for `<operator_name>`: 调用算子时配置的参数不符合要求，未找到匹配的算子实现。
- Failed to emit operator description: 发射算子失败，可能是参数校验失败；如果是自定义算子，则可能是算子描述指针为空，或者 `param` 的 `buffer` 超出上限（32 个）。
- Failed to initialize buffers: `input`、`output` 或 `param` 的 `fd` 配置异常，请检查 `buffer` 数量和描述信息。
- Failed to load operator: 加载指定算子失败，请检查 DSP 算子库是否正确编译，或者检查 `emit` 发射的算子名是否符合要求。
- Failed to submit task for operator: 计算执行失败，请结合 `driver` 与 `framework` 侧日志进一步定位。

### 4.5.2. 日志级别

| 日志等级          | 值 | 输出信息     |
|---------------|---|----------|
| ES_LOG_ERR    | 3 | 仅错误日志    |
| ES_LOG_WARN   | 4 | 错误和警告    |
| ES_LOG_NOTICE | 5 | 错误、警告、提示 |
| ES_LOG_INFO   | 6 | 信息级别     |
| ES_LOG_DBG    | 7 | 调试级别     |

### 4.5.3. Python 调试示例

```
from es_py import ak

# 启用详细日志用于调试
ak.set_log_level(level=7)
```

## 4.6. 使用注意事项

### 4.6.1. 参数与 Tensor 配置

在调用 AcceleratorKit 接口前，需要先配置算子对应的 `config` 结构体作为入参，例如调用 `argmax` 算子前应提前做好对应结构体。

此外，`inputs`、`outputs` 的数据 `shape`、`type` 也需要提前做好配置，配置数据范围需满足接口要求，否则无法通过参数校验。



#### 4.6.2. 初始化和清理

```
from es_py import ak, common

try:
    # 1. 初始化 AK 模块
    ak.init()

    # 2. 设置日志级别
    ak.set_log_level(common.ES_AK_LOG_INFO)

    # 3. 配置设备
    devices = [common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0]
    ak.set_device(devices)

    # 4. 使用 AK 加速功能
    # ...

finally:
    # 5. 清理资源
    ak.deinit()
```

#### 4.6.3. 内存管理

- 所有输入/输出 tensor 必须预先分配内存。
- 张量的 `dataType` 和 `shape` 必须与输入数据匹配。
- 及时释放不需要的内存。

```
from es_py import common, memory

# 分配内存示例
fd = memory.memory_alloc(
    common.SYS_CACHE_MODE_E.SYS_CACHE_MODE_NOCACHE,
    "tensor_input",
    "mmz_nid_0_part_0",
    tensor_size
)

# 使用完毕后释放
memory.memory_free(fd)
```

#### 4.6.4. 设备选择

- 根据实际硬件可用性选择设备。
- 多个相同类型的设备可并行使用。
- 在中断使用后及时释放设备。

```
from es_py import common

# 查询可用设备并配置
devices = []
if hardware_supports_dsp:
    devices.append(common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0)
if hardware_supports_gpu:
    devices.append(common.ES_AK_DEVICE_E.ES_AK_DEV_GPU_0)

ak.set_device(devices)
```

#### 4.6.5. 错误处理

所有 AK 操作都可能抛出异常，需要正确处理：

```
from es_py import ak, common

try:
    ak.init()

    # 配置设备
    devices = [common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0]
    ak.set_device(devices)

    # 执行加速计算
    ak.dsp_warp_affine(
        device_id=common.ES_AK_DEVICE_E.ES_AK_DEV_DSP_0,
        input=input_tensor,
        output=output_tensor,
        m=transform_matrix,
        channel_first=0,
        flag=common.ES_INTER_FLAG_E.ES_INTER_LINEAR,
        border_mode=common.ES_BORDER_TYPES_E.BORDER_CONSTANT,
        border_value=0
    )

except Exception as e:
    print(f"AK 操作失败: {e}")
    error_msg = str(e)
    if "INVALID_DEVID" in error_msg:
        print("设备 ID 配置错误")
    elif "ILLEGAL_PARAM" in error_msg:
        print("参数配置错误")
    elif "OP_EVAL_FAILED" in error_msg:
        print("算子执行失败，请检查输入数据")

finally:
    ak.deinit()
```

#### 4.6.6. 性能优化建议

- 批量处理数据以提高吞吐量。
- 使用适当的插值方法平衡精度和性能。
- 充分利用多个 DSP 核心进行并行处理。
- 避免频繁的初始化/释放操作。